深圳天软科技

天软多因子框架 TSMultiFactor

应用专题-多因子 01

1 更新日志

1.1 更新日志

更新日期	更新说明
2026-03-09	1. 支持分组进行因子正交化，参考因子的附加属性，新增 OrthDataCore 正交化处理接口
2026-01-09	1. 修改，接口说明增加返回结果控制项 retType： GetInformationCoefficient、GetDecay_InformationCoefficient 2. 新增，支持每日 IC 衰减计算，见接口： GetDecay_IC、GetDecay_InformationCoefficientByICType、GetDecay_ICByICType 3. 新增数据处理相关配置说明
2025-11-20	1. 业务类型函数新增压缩去极值 MAD 法与 IC 类型
2025-11-19	1. 因子极值处理中增加不同极值处理方法差异对比，同时调整鲁棒标准差法说明 2. 调整 ICType 取值及说明 3. 调整 FFillMissType 参数选项说明
2025-11-13	1. 因子极值处理中增加“压缩去极值 MAD 法”算法说明并对相应的多因子框架属性参数 FAbnormalType 进行新增
2025-10-28	1. 调整因子缺失值处理中性值填充及市值回归填充算法说明内容
2025-10-22	1. FFillMissType 新增中性值填充和按行业中性值填充 2. 因子缺失值处理中新增中性值填充及市值回归填充算法说明
2025-10-14	1. 因子的附加属性增加“中性化处理方法”说明，支持指定因子参与中性化 2. FFillMissType 新增市值回归填充方法，并且增加按行业分组处理的缺失值处理方法
2025-09-28	1. 数据处理方法增加设置为 nil 的说明，具体参考 FFillMissType 等方法属性
2025-09-24	1. 调整因子延续性检验中 IC 计算方法一节中的说明 2. 新增因子延续性评价指标 IC 查询接口 GetInformationCoefficientbyICType 的说明
2025-09-19	1. 调整因子附加属性小节中标准化处理方法的说明
2025-09-12	1. 新增，提供接口 updateStockInfoData 用于当期股票基本信息的添加，统一控制市值（中性化市值、加权用市值、标准化用市值）和行业 2. 新增，数据处理过程中可通过接口 fvtmpdata_queryfactortable、fvtmpdata_querystocks、fvtmpdata_querystockinfodata 获取当期信息，方便自定义数据处理管道的重写 3. 优化，新增对于因子暴露、分组详情的查询接口 factorValue_QueryfactorfullTable、factorValue_QueryfactorGroupfullTable；支持定义查询内容（区分原始因子、描述因子、综合因子） 4. 新增，增加类型函数，方便参数设置 5. 废弃属性 FIndustryEval，中性化中的行业改由 updateStockInfoData 返回，统一由 IndustrializeData 控制。
2025-09-09	1. 优化文档结构

	2. 修改因子的附加属性说明并新增相应的使用场景说明，支持一套因子库中对不同因子用不同数据处理方法
2025-09-08	1. 修改数据处理相关接口的输入、输出以及范例 2. 调整文档结构
2025-09-05	1. 增加分组方式 FGroupType=3 表示权重分组法并调整相应的说明 2. 新增极值处理方法 FAbnormalType 参数选项 200~204，表示在行业内进行极值处理 3. 新增标准化处理方法 FStandarizeType 参数选项 201~204，表示在行业内进行标准化处理
2025-03-31	1. 修改行业分类 FIndustryType=-1 时需要重写的方法，同时增加 FIndustryType 作用场景 2. 增加 行业表达式 FIndustryEval 作用场景说明
2024-11-05	1. 新增极值处理方法：十分位法、鲁棒标准差 2. 属性 FAbnormalType 新增对应极值处理方法设置说明
2024-03-19	1. 修复链接跳转错误问题
2024-01-15	1. GetGroupReturn 参数 pType 增加选项 5，增加返回相对净值 2. GetTrailingReturn 增加参数 if_benchmark，可选返回“基准”组 3. 调整常见方法汇总表
2023-12-20	1. 优化部分说明
2023-08-04	1. 修改因子显著性检验和因子区分度检验的评价指标中关于 P-Value 的备注说明 2. 修改章节因子的附加属性，增加附加属性的备注说明 3. 增加章节单因子与多因子的处理差异
2023-05-06	1. 新增属性 FBMType、FBMDetail，支持复合基准 2. 新增方法 GetBenchmarkDailyReturn，可重写
2023-04-21	1. 增加值区间分组方式 FGroupType=2 2. Fgroups 支撑多种设置方式，方便使用： ● 百分位分组，支持 Fgroups 直接给分组数的使用方式 ● 个数分组，支持-1 表示最后一个
2022-07-13	1. 修正极差标准化算法说明
2022-04-25	1. 因子公式自定义公式参数，以及使用网格时系统参数的传递功能说明，新增 GetFactorParams 用于支持动态因子公式参数
2022-03-08	完善因子值获取接口的说明
2022-03-02	增加多因子框架可转债因子的使用范例
2021-11-29	完善自定义资金配比使用说明。
2021-04-22	（1） 分组分析增加组合换手率 （2） 增加因子信号稳定性分析 （3） 增加衰减分析：ic 衰减、稳定性衰减 （4） 延续性分析可选 IC：rankic、norm ic，并增加 p-value

2021-01-29	（1）新增 FBacktestType 属性，控制清算方式，支持可选：快速清算和模拟仿真清算 （2）升级 Backtest 成员方法，增加支持简单清算
2016-06-06	（1）添加 FindustryType=4 的行业分类：申万一级行业扩展，将非银金融细分成券商、保险、多元金融三个行业 （2）添加 AllocateWeightByGroup 方法，支持只对 T 期某组的个券比例分配重写。原有的 AllocateWeight 是对 T 期所有组数重写，因此重写 AllocateWeight 就不必重写 AllocateWeightByGroup，反之亦然。 （3）支持对回测变量设置：FpriceType、FvolModType、FdividendType、FallotmentType （4）支持落地服务器环境下使用公用多因子版本,但使用行业中性时仍需要自定义行业占比.
2015-11-30	（1）对适度因子处理方式加以说明 （2）对本地因子处理方式加以说明 （3）添加因子的两个不常用属性：是否去奇异值，是否标准化。特殊情况下才使用者两属性
2014-09-20	（1）分组时增加两种考虑行业的处理方法 （2）增加行业资金分配的行业中性处理方法 （3）添加两种评价方法： （a）最小二乘参数估计法（带常数项） （b）主成分回归法(带常数项)
2014-07-10	（1）极值处理方法增加： “四分位法”和”不做极值处理” （2）无量纲处理方法增加：“百分比排名打分法” （3）用户在使用多因子框架的时候，即可以使用天软提供的因子及其值，也可以使用自己提供的因子值。在此种应用场景下，需要重写 UserCalFactors()，详细范例见：TSFL_TSMutiFactor_02
2014-06-08	千呼万唤始发布，期待意见和后续的再完善和扩充

1.2 摘要

1. 推出了天软多因子框架 TSMultiFactor
2. 简要描述了如何使用天软多因子框架开发用户自己的多因子
3. 较为详细地描述了多因子开发的各个步骤，主要有：
   （1） 设置调仓周期
   （2） 设置初始样本
   （3） 设置因子名称、因子比例、因子方向、因子比例(%)
   （4） 因子缺失值处理
   （5） 因子极值处理
   （6） 因子无量纲处理
   （7） 中性化处理
   （8） 正交化处理
   （9） 因子分组
   （10） 因子资金分配

4. 较为详细地描述了多因子评价的常用方法。主要有：收益率检验、显著性检验、区分度检验、延续性检验
5. 提供了天软多因子框架的详细接口
6. 给出两个简要多因子范例，本范例只适合开题用。具体范例见：TSFL_TSMutiFactor_01
7. Email：TSMultiFactorTeam@tinysoft.com.cn

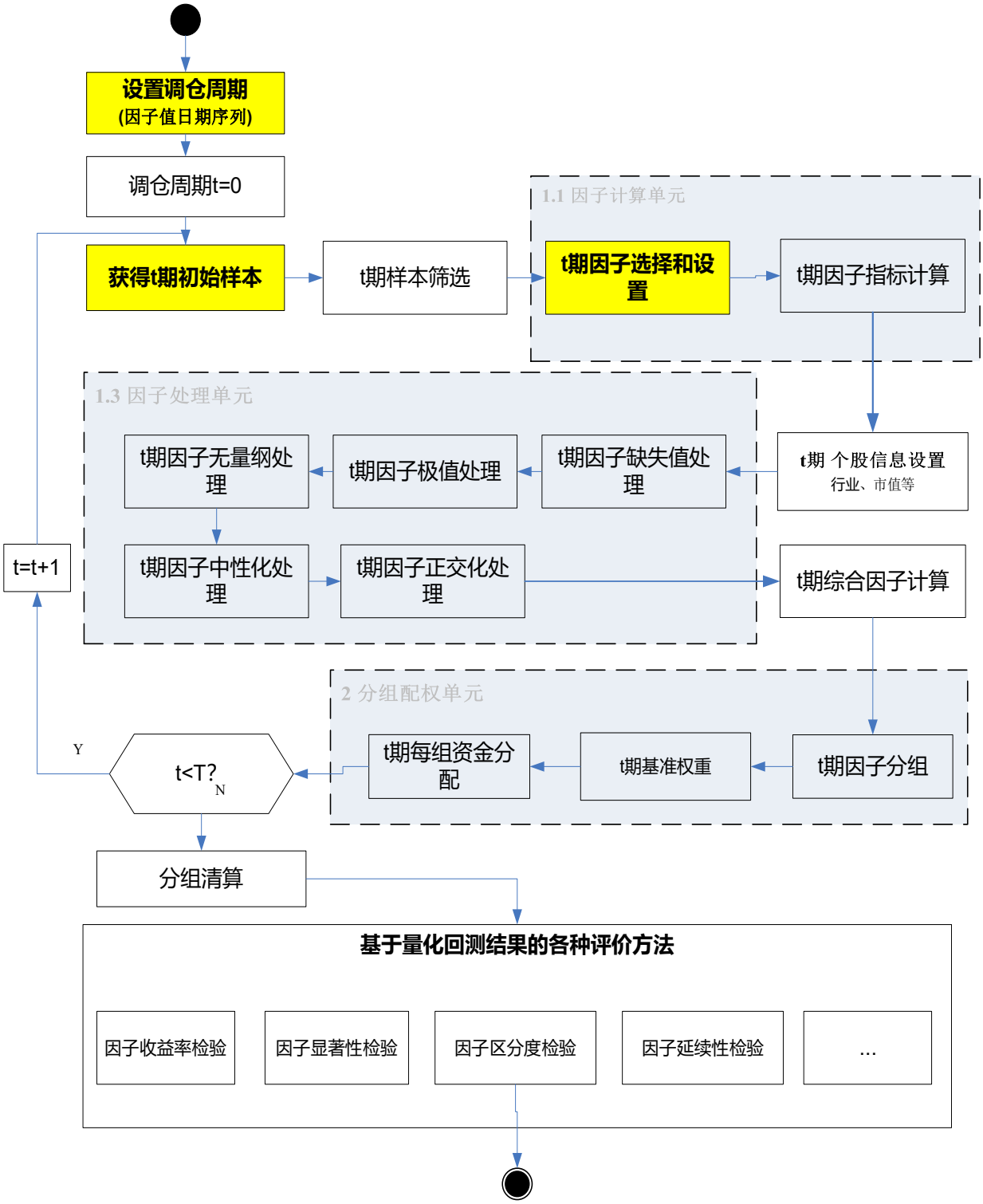
2 多因子总体流程

2.1 多因子框架是做什么的?

人们发现证券市场上，某些指标与证券的收益存在一定的相关性，比如盈利能力（如：ROE）高的股票往往之后的会有一个很好的收益。于是，我们想，能否将这类区分度良好的因子找出来？把这些因子放到一个因子库中，这样以后根据这个因子库选择股票的把握就会很大。这个因子分析的过程就是多因子框架所做的事。

2.2 天软因子处理流程

黄色部分和金色部分是用户重载或者需要重点考虑的部分，其他部分由天软多因子框架为您解决


设置调仓周期
(因子值日期序列)
调仓周期t=0
获得t期初始样本
t期样本筛选
1.1 因子计算单元
t期因子选择和设置
t期因子指标计算
t期 个股信息设置
行业、市值等
1.3 因子处理单元
t期因子缺失值处理
t期因子极值处理
t期因子无量纲处理
t期因子中性化处理
t期因子正交化处理
t期综合因子计算
t=t+1
2 分组配权单元
t期因子分组
t期基准权重
t期每组资金分配
t<T?
Y
N
分组清算
基于量化回测结果的各种评价方法
因子收益率检验
因子显著性检验
因子区分度检验
因子延续性检验
...

2.3 用户需要做什么?

提供调仓日期
提供每期调仓点的样本（每个调仓点样本可能不同）
提供每期调仓点的因子配置（所选择的因子、因子比例、因子方向）

2.4 天软多因子框架的特点

1) 完善的数据预处理方法：极值处理方法和无量纲化方法。
2) 天软量化回测
3) 基于回测结果和预处理后数据的多个层面的评价结果（收益率检验、显著性检验、区分度检验、延续性检验）

3 天软因子分析框架

3.1 获得调仓周期

3.1.1 什么是调仓周期?

故名思议，就是多长时间，对组合中的证券和证券比例进行调整。
对组合中的证券进行调整，实际上含有两层含义：
（1）对组合中的证券进行调入、调出操作
（2）对组合中的证券进行比例重新分配
根据策略的不同，调仓周期一般设置不同。
一般在进行因子研究的时候，周期不宜设置过短。
（1）对于只考虑财务类因子，调仓周期可以设置为一个季度，或者在一季报、中报、三季报、年报公布完成后的 4 月底、8 月底、10 月底
（2）对于考虑动量类因子和估值类，调仓周期一般设置为一个月
（3）一般而言，我们建议调仓周期为一个月

3.1.2 常见的调仓周期

序号	分类	调仓周期	说明
01	固定周期	每月	在因子分析中，多用一个月调仓一次
		每季度	一般仅限财务类因子
		每半年	少用
		每年	极少用

02	用户自定义调整周期		2008-12-31，2009-03-30

3.2 因子计算单元

3.2.1 设置初始样本

在每个调仓周期，根据研究对象的不同，因子研究的初始样本可以相同，也可以不同

（1）如果研究对象是全市场样本，初始样本可以相同（如：所有 A 股）

（2）如果研究对象本身就是随着时间调整的（如：常见的指数成分类），在每个调仓周期，在每个调仓周期，初始样本可能不同

3.2.1.1 常见的初始样本类型

序号	分类	说明	备注
01	市场板块	上证 A 股、深证 A 股、中小企业版、创业板等	
02	行业板块	申万行业、证监会行业、中证行业等	
03	指数成分	规模类：沪深 300、中证 800、上证 50 等 风格类：300 成长、300 价值等 主题类：中证红利、内地消费等 策略类：基本面 200、基本面 400 等 行业类：中证金融、中证信息等	
04	用户板块		
05	实际组合		

3.2.2 样本剔除

3.2.2.1 为什么要进行样本剔除?

在实际的量化投资过程中，投资者对于一些收益差或者曾经异常波动的证券往往采取回避的手段。同样的在因子分析中，为了和现实的投资过程接近，分析的因子特性往往也是针对于剔除了这些异常样本之后的。比如，我们常常剔除每股收益小于 0 的股票，或者有时候剔除金融类股票、ST 股票等等。

3.2.2.2 如何设置样本剔除条件?

多因子框架中设置剔除条件很简单，只需要设置成员变量 FConstr 即可。

3.2.2.2.1 一个筛选条件

如果你要对样本满足某个条件的股票进行删除(默认无删除条件)，比如你要删除每股收益小于等于 0 的股票，那么你只需要重设一个属性：

FConstr :=”Last12MData(RDate,9900000)<=0”;

注意到这里后面字符串的几个组成部分。

- 函数名称

函数名称可以是天软提供公用函数的名称，也可以是你自定义的函数的名称。比如上面的函数名称是 Last12MData 是一个公用函数的名称。

- 参数

参数***只能为***两种 RDate、EndT，而且写法也必须是这样，其中 EndT 就是指日期，RDate 是指 EndT 之前的最新报告期。因为无论是因子或是剔除条件，都是知道了一个调仓日之后采取判断的，那么 EndT 就是指这个调仓日，RDate 就是指离这个这个调仓日最近的报告期（按公布日）。

- 含义

我们对这个字符串是采取解析的形式，解析之后的值必须是 bool 类型的。我们一般的构造方式是一个表达式的然后和某个东西比较。

3.2.2.2.2 多个筛选条件

多个条件的筛选，肯定这其中各个条件之间存在着且还有或这种逻辑关系，***使用 and/or 连接各个条件。***

比如要剔除每股收益或者每股净资产小于等于 0 的股票，那么只要：

FConstr:= “Last12MData(RDate,9900000)<=0 or Last12MData(RDate,9900003)<=0”;

3.2.2.2.3 常见错误

因为剔除条件经常是要在前面加一个 not 的，所以用户在设置剔除条件的时候常常会犯的错误是搞不清该用 and 还是 or，故在此特举例说明。

范例：剔除上市不足 2 天，在调仓日是 PT 股，以及调仓日是 ST 股的股票。

用户常常会这么设置：FConstr:= “not IsStockGoMarket2(EndT-1) and not IsPT_3(EndT) and not IsST_3(EndT)”;这样设置是错误的。我们把上面设置的剔除条件说的通俗一点，就是，剔除***同时***满足下面三个条件的股票：

（1） 上市不超过两天。

（2） 不是 PT 股。

（3） 不是 ST 股。

要同时满足上面三个条件股票是没有的，所以这么设置用户并没有剔除任何股票。

其实要剔除的股票是要满足如下条件：

或者上市不超过两天，或者是 PT 股，或者是 ST 股。

故设置方法如下：

FConstr:= “not IsStockGoMarket2(EndT-1) or IsPT_3(EndT) or IsST_3(EndT)”;

所以，设置这个剔除条件的时候，特别要注意的一点就是：***你设置的需要剔除的股票所满足的条件***。

3.2.3 因子选择和设置

3.2.3.1 因子基本属性(FFactorArr)

这部分是用户使用因子框架的核心。不管单因子/多因子，在天软因子框架中，都必须配置因子名称、因子公式、因子方向、因子比例(%)这四个基本属性。

属性	类型	说明	备注
因子名称	Varchar	一个因子区别于其他因子的标示	
因子公式	Varchar	因子如何计算？可以是用天软中已有函数，也可以是用户自定义计算方法 一般用一个表达式来表示	
因子方向	Int	因子是正向还是反向？0 代表反向，1 代表正向	
因子比例(%)	Numberic	单因子的比例是 100%，多因子是各个因子分配的权重	

3.2.3.2 因子名称

- 一个因子区别于其他因子的标识；
- 因子的名称命名可以是一个英文名称，也可以是一个中文名称；
- 在多因子分析的时候，多个因子之间的名称不允许重复。

3.2.3.3 因子公式

即日常我们所说的指标/Factor。在因子分析中，这些指标一般都是与证券，或者证券所属的行业有关

一个因子公式至少包含以下两部分：

（1）因子/指标的模型名称

（2）因子/指标模型对应的参数

如：ReportofAll(9900000,20121231)表示 2012 年年报的每股收益。其中的 ReportofAll 是因子模型名称，9900000,20121231 是模型的参数

因子公式，在天软多因子框架中，本质是一个字符串。如：” Last12MData(RDate,9900000)”表示最近 12 个月的每股收益，”StockClose(EndT)”表示截止日的收盘价，”StockMarketValue(EndT)”代表截止日流通市值等等，内置的因子公式参数包含’Stockid’，’EndT’，’Rdate’，分别是证券代码、调仓日、报告期。

3.2.3.3.1 与日期和报告期有关的因子公式，需要用变量来表示

在做因子分析的时候，我们几乎不可能直接使用 ReportofAll(9900000,20121231)作为因子公式。为什么呢？因此，**因子分析是在不同调仓周期（或者是在不同日期）进行的。一切有关日期的内容，我们都是要需要用一个变量来表示**。如：

```
r:=array();
n:=0;
for nI:=0 to length(StockArr)-1 do
begin
  StockId:=StockArr[nI];
  SetSysParam(PN_Stock(),StockId);
     //系统当前时间为 EndT
  SetSysParam(pn_Date(),EndT);
```

```
    //与 EndT 对应的报告期是 RDate
  RDate:=NewReportDateOFEndT2(EndT);
  if not IsStockGoMarket(EndT-1) then continue;
  r[n]['代码']:=StockId;
  r[n]['名称']:=StockName(StockId);
  r[n]['日期']:=EndT;
    //与 RDate 对应的最近 12 个月 EPS
  r[n]['EPS-TTM']:=Last12MData(RDate,9900000);
    //与 EndT 对应的最近 1 月涨幅
  r[n]['Return']:=StockZf(IncMonth(EndT,-1),EndT);
  n++;
end;
return r;
```

其中的 EndT&RDate，**就是在因子分析中，常用的两个变量**。这两个变量，一个与调仓日有关，一个与报告期有关（本质也是与日期有关）。有关报告期与日期之间关系的详情，可参阅：http://www.tinysoft.com.cn/tsdn/helpdoc/index.tsl?itemid=10986

而这两个变量的值，会在天软因子框架中提供。对于用户而言，可以直接在因子公式中，包含这两个变量

3.2.3.3.2 与报告期有关的因子公式

在因子分析中，常见与报告期有关的因子有：

（1）盈利类因子（如：ROE）

（2）偿债类因子（如：资产负债率）

（3）成长类因子（如：营业收入增长率）

（4）经营能力类因子（如：存货周转率）

（5）分红和投资收益类因子（如：股息支付率）

（6）现金流类因子（如：销售现金比率）

（7）机构持股类因子（如：第 1 大股东持股比例(%)）

这些常用因子有个共性的地方，都是与证券的财务报告期有关。在目前国内上市公司披露准则下，类似的数据是与一季报、中报、三季报、年报等相关的

在天软多因子框架中，类似的财务类因子中，至少都必须包含有报告期参数，且在天软因子公式定义中必须命名为 RDate

3.2.3.3.2.1 与基本面有关的因子范例（部分）

天软因子公式规范	基础模型	因子公式	说明
财务类因子，至少都必须包含有报告期参数，且必须命名为 RDate	报告期数据 ReportofAll(InfoId,RDate)	ReportofAll(9900000,RDate)	取 RDate 报告期对应的数据
	季度数据 LastQuarterData(RDate,InfoId,0)	LastQuarterData(RDate,9900000,0)	取 RDate 报告期对应的季度数据

	最近 12 个月数据 Last12MData(RDate,InfoId)	Last12MData(RDate,9900000)	取截止 RDate 报告期对应的最近 12 个月数据
	股东持股	StockMSHPercentage1(RDate,0)	
	股东户数	StockSHAmount(RDate)	

说明：上述几个模型，都与报告期有关。因此在因子公式中

（1） 都必须至少包含 RDate 参数。

（2） 每个模型其他参数，可以根据实际情况进行增减（如：ReportofAll 中的 9900000，StockMSHPercentage1 中的 0），这些参数一般都是一些常量

3.2.3.3.3 与日期有关的因子公式

在因子分析中，常见与日期有关的因子有：

（1）动量类因子（如：最近 20 日涨幅）

（2）规模类因子（如：指定日流通股本、流通市值）

（3）风险类因子（如：最近 1 年 Beta）

这些与日期有关的因子，从日期的角度，分为两类：

序号	日期类别	常见描述	范例
01	仅与截止日有关	指定日	指定日流通市值
02	与开始日和截止日有关	（1）最近 1 个月、最近半年、最近 1 年 （2）年初以来	最近 20 日涨幅 用最近 1 年数据计算的 Beta 系数

与报告期有关的因子公式不同之处，与日期有关的因子公式定义稍微复杂一些。

3.2.3.3.3.1 与截止日有关，且模型参数中包含截止日

天软因子公式规范	因子公式	说明
模型中如果包含截止日，在天软因子公式定义中必须命名为 EndT	StockMarketValue(EndT)	指定日流通市值
	StockTotalShares(EndT)	指定日总股本

对应的 StockMarketValue(EndT)模型是这样开发的：

```
Function StockMarketValue(EndT);
Begin
   oV:=BackUpSystemParameters();
   SetSysParam(pn_rate(),0);
  //EndT 是模型参数
   NShares:=StockNegotiableShares(EndT);
    //EndT 是模型参数
   C:=StockClose(EndT);
   RestoreSystemParameters(oV);
   return NShares*C/10000;
End;
```

3.2.3.3.3.2 与截止日有关，且模型参数中不包含截止日

天软因子公式规范	因子公式	说明
模型中可以不包含截止日 EndT。 对于此类情况，EndT 是由天软因子框架在计算因子值时设置。此时，截止日 EndT 是个系统参数 关于系统参数，请参见： http://www.tinysoft.com.cn/tsdn/helpdoc/display.tsl?id=1862	StockMarketValue2(10)	最新 10 日平均流通市值
	StockMarketValue3()	最新流通市值

StockMarketValue2 (N)模型是这样开发的：

```
Function StockMarketValue2(N);
Begin
        //截止日是个系统参数.是由天软因子框架，在计算因子数值时设置的
   t:=Nday3(n,StockMarketValue3());
   if isTable(t) then
      return mean(t)
   else
      return real(0);
End;
```

StockMarketValue3()模型是这样开发的：

```
Function StockMarketValue3();
Begin
        //截止日是个系统参数.是由天软因子框架，在计算因子数值时设置的
   return StockMarketValue(sp_time());
End;
```

3.2.3.3.3.3 与开始日和截止日有关，且模型参数中包含开始日和截止日

天软因子公式规范	因子公式	说明
（1）模型中如果包含截止日，在天软因子公式定义中必须命名为 EndT （2）开始日可以按照日历日，也可以按照交易日来度量	StockZf(IncDay(EndT,-30)+1,EndT)	最近 1 个月涨幅（日历日）
	StockZf(StockEndTPrevNDay(EndT,20-1),EndT)	最近 20 个交易日涨幅（交易日）

而 StockZf(BegT,EndT)是这样开发的：

```
Function StockZf(BegT,EndT);
Begin
   oV:=BackUpSystemParameters();
   SetSysParam(PN_Cycle(),cy_day());
   SetSysParam(PN_Rate(),1);
   SetSysParam(PN_RateDay(),EndT);
        //开始日和截止日是模型参数
   bC:=specdate(Close(),BegT-1)
   eC:=specdate(Close(),EndT);
   RestoreSystemParameters(oV);
   return(eC/bC-1)*100.
```

```
end;
```

从中我们可以看到，在因子框架中，开始日可以按照日历日来度量，也可以按照交易日来度量。到底选择哪种方案，见仁见智。因为证券存在停牌等，**我们一般推荐采取交易日来度量开始日**

按日历日来度量开始日，常用的的模型：

序号	天软日期函数	含义	开始日范例
01	IncDay(AValue,ANumberOfDays)	天数推移	30 天前：IncDay(EndT,-30)+1
02	IncWeek(AValue,ANumberOfWeeks)	周推移	一周前：IncWeek(EndT,-1)+1
03	IncMonth(DateTime,NumberOfMonths)	月推移	一月前：IncMonth(EndT,-1)+1
04	IncQuarter(AValue,Number)	季度推移	一个季度前：IncQuarter(EndT,-1) +1
05	IncHalfYear(AValue,Number)	半年推移	半年前：IncHalfYear(EndT,-1)+1
06	IncYear(AValue,ANumberOfYears)	年推移	一年前：IncYear(EndT,-1)+1
07	StartOfTheWeek(EndT)	本周	StartOfTheWeek(EndT)
08	StartOfTheMonth(EndT)	本月	StartOfTheMonth(EndT)
09	StartOfTheYear(EndT)	本年	StartOfTheYear(EndT)

按交易日度量开始日，常用的的模型：

序号	天软日期函数	含义		开始日范例
01	StockEndTPrevNDay(EndT,N)	向前推移 N 个交易日	最近 5 个交易日	StockEndTPrevNDay(EndT,5-1)
			最近 20 个交易日	StockEndTPrevNDay(EndT,10-1)
			最近 60 个交易日	StockEndTPrevNDay(EndT,60-1)
			最近 120 个交易日	StockEndTPrevNDay(EndT,120-1)
			最近 240 个交易日	StockEndTPrevNDay(EndT,240-1)

3.2.3.3.3.4 与日期有关的因子公式总结

从上面的描述我们可以看到，与日期有关的因子公式每种方法都有各自的优缺点。假设我们要计算 N 日涨幅，从模型开发的角度，我们有下面几种写法

写法 1：模型参数中包含截止日

```
Function MyZF1(EndT,N);
Begin
   oV:=BackUpSystemParameters();
   SetSysParam(PN_Cycle(),cy_day());
   //模型中包含 EndT，关于截止日的处理，必须在模型中进行
   SetSysParam(PN_Date(),EndT);
   SetSysParam(PN_Rate(),1);
   bC:=ref(Close(),N);
   eC:=Close();
   v:=(eC/bC-1)*100;
   RestoreSystemParameters(oV);
```

```
    return v;
End;
```

写法 2：模型参数中不包含截止日

```
Function MyZF2(N);
Begin
    oV:=BackUpSystemParameters();
    SetSysParam(PN_Cycle(),cy_day());
    SetSysParam(PN_Rate(),1);
    //截止日是个系统参数.是由天软因子框架，在计算因子数值时设置的
    bC:=ref(Close(),N);
    eC:=Close();
    v:=(eC/bC-1)*100;
    RestoreSystemParameters(oV);
    return v;
End;
```

写法 3：模型参数中包含开始日和截止日

```
Function MyZf3(BegT,EndT);
Begin
    oV:=BackUpSystemParameters();
    SetSysParam(PN_Cycle(),cy_day());
    SetSysParam(PN_Rate(),1);
    //开始日和截止日是模型参数
    bC:=specdate(Close(),BegT-1)
    eC:=specdate(Close(),EndT);
    RestoreSystemParameters(oV);
    return(eC/bC-1)*100.
end;
```

从结果上而言，这三个模型在计算最近 5 个交易日涨幅的时候，结果肯定是相同的。但是在因子分析配置中，复杂程度不一致

因子公式规范	因子公式	应用场合推荐
与截止日有关，且模型参数中包含截止日	MyZF1(EndT,5)	如果仅与截止日有关，此种写法最常见
与截止日有关，且模型参数中不包含截止日	MyZF2(5)	在因子分析中，即使用到开始日，一般都用类似最近 N 个交易日来描述（如：20 日涨幅）
与开始日和截止日有关，且模型参数中包含开始日和截止日	MyZf3(StockEndTPrevNDay(EndT,5-1),EndT)	如果因子有涉及到开始日，此方法公式最复杂，一般用在有明确的开始日期（如：年初以来）

3.2.3.4 因子方向

因子方向，即因子的排序方向。常见的排序方向有：

因子方向	含义	说明
1	正序	因子的值越大越好
0	逆序	因子的值越小越好

在实际应用过程中，单个因子即可以作为正序，也可以作为逆序。如流通市值因子：

（1）如果设置为正序，即希望市值越大越好。按照此因子值和因子方向排序，排在前面的都是大市值股票

（2）如果设置为逆序，即希望市值越小越好。按照此因子值和因子方向排序，排在前面的都是小市值股票

不同因子可以在因子方向设置不同值，但是对于解读这个因子的人，必须对此很清楚。否则，就可能出现整体性的偏差

3.2.3.4.1 因子方向设置可能导致的误导性

对于多因子分析的时候，如果多个因子的因子方向不一致，可能对整体性解读因子，造成间歇性的短路。我们看一个实例：

因子	因子方向	说明
市值	逆序	规模越小越好
收益	正序	收益越高越好
综合因子	综合得分最好的证券，是小盘蓝筹	

把市值因子的因子方向换一下，看一下效果：

因子	因子方向	说明
市值	正序	规模越大越好
收益	正序	收益越高越好
综合因子	综合得分最好的证券，是大盘蓝筹	

还是那句话，因子方向是正序、逆序无所谓，但是解决每个因子的综合得分的人很关键。

3.2.3.4.2 类似于 P/E、P/B、P/SR、P/CF、资产负债率等通常意义上的逆序因子的特殊处理

一般而言，P/E、P/B、P/SR、P/CF、资产负债率等因子，通常都认为越小越好（即：因子方向为逆序）。如果按照纯粹数学上的排序规则，以下几个证券 2013-11-30P/E 排名如下：

代码	名称	P/E	EPS-TTM	收盘	排名
SZ000061	农 产 品	-3,554.18	-0.0024	8.50	1
SZ000878	云南铜业	-8.75	-1.0235	8.96	2
SH600096	云天化	-7.73	-1.1576	8.95	3
SH600048	保利地产	6.88	1.2930	8.90	4
SZ000002	万 科A	7.08	1.2377	8.76	5

SH600660	福耀玻璃	10.10	0.8507	8.59	6
SH600655	豫园商城	10.69	0.8139	8.70	7
SH600403	大有能源	14.12	0.5956	8.41	8
SH600348	阳泉煤业	14.78	0.5798	8.57	9
SH601139	深圳燃气	26.17	0.3236	8.47	10

数学上的最小值（SZ000061 农 产 品-3,554.18）并不是我们通常意义上估值最好的因子（我们希望估值越低越好）。我们暂且称此类指标为**适度因子**，适度因子指趋向于某个值评价就越好，它既不是最大好也不是最小评价就越好，因子在处理适度因子的时候，都需要将因子趋势化，即将因子指标转变成正向因子(底层处理时反向因子都要转化成正向因子)。

在适度因子转化成正向因子的时候，不同的因子有不同变换公式，例如 P/E 因子是越小越好(>0)，倒数法可将因子理想地转化成正向因子，但是如资产负债率因子如果也采取取倒数法，就变成负债率越小越好，从业务上是不符合逻辑的。

因子适度因子需要用户自定义公式，即在定义因子函数时转化，常见的转换方法如下：

（1）倒数法：$x' = \frac{1}{x}$,将原值取倒数，如处理 P/E 因子

（2）绝对值法：$x' = |x - k|, k$ 指评价最好的值，如处理资产负债率

下图为 P/E 因子经变换后，新的因子 E/P（正序）和结果如下：

代码	名称	P/E	EPS-TTM	收盘	P/E 排名（纯数学上）	E/P	排名
SH600048	保利地产	6.88	1.2930	8.90	4	0.1453	1
SZ000002	万 科A	7.08	1.2377	8.76	5	0.1413	2
SH600660	福耀玻璃	10.10	0.8507	8.59	6	0.0990	3
SH600655	豫园商城	10.69	0.8139	8.70	7	0.0935	4
SH600403	大有能源	14.12	0.5956	8.41	8	0.0708	5
SH600348	阳泉煤业	14.78	0.5798	8.57	9	0.0677	6
SH601139	深圳燃气	26.17	0.3236	8.47	10	0.0382	7
SZ000061	农 产 品	-3,554.18	-0.0024	8.50	1	-0.0003	8
SZ000878	云南铜业	-8.75	-1.0235	8.96	2	-0.1142	9
SH600096	云天化	-7.73	-1.1576	8.95	3	-0.1293	10

3.2.3.5 因子比例(%)

3.2.3.5.1 单因子的比例

对于单因子，因子比例必须等于 100

3.2.3.5.2 多因子的比例

对于多因子，各个单因子比例的和必须等于 100

对于多因子中，各个单因子的比例如何分配，已经超过本文档的范畴

标准的处理流程应该是，先对每个单因子进行深入研究后，结合投资者的经验和收益风险偏好，按照自己的目标，设定不同单因子的比例。多因子的各个比例设置本质是个艺术范畴，不是个技术问题

3.2.3.5.3 单因子与多因子的处理差异

单因子实际上就是多因子的一个特例（只有一个因子，比例等于 100），最终单因子数据处理后的因子值，因为比例等于 100，可以认为就是综合因子得分；但是如果因子不止一个，综合因子得分就是各个因子数据处理后的因子值根据因子比例进行加权得到的和。

3.2.3.5.4 用户习惯的先大类因子配置，再子类因子配置如何转换为标准因子配置格式

通常意义上，在多因子配置的时候，用户习惯上先配置大类因子比例，再配置子类因子比例。如：

大类因子	大类比例(%)	因子名称	因子公式	因子方向	子类占大类比例(%)
估值因子	60	E/P	1/StockPE_VI(EndT)	1	80
		B/P	1/StockPNA_VI(EndT)	1	20
				小计	100
盈利因子	40	ROE	Last12MData(RDate,9900100)	1	90
		ROA	Last12MData(RDate,9900102)	1	10
合计	100			小计	100

对于此类配置模式，天软因子框架不可能识别诸如此类的格式。

用户首先需要把非标的配置格式，转换为天软因子框架可识别的格式，即：

因子比例(%)=大类比例(%)*子类占大类比例(%)

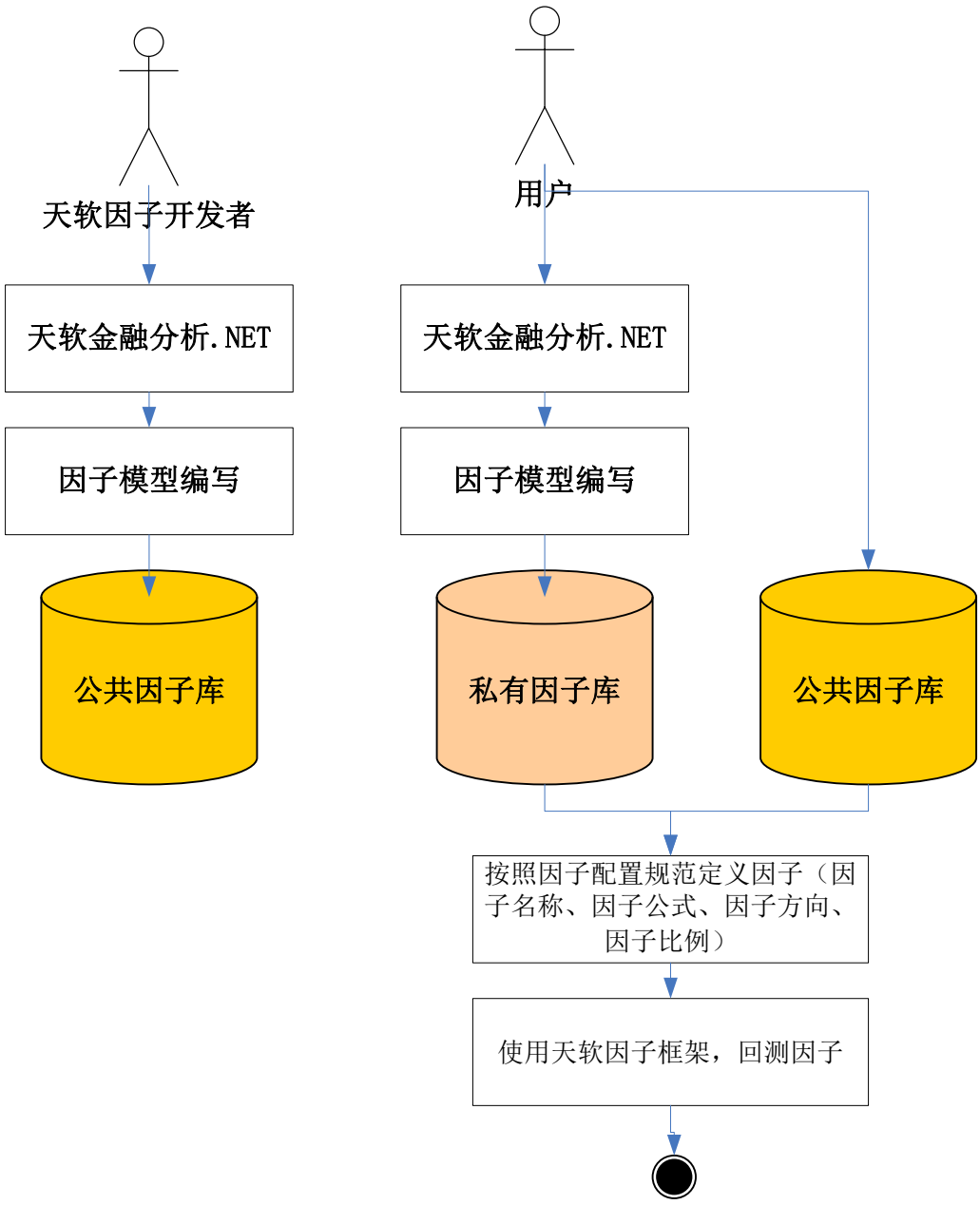
因子名称	因子公式	因子方向	因子比例(%)	说明
E/P	1/StockPE_VI(EndT)	1	48	60%*80%
B/P	1/StockPNA_VI(EndT)	1	12	60%*20%
ROE	Last12MData(RDate,9900100)	1	36	40%*90%
ROA	Last12MData(RDate,9900102)	1	4	40%*10%
		合计	100	

3.2.3.6 用户如何扩展自己的因子?

3.2.3.6.1 使用天软内含因子&扩展自己的因子

用户在使用天软因子分析框架的时候，可以使用天软系统本身提供的因子，也可以按照自己研究的需要，使用天软金融分析.NET 编写自己的因子。

因子编写完成后，按照因子配置规范定义因子（因子名称、因子公式、因子方向、因子比例），使用天软因子框架，回测自己的因子。



3.2.3.6.2 扩展自己的因子（范例）

在天软金融分析.NET 中扩展和开发自己的因子

```
//MyPEPB 是用户使用 TSL 扩展的因子
Function MyPEPB(EndT);
Begin
   vPE:=StockPE_VI(EndT);
   vPB:=StockPNA_VI(EndT);
   if vPE<0 or vPB<0 then
      return -abs(vPE*vPB)
   else
      return vPE*vPB;
```

```
End;
```

3.2.3.6.3 配置自己的因子（范例）

按照天软因子配置规范定义因子（因子名称、因子公式、因子方向、因子比例）

因子名称	因子公式	因子方向	因子比例(%)	说明
E/P*B/P	1/MyPEPB(EndT)	1	50	用户自定义因子 取倒后，正向因子
ROE	Last12MData(RDate,9900100)	1	40	天软内含因子
ROA	Last12MData(RDate,9900102)	1	10	天软内含因子
		合计	100	

3.2.3.7 常用天软内含因子及其配置示例

在目前天软金融分析.NET 上，已经为用户提供了多层次、多种类因子，在此不一一列举。下表我们列举了一些常用因子、因子公式、因子方向

大类因子	因子名称	因子公式	因子方向	因子描述
盈利因子	净资产收益率(%)ROE	Last12MData(RDate,9900100)	1	净利润/平均净资产
	总资产报酬率(%)ROA	Last12MData(RDate,9900105)	1	利润总额/平均总资产
	销售毛利率(%)	Last12MData(RDate,9900103)	1	(营业收入－营业成本)/营业收入
	销售净利率(%)	Last12MData(RDate,9900102)	1	净利润/营业收入
成长因子	营业收入增长率(%)	Last12MData(RDate,9900600)	1	
	营业利润增长率(%)	Last12MData(RDate,9900602)	1	
	利润总额增长率(%)	Last12MData(RDate,9900603)	1	
	归属于母公司净利润增长(%)	Last12MData(RDate,9900604)	1	
经营能力	存货周转率	Last12MData(RDate,9900400)	1	营业成本/平均存货
	应收账款周转率	Last12MData(RDate,9900404)	1	营业成本/平均应收账款
	总资产周转率	Last12MData(RDate,9900416)	1	营业成本/平均总资产
负债因子	流动比率	Last12MData(RDate,9900200)	1	流动资产/流动负债
	速动比率	Last12MData(RDate,9900201)	1	(流动资产-存货)/流动负债
	资产负债率(%)	1/Last12MData(RDate,9900203)	1	总负债/总资产
规模因子	总市值	Ln(StockTotalValue(EndT))	1	
	流通市值	Ln(StockMarketValue(EndT))	1	

	总股本	Ln(StockTotalShares(EndT))	1	
	流通股本	Ln(StockHasNAShares(EndT))	1	
	总资产	Last12MData(RDate, 44059)	1	
估值因子	市盈率(TTM)	1/StockPE_VI(EndT)	1	归属净利润/总市值
	市净率(TTM)	1/StockPNA_VI(EndT)	1	归属净资产/总市值
	市销率(TTM)	1/StockPMI_VI(EndT)	1	营业收入/总市值
	市现率(TTM)	1/StockPCF_VI(EndT)	1	经营现金净额/总市值
动量因子	最近 1 个月涨幅(%)	StockZf2(20)	1	
	最近 3 个月涨幅(%)	StockZf2(60)	1	
	最近 6 个月涨幅(%)	StockZf2(120)	1	
	最近 12 个月涨幅(%)	StockZf2(240)	1	
其他因子	收盘价	StockClose(EndT)	1	
	最近 1 个月日均换手率	StockAveHsl2(10)	1	
	最近 3 个月日均换手率	StockAveHsl2(60)	1	
风险因子	标准差(%)	StockRisk(“SH00300”,IncYear(EndT,-1),EndT)	0	用最近 1 年数据，计算标准差
	Beta	StockBeta(“SH000300”, IncYear(EndT,-1),EndT)	0	用最近 1 年数据，计算 Beta

注：除了此因子列表之后，可以访问 BasicInfoExtId_SourceCode 函数获取天软所有财务因子，技术因子可参考路径：公用函数—>拓展函数—>.net 拓展函数→技术分析。

3.3 因子处理单元

3.3.1 因子缺失值处理（FFillMissType）

根据因子公式计算的因子值可能是 nan 值等非正常数值或甚至是非数值，需要对这些数据进行处理，避免影响因子的接下来的处理和分析。

3.3.1.1 常见的缺失值处理方法

3.3.1.1.1 不处理

保留缺失值，一般很少用到该选项，只用来观察原始数据。

3.3.1.1.2 剔除

直接将缺失值数据剔除，只要是某个个券的某个因子值为缺失值，就会把该个券剔除，不会再对该个券进行分析。

3.3.1.1.3 平均值填充

将缺失值以当期的所有的个券的对应因子值的平均值填充。

3.3.1.1.4 中位数填充

将缺失值以当期的所有的个券的对应因子值的中位数填充。

3.3.1.1.5 中性值填充

采用中性值填充时，会先跳过缺失值，对非缺失值按照相应的极值处理方法与无量纲化处理方法进行数据处理，完成无量纲化处理后再对缺失值进行中性值填充，且填充用的中性值与选择的无量纲化方法相关。

1. 当无量纲化处理为极差正规化法时，对应的填充用中性值为 0.5；
2. 当无量纲化处理为百分比打分法时，对应的填充用中性值为 50%；
3. 当无量纲化处理为其他方法时（Z 值法、极差标准化法等），对应的填充用中性值为 0。

3.3.1.1.6 市值回归填充

将非缺失值序列作为因变量 Y，对应的对数流通市值（已进行极值与标准化处理后）序列作为自变量 X，对两个序列进行回归，获得相应的回归方程

$$Y = \beta_0 + \beta_1 X$$

再将缺失值对应的对数流通市值代入回归方程中推算出缺失值，将其作为填充值进行填充。

3.3.2 因子极值处理（FAbnormalType）

3.3.2.1 为什么要进行极值处理?

因子分析的过程是一个由样本分析系统的过程，样本有它的局限性：1、样本数有限，通过有限的样本去分析系统的性质肯定会存在一些差异。2、样本中可能存在小概率事件，小概率事件会影响到对整体的分析。比如我们分析小明的学习情况，他考了 10 次试，其中有 9 次都是六十多分，1 次是九十多分，那么这唯一一次的九十多分分可能就是一个低概率事件（可能小明考 100 次才能考中一次），那么这个样本就会对我们的分析产生影响。

异常数据的判断和剔除是数据处理中必不可少的一环。尤其是金融数据处理中，不进行极值处理，会对数据最终的分析造成很大的误差和误判

3.3.2.2 常见的因子极值处理方法

3.3.2.2.1 中位数去极值法（MAD_3.5std）

（1）任何一个序列 $X_i(1 <= i <= N)$

（2）计算 X 序列的中位数：$M_{1/2} = Median(X_i)$

（3）计算原序列与中位数偏差的绝对值：$Z_i = abs(X_i - M_{1/2})$

（4）计算偏差绝对值序列的中位数：序列 Z_i 的中位数为 $ZMid = Median(Z_i)$

（5）取极值后的序列为：

如果 $X_i <= M_{1/2} - 5.2 * ZMid$，则 $X_i = M_{1/2} - 5.2 * ZMid$；

如果 $X_i >= M_{1/2} + 5.2 * ZMid$，则 $X_i = M_{1/2} + 5.2 * ZMid$

3.3.2.2.2 3 倍标准差法

（1） 任何一个序列，其值为 $X_i(1 <= i <= N)$

（2） 此序列的均值 $\mu = \frac{\sum_{i=1}^{N} X_i}{N}$

（3） 此序列的标准差 $\sigma = \sqrt{\frac{\sum_{i=1}^{N}(X_i - \mu)}{N-1}}$

（4） 极值处理后的序列为：

如果 $X_i <= \mu - 3 * \sigma$，则 $X_i = \mu - 3 * \sigma$

如果 $X_i >= \mu + 3 * \sigma$，则 $X_i = \mu + 3 * \sigma$

3.3.2.2.3 四分位法

（1）计算 X 序列的中位数：$M_{1/2} = Median(X_i)$

（2）计算 X 序列的 1/4 分位的值 $M_{1/4}$ 和 3/4 分位的值 $M_{3/4}$（升序）

（3） $gap1 = M_{3/4} - M_{1/2}$，$gap2 = M_{1/2} - M_{1/4}$

（4）极值处理后的序列为：

如果 $X_i < M_{1/4} - 1.5 * gap2$，则 $X_i = M_{1/4} - 1.5 * gap2$

如果 $X_i > M_{3/4} + 1.5 * gap1$，则 $X_i = M_{3/4} + 1.5 * gap1$

3.3.2.2.4 十分位法

（1） 计算 X 序列的 1/10 分位的值 $M_{1/10}$ 和 9/10 分位的值 $M_{9/10}$（升序）

（2） $gap1 = M_{1/10}$，$gap2 = M_{9/10}$

（3）极值处理后的序列为：

如果 $X_i < gap1$，则 $X_i = gap1$

如果 $X_i > gap2$，则 $X_i = gap2$

3.3.2.2.5 鲁棒标准差法

本节鲁棒标准差法适用于以 0 为中心随机波动的数据序列，如因子模型横截面回归后的残差序列，如果是对未知中心数据序列进行极值处理，不建议使用本方法，可使用中位数去极值法或压缩去极值 MAD 法。

（1） 计算 X 序列的中位数：$M_{1/2} = Median(X_i)$

（2） 计算 X 序列与中位数的差的绝对值序列X_{ad}

（3） 计算X_{ad}序列的中位数M_{ad}

（4） $gap1 = 4 \times 1.4826 \times M_{ad}$，$gap2 = 4 \times 1.4826 \times M_{ad}$

（5） 极值处理后的序列为：

如果$X_i < gap1$，则$X_i = gap1$

如果$X_i > gap2$，则$X_i = gap2$

3.3.2.2.6 压缩去极值 MAD 法（MAD_3std）

（1） 计算 X 序列的中位数：$M_{1/2} = Median(X_i)$

（2） 计算 X 序列与中位数的差的绝对值序列$X_{ad_i} = abs(X_i - M_{1/2})$

（3） 计算X_{ad}序列的中位数$M_{ad_1/2} = Median(X_{ad_i})$

（4） 计算上下界：

下界：$gap1 = M_{1/2} - 3 \times 1.4826 \times M_{ad_1/2}$

上界：$gap2 = M_{1/2} + 3 \times 1.4826 \times M_{ad_1/2}$

（5） 压缩极端值：

i. 下极端值

对于 X 序列中小于下界值的部分从小到大排序，将排序后的值均匀映射到区间$[gap1 - 0.5 * 1.4826 * M_{ad_1/2}, gap1]$，即

$$新值 = gap1 - \left(1 - \frac{当前值的排序位置 - 1}{总个数 - 1}\right) \times \left(0.5 \times 1.4826 \times M_{ad_1/2}\right)$$

ii. 上极端值

对于 X 序列中大于上界值的部分从小到大排序，将排序后的值均匀映射到区间$[gap2，gap2 + 0.5 * 1.4826 * M_{ad_1/2}]$，即

$$新值 = gap2 + \left(\frac{当前值的排序位置 - 1}{总个数 - 1}\right) \times \left(0.5 \times 1.4826 \times M_{ad_1/2}\right)$$

（6） 极值处理后的序列为：

如果$X_i < gap1$，则X_i按照下极端值压缩方式进行处理

如果$X_i > gap2$，则X_i按照上极端值压缩方式进行处理

如果$gap1 \le X_i \le gap2$，则X_i保留原值

3.3.2.2.7 不同极值处理方法间的差异

方法名称	边界计算方式	目标	处理方式	备注
中位数去极值	$M_{1/2} \pm 5.2 * ZMid$	将数据压缩至±3.5 倍标准差内（不受极值影响）	截断	

3 倍标准差	$\mu \pm 3\sigma$	将数据压缩至±*3 倍标准差内（受极值影响）*	截断	将异常值压缩到一个点（边界值） 上
四分位法	$M_{1/4} - 1.5 * gap2$ $M_{3/4} + 1.5 * gap1$	将数据压缩至上下界内	截断	
十分位法	$M_{1/10}$，$M_{9/10}$，	将数据压缩至 10%分位点与 90%分位点区间	截断	
压缩去极值 MAD 法	$M_{1/2}$ $\pm 3 \times 1.4826 \times M_{ad}$	将数据压缩至±*3 倍标准差内（不受极值影响）*	非截断，压缩映射	将异常值按其原始顺序映射到边界外的一个狭窄区间内，保留了异常值之间的相对顺序关系

3.3.2.2.8 为什么提供以上极值处理方法?

抨击别人是证明自己优势的最好方法，先来一个反例。

- ***反例:***

1、先对样本序列做 zscore 标准化。

2、然后对小于-5 的样本赋值为-5，对大于 5 的赋值为 5。

- ***错误点 1:***

zscore 无量纲化法必须在极值处理之后做。否则可能出现（1）之后进行的极值处理又可能导致量纲不统一；（2）在未进行极值处理前进行无量纲化，这个伸缩平移的过程会受到奇异值的影响。所以，***必须先进行极值处理，然后再进行无量纲化。***

- ***错误点 2:***

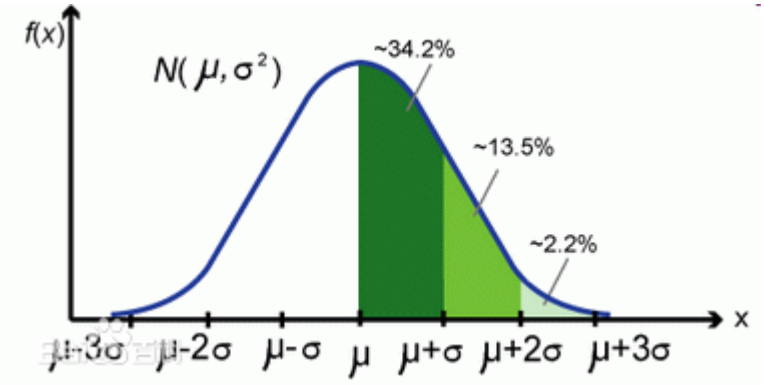
±5 的边界设置不合理。首先，第一步进行的 zscore 标准化之后，我认为你处理之后的样本服从 N(0,1)的正态分布，那么这样一个正态分布发生一个在±5 之外的值的概率究竟是多少呢？答案是 $2.867\times10^{-5}\%$，如此低的概率，可以说用上面的方法基本上处理不到任何样本。

注：可通过如下代码测试

```
x:=array(-10,0.02)->10;
sf_normcdf(x,0,1,y);
r:=`x|`y;
return r;
```

- ***三倍标准差法为什么好？为什么是 3 倍标准差，不是 2 倍，也不是 4 倍。***

看下面的正态分布的图，



我们看到，函数曲线下 68.27%的面积在一个标准差σ范围内，95.45%的面积在两个标准差2σ的范围内，99.73%的面积在三个标准差3σ的范围内.99.99%的面积在四个标准差4σ的范围内。如果使用 2 倍标准差，那么概率高达 4.55%的样本会被当成小概率事件而被处理掉，也就是说大概 5 次出现 1 次的样本会被当成奇异值，显然是不合理的。如果使用 4 倍标准差，大概是万分之零点几的概率的样本才会被处理到。考虑到实际过程中样本数的大小，4 倍标准差几乎处理不到任何样本。3 倍标准差之外发生的概率大概是千分之二到三，而现实情况中的样本也是几十个到一两千个，此时把 3 倍标准差之外的样本当成是小概率事件的极值处理是比较合理的。

3.3.2.2.9 不进行极值处理

在实务操作中，不对极值进行处理应用场合比较少，一般要和其他方法一起配合使用。一般情况下，不对极值处理需要在无量纲化处理中选择“百分比打分法”

3.3.3 因子无量纲化处理（FStandarizeType）

3.3.3.1 为什么要进行因子无量纲处理?

在多因子分析的时候，最终一般都会采取打分法确定综合因子。而每个单因子所代表的含义和数量级不尽相同。

要对多组不同量纲数据进行比较，需要先将它们标准化转化成无量纲的标准化数据

3.3.3.2 常见的因子无量纲处理方法

假设证券 i 的第 j 个因子值为$X_{i,j}$

其中：$1<=i<=N$，N 为证券的个数

$1<=j<=M$，M 为因子的个数

3.3.3.2.1 标准化法（Z 值法，默认方法，最常用）

证券 i 因子 j 在所有样本中的 Z 值为$Z_{i,j}=\frac{X_{i,j}-\mu_j}{\sigma_j}$

其中：

μ_j：第 j 个因子的均值

σ_j：第 j 个因子的标准差

3.3.3.2.2 极差标准化

将数据按伸缩平移变换映射到[-1,1]，消除量纲。

$$Z_{i,j} = 2 \times \frac{X_{i,j} - Min_j}{Max_j - Min_j} - 1$$

3.3.3.2.3 极差正规化

将数据按伸缩平移变换映射到[0,1],消除量纲。

$$Z_{i,j} = \frac{X_{i,j} - Min_j}{Max_j - Min_j}$$

3.3.3.2.4 百分比打分法

对各个因子采用百分比排名的方式打分。一般而言，百分比打分后的分数分布在 0-100 之间

说明：

（1） 采用此种打分方式的目的是为了消除标准差非常大的而差距没有那么大的序列的影响。

（2） 对于同名的采用向上打分，和 excel 中的 percentrank 向下打分略有不同。这种差异仅仅会在处理同名时体现。

3.3.3.2.5 重写无量纲化方法特别注意：

如果因子是反向因子，在进行无量纲化的时候，要根据无量纲化方法，采取不同的方法。如：常见几种无量纲方法处理正向、反向因子计算方法如下：

无量纲方法	正向因子	反向因子
Z 值法	$Z_{i,j} = \frac{X_{i,j} - \mu_j}{\sigma_j}$	$Z_{i,j} = -\frac{X_{i,j} - \mu_j}{\sigma_j}$
极差标准化	$Z_{i,j} = 2 \times \frac{X_{i,j} - Min_j}{Max_j - Min_j} - 1$	$Z_{i,j} = 1 - 2 \times \frac{X_{i,j} - Min_j}{Max_j - Min_j}$
极差正规化	$Z_{i,j} = \frac{X_{i,j} - Min_j}{Max_j - Min_j}$	$Z_{i,j} = 1 - \frac{X_{i,j} - Min_j}{Max_j - Min_j}$
百分比打分法	通过该因子大小在整个股票中的百分位作为打分 正向因子，最大为 100，最小为 0 反向因子，最大为 0，最小为 100	

3.3.4 因子中性化处理（FNeuType）

在对单因子进行有效性分析时，为了更加有效地验证该因子的有效性，需要剔除常见的因子，例如行业、市值因子等，跟所研究因子的相关性。该过程就是中性化处理。常用的方法是就是原始因子值作为因变量，需要中性化的因子作为自变量，进行回归，取残差作为新的因子值。

框架支持的内置中性化包括行业中性化、对数市值中性化。其中行业中性化支持自定义行业表达式。

3.3.4.1 中性化处理方法原理

采用**多元线性回归残差法**：将行业设为虚拟变量，市值为连续变量，共同作为自变量对原始因子值回归，残差即为中性化后的因子值。（如果不进行某个中性化处理，则没有对应的自变量项）

设：

- i 为股票，k 为行业总数，构造 k−1 个行业虚拟变量 $D_{i1}, D_{i2}, \ldots, D_{i(k-1)}$（避免多重共线性）
- $ln(MktCap_i)$ 为股票 i 的对数市值
- $\beta 0$ 为截距项，$\beta 1, \beta 2, \ldots, \beta k-1$ 为行业虚拟变量系数，βm 为市值系数
- F_i^{Neu} 为最终中性化因子值

多元回归方程：

$$F_i = \beta_0 + \sum_{j=1}^{k-1} \beta_j D_{ij} + \beta_m ln(MktCap_i) + \varepsilon_i$$

中性化因子值（残差）：

$$F_i^{Neu} = \varepsilon_i = F_i - \widehat{\beta_0} - \sum_{j=1}^{k-1} \widehat{\beta_j} D_{ij} - \widehat{\beta_m} ln(MktCap_i)$$

3.3.5 因子正交化处理（FOrthType）

在做因子合成时，各因子之间的相关性是存在的，因子正交化做的就是剔除因子间的相关性。

其本质上是对原始因子（通过一系列线性变换）进行旋转，旋转后得到一组两两正交的新因子，它们之间的相关性为零并且对于收益的解释度（即整体的方差）保持不变。

N 表示股票池的个券数量，K 表示因子数量

$f^k = [f_1^k, f_2^k, f_3^k, \ldots, f_n^k]'$表示全部个券在第 k 个因子上的取值

对每个因子进行归一化后得到$F_{N*K} = [f^1, f^2, f^3, \ldots, f^k]$

即求一个过渡矩阵 S_{K*K} 使正交矩阵 $\tilde{F}_{N*K} = F_{N*K} * S_{K*K}$

经过一系列推导：

$$S_{K*K} = U_{K*K} D_{k*k}^{-1/2} U'_{k*k} C_{K*K}$$

其中：

U_{K*K} 和 D_{K*K} 为 F_{N*K} 的重叠矩阵 M_{K*K} 的**特征向量矩阵**和**特征根对角矩阵**

$M_{K*K} = (N-1)\varepsilon_{k*k}$ （ε_{k*k} 为 F_{N*K} 的**协方差矩阵**）

$$D_{k*k}^{-1/2} = \begin{bmatrix} 1/\sqrt{\lambda_1} & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & 1/\sqrt{\lambda_K} \end{bmatrix}$$

C_{K*K} 为任意正交矩阵

3.3.5.1 常见的正交化处理方法

3.3.5.1.1 施密特正交

给定一组向量集合，按照给定因子的顺序将每个向量与之前的所有因子值向量进行正交并归一化。

从一组线性无关的因子列向量$f^1,f^2,f^3,\dots,f^k$，我们可以逐步构造一组正交的向量组$\tilde{f}^1,\tilde{f}^2,f^3,\dots,\tilde{f}^k$，正交后的向量为：

$$\tilde{f}^1 = f^1$$
$$\tilde{f}^2 = f^2 - \frac{\langle f^2,\tilde{f}^1\rangle}{\langle \tilde{f}^1,\tilde{f}^1\rangle}\tilde{f}^1$$
$$\tilde{f}^3 = f^3 - \frac{\langle f^3,\tilde{f}^1\rangle}{\langle \tilde{f}^1,\tilde{f}^1\rangle}\tilde{f}^1 - \frac{\langle f^3,\tilde{f}^2\rangle}{\langle \tilde{f}^2,\tilde{f}^2\rangle}\tilde{f}^2$$
$$\dots = \dots\ \dots\ \dots$$
$$\tilde{f}^k = f^k - \frac{\langle f^k,\tilde{f}^1\rangle}{\langle \tilde{f}^1,\tilde{f}^1\rangle}\tilde{f}^1 - \frac{\langle f^k,\tilde{f}^2\rangle}{\langle \tilde{f}^2,\tilde{f}^2\rangle}\tilde{f}^2 - \dots - \frac{\langle f^k,\tilde{f}^{k-1}\rangle}{\langle \tilde{f}^{k-1},\tilde{f}^{k-1}\rangle}\tilde{f}^{k-1}$$

由此可以得到一组相互正交的向量组$\tilde{f}^1,\tilde{f}^2,f^3,\dots,\tilde{f}^k$，再对它们单位化后

$$e^i = \frac{\tilde{f}^i}{\|\tilde{f}^i\|}(i = 1,2,\dots,k)$$

3.3.5.1.2 规范正交

规范正交选取了特殊的正交矩阵$C_{K*K} = U_{K*K}$，即过度矩阵是

$$S_{K*K} = U_{K*K}D_{k*k}^{-1/2}U'_{k*k}U_{K*K}$$
$$= U_{K*K}D_{k*k}^{-1/2}$$

正交后的因子的第一主成分由方差最大的方向确定，不同截面上进行规范正交后的因子没有稳定的对应关系。

3.3.5.1.3 对称正交

对称正交选取特殊的正交矩阵$C_{K*K} = I_{K*K}$，I_{K*K}是单位矩阵，即有

$$S_{K*K} = U_{K*K}D_{k*k}^{-1/2}U'_{k*k}I_{K*K}$$
$$= U_{K*K}D_{k*k}^{-1/2}U'_{k*k}$$

S_{K*K} 是一个对称矩阵，对称正交名称由此得来。

主要的性质：

1. 相对于施密特正交，对称正交不需要提供正交次序，对每个因子平等看待
2. 在所有的正交过渡矩阵，对称正交后的矩阵与原矩阵的相似性最大，即正交前后的矩阵距离最小。变化前后的矩阵的距离用 Frobenius 范数φ来衡量因子正交前后变化的大小

$$\varphi = \sum_{I=1}^{K}\|f^i - \tilde{f}^i\|^2$$

可以证明，对称正交的过度矩阵是φ最小的唯一解[Aiken 1980]。

3.4 分组配权单元

3.4.1 因子分组

3.4.1.1 为什么要对因子进行分组?

在做因子量化的时候，一般要确认如下内容：

- 因子是否有效。即：按照因子，对样本分组，不同组别之间的样本，收益率是否有显著性差异。
- 因子的相关性如何？
- 因子是否有稳定的超额收益？
- 因子是否对未来有解释作用？解释能力如何？

基于此，经常需要按照因子的值，对样本进行分组。

3.4.1.2 常见因子分组方法

序号	方法	备注
01	不分组	多用在仅仅条件筛选的量化模型中
02	三分位	前 1/3，中间 1/3，后 1/3
03	五分位（模型默认分类方法）	0%-20%、20%-40%、40%-60%、60%-80%、80%-100%
04	十分位	
05	用户自定义（用户修改成员变量）	"0,20;20,40;40,60;60,80;80,100";

3.4.1.3 天软多因子框架中的分组方法

进入分组单元时，股票已按照综合因子值，从大到小排序。

3.4.1.3.1 个数分组

- 当为按***个数***分组时
- 参见 FGroupType 中取值为 0 部分叙述。

3.4.1.3.2 (个股数)百分位分组

- 当按百分比分组时，按股票个数百分比分组，如 ：

FGroupType:=1;

FGroups:="0,20;20,40;40,60;60,80;80,100";具体是如何分组的呢？

对于***起点***的百分位 a 我们取到对应的第几个的取法是：x=int(a*len)+1;

对于***末点***的百分位 b 我们取到对应的第几个的取法是：y=int(b*len);

那么每段就是[x,y]，从 1 开始。

- 参见 FGroupType 中取值为 2

3.4.1.3.3 值区间分组

- 当按值区间分组时，按因子值的范围分组，比如 ROE>10、ROE<10 的部分进行分组。参见 FGroupType 中取值为 2 部分叙述。

3.4.1.3.4 权重分组

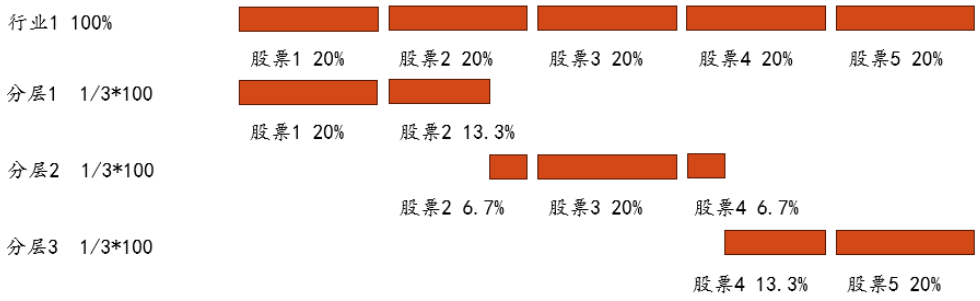
- FGroupType=3 时，表示按个股累计权重分组。以考虑行业因素分组，且分 3 组为例：

在行业 1 中，共有 5 只股票，此处为了方便说明，我们给每只股票等权重赋权，即每只股票权重为 20%，需要分为 3 组，则每组的累计权重阈值为 1/3*100=33.3%。按照因子值排序后，对个股权重进行累计：

（1） 在组 1 中，股票 1 的权重（20%）+股票 2 的权重（13.3%）满足累计权重阈值，故第 1 组中包含股票 1 和股票 2；

（2） 在组 2 中，股票 2 的权重（6.7%）+股票 3 的权重（20%）+股票 4 的权重（6.7%）满足累计权重阈值，故第 2 组中包含股票 2、股票 3 和股票 4；

（3） 在组 3 中，股票 4 的权重（13.3%）+股票 5 的权重（20%）满足累计权重阈值，故第 3 组中包含股票 4 和股票 5



组别	代码	权重	累计权重	行业比例(%)	比例(%)
1	股票1	0.2	0.2	25	5
1	股票2	0.133	0.333		3.333
2	股票2	0.067	0.4		1.667
2	股票3	0.2	0.6		5
2	股票4	0.067	0.667		1.667
3	股票4	0.133	0.8		3.333
3	股票5	0.2	1.0		5

3.4.1.3.5 几个分组特例

3.4.1.3.5.1 值等宽法

值区间分组法的特例，等宽分组法，目标是每个组的因子值区间一样。

3.4.1.3.5.2 个数等分法

百分位分组的特例，按个股数等分 5 组，等频分组法，目标是每个组包含的个股数一样

3.4.1.3.5.3 权重等分法

权重分组法的特例，按个股累计市值等分 5 组

3.4.1.4 分组时考虑行业因素（FGroupHy）

一些因子的值是和行业相关的。如：金融行业的市值一般比较大，而电子行业的市值一般比较小。如果在分组的时候，不考虑行业因素，可能会集中把一些行业分到一个组中，从而使得组间的行业分布不均匀

我们看个范例。假设收盘因子为正向因子，不进行极值处理、标准化方法采取百分比打分法后的综合因子和排名如下：

代码	名称	收盘	奇异处理后	标准化后	综合因子	行业名称	排名

SH600010	包钢股份	5.40	5.40	27.20	27.20	申万钢铁	9
SH600019	宝钢股份	4.89	4.89	18.10	18.10	申万钢铁	10
SH600030	中信证券	13.36	13.36	45.40	45.40	申万金融服务	7
SH600036	招商银行	13.75	13.75	54.50	54.50	申万金融服务	6
SH600068	葛洲坝	5.49	5.49	36.30	36.30	申万建筑建材	8
SH600585	海螺水泥	18.45	18.45	72.70	72.70	申万建筑建材	4
SH601166	兴业银行	16.69	16.69	63.60	63.60	申万金融服务	5
SH601318	中国平安	45.29	45.29	100.00	100.00	申万金融服务	1
SH601601	中国太保	22.50	22.50	90.90	90.90	申万金融服务	2
SH601628	中国人寿	21.40	21.40	81.80	81.80	申万金融服务	3
SH601668	中国建筑	3.90	3.90	9.00	9.00	申万建筑建材	11
SH601669	中国电建	3.82	3.82	0.00	0.00	申万建筑建材	12

3.4.1.4.1 不考虑行业因素的分组

不考虑行业因素的分组本质是按照综合因子的得分，得到综合因子在所有样本中的排名，然后再分组

如果不考虑行业因素，把所有股票分成三组后的结果为：

代码	名称	收盘	奇异处理后	标准化后	综合因子	行业名称	排名	分组
SH601318	中国平安	45.29	45.29	100.00	100.00	申万金融服务	1	1
SH601601	中国太保	22.50	22.50	90.90	90.90	申万金融服务	2	1
SH601628	中国人寿	21.40	21.40	81.80	81.80	申万金融服务	3	1
SH600585	海螺水泥	18.45	18.45	72.70	72.70	申万建筑建材	4	1
SH601166	兴业银行	16.69	16.69	63.60	63.60	申万金融服务	5	2
SH600036	招商银行	13.75	13.75	54.50	54.50	申万金融服务	6	2
SH600030	中信证券	13.36	13.36	45.40	45.40	申万金融服务	7	2
SH600068	葛洲坝	5.49	5.49	36.30	36.30	申万建筑建材	8	2
SH600010	包钢股份	5.40	5.40	27.20	27.20	申万钢铁	9	3
SH600019	宝钢股份	4.89	4.89	18.10	18.10	申万钢铁	10	3
SH601668	中国建筑	3.90	3.90	9.00	9.00	申万建筑建材	11	3
SH601669	中国电建	3.82	3.82	0.00	0.00	申万建筑建材	12	3

各组中每个行业分布为：

分组	申万金融服务	申万建筑建材	申万钢铁	小计

1	3	1	0	4
2	3	1	0	4
3	0	2	2	4
小计	6	4	2	12

从上表可看到，因为申万金融服务大多属于高价股，而申万钢铁大多属于低价股。不考虑行业因素后，申万金融服务分配到了第 1、2 组，申万钢铁全部分配到了第 3 组。因为没有考虑行业因素，各个组别之间行业分布不均匀

做多因子和分组的目的，是为了检验分组后各个组别之间的收益率是否有显著差异。如果组间收益率有显著差异，说明此因子可以把表现好的股票和表现差的股票分离出来。不考虑行业因素后，各个组别的收益率差异可能也有显著性差异，但是这些显著性差异可能不是由因子贡献的，而可能是行业分布不均匀贡献的。组间的收益率可能是由于行业配置造成的，不是我们做多因子的初衷

因此在分组的时候，我们建议：**考虑行业因素的影响**

3.4.1.4.2 考虑行业因素分组

分组时考虑行业因素，本质是分行业按照综合因子的得分，得到综合因子在行业内的排名，然后再分组。

（1）以（个股数）百分位分组为例

考虑行业因素，把所有股票分成三组后的结果为：

代码	名称	收盘	奇异处理后	标准化后	综合因子	行业名称	行业内排名	分组
SH600585	海螺水泥	18.45	18.45	72.70	72.70	申万建筑建材	1	1
SH601318	中国平安	45.29	45.29	100.00	100.00	申万金融服务	1	1
SH601601	中国太保	22.50	22.50	90.90	90.90	申万金融服务	2	1
SH600010	包钢股份	5.40	5.40	27.20	27.20	申万钢铁	1	2
SH600068	葛洲坝	5.49	5.49	36.30	36.30	申万建筑建材	2	2
SH601628	中国人寿	21.40	21.40	81.80	81.80	申万金融服务	3	2
SH601166	兴业银行	16.69	16.69	63.60	63.60	申万金融服务	4	2
SH600019	宝钢股份	4.89	4.89	18.10	18.10	申万钢铁	2	3
SH600036	招商银行	13.75	13.75	54.50	54.50	申万金融服务	5	3
SH600030	中信证券	13.36	13.36	45.40	45.40	申万金融服务	6	3
SH601668	中国建筑	3.90	3.90	9.00	9.00	申万建筑建材	3	3
SH601669	中国电建	3.82	3.82	0.00	0.00	申万建筑建材	4	3

考虑行业因素分组后，各组每个行业分布为：

分组	申万金融服务	申万建筑建材	申万钢铁	小计
组 1	2	1	0	3
组 2	2	1	1	4
组 3	2	2	1	5
小计	6	4	2	12

与不考虑行业因素分组相比：

（1） 申万金融服务行业被平均分配到三个组

（2） 申万钢铁本身只有两个样本，因此只能分配到两组中

从上表看到，考虑了行业后的分组，组间行业分布已经比不考虑行业因素均匀多了

3.4.1.4.3 考虑行业因素后行业样本个数小于分组数的处理方法

（1）（个股数）百分位分组专用

考虑行业因素后，如果行业样本个数小于分组个数，也存在在某个组别不存在行业的情况（如：申万钢铁只有两个样本，只能分配到两个组别中）

为了让每个组别都有行业的指定样本，如果这个行业中的股票数小于分组数，那么不对这个行业分组，而是让每个组中都包含这个行业的股票；如果行业中样本个数大于分组数时，正常的对这个行业中的股票分组

按此方法处理后，所有股票分成三组后的结果为：

代码	名称	收盘	奇异处理后	标准化后	综合因子	行业名称	行业内排名	分组
SH600010	包钢股份	5.40	5.40	45.40	45.40	申万钢铁	1	1
SH600019	宝钢股份	4.89	4.89	18.10	18.10	申万钢铁	2	1
SH600585	海螺水泥	18.45	18.45	81.80	81.80	申万建筑建材	1	1
SH601318	中国平安	45.29	45.29	100.00	100.00	申万金融服务	1	1
SH601601	中国太保	22.50	22.50	90.90	90.90	申万金融服务	2	1
SH600010	包钢股份	5.40	5.40	27.20	27.20	申万钢铁	1	2
SH600019	宝钢股份	4.89	4.89	18.10	18.10	申万钢铁	2	2
SH600068	葛洲坝	5.49	5.49	72.70	72.70	申万建筑建材	2	2
SH601628	中国人寿	21.40	21.40	88.40	88.40	申万金融服务	3	2
SH601166	兴业银行	16.69	16.69	80.50	80.50	申万金融服务	4	2
SH600010	包钢股份	5.40	5.40	27.20	27.20	申万钢铁	1	3
SH600019	宝钢股份	4.89	4.89	18.10	18.10	申万钢铁	2	3
SH601668	中国建筑	3.90	3.90	9.00	9.00	申万建筑建材	3	3
SH601669	中国电建	3.82	3.82	0.00	0.00	申万建筑建材	4	3
SH600036	招商银行	13.75	13.75	78.50	78.50	申万金融服务	5	3
SH600030	中信证券	13.36	13.36	78.20	78.20	申万金融服务	6	

因为申万钢铁样本个数为 2，而分组数等于 3。对申万钢铁不再进行分组，在每个分组中，都包含申万钢铁的所有股票

考虑行业因素且行业样本个数小于分组数，此行业不再分组，各组每个行业分布为：

分组	申万金融服务	申万建筑建材	申万钢铁	小计
1	2	1	2	5

2	2	1	2	5
3	2	2	2	6
小计	6	4	6	16

分析如下：

（1） 申万金融服务有 6 个样本，分组数为 3，则每个组分配 2 个样本

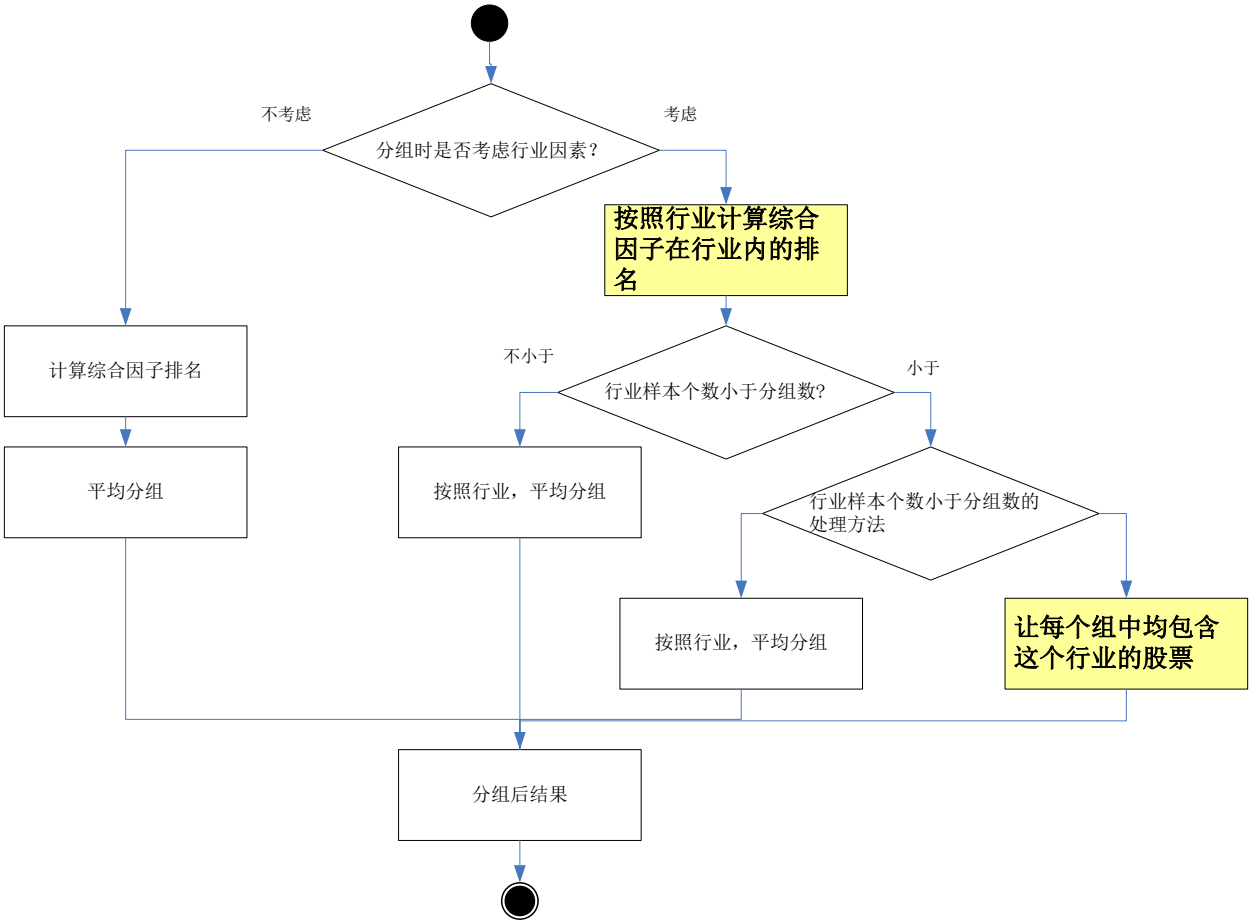
（2） 申万建筑建材有 4 个样本，分组数为 3，则 1 组、2 组分配 1 个样本，3 组分配 2 个样本

（3） 申万钢铁有 2 个样本，分组数为 3，行业样本个数小于分组数，此行业不再分组。在每个组别都有申万钢铁的所有股票

这样处理的结果，组间一般不再有行业分布的不均匀。但是存在一个样本存在不同组间的可能

3.4.1.4.4 分组算法流程图

以（个股数）百分位分组



3.4.1.4.5 因子分组常见方法及优缺点

分组方式	优点	缺点	应用场合
分组时不考虑行业	方法简单	组间行业分布可能很不均匀。组间收益率差很可能是行业带来的，不一定是因子带来的	最不推荐使用

按行业分组	分组方法相对复杂	当行业样本个数<分组时，组间的行业分布也可能不均匀	一般推荐
按行业分组，行业样本小于组数时不再分组	分组方法最复杂	组间有重复样本	推荐使用

3.4.2 每组个股资金分配方法(FRateType)

多因子框架中提供的每组个股资金分配方式如下：

方法	备注
总股本加权	$W_i = \frac{P_i \times TS_i}{\sum_{i=1}^{N} P_i \times TS_i} \times 100$
流通股本加权	$W_i = \frac{P_i \times NS_i}{\sum_{i=1}^{N} P_i \times NS_i} \times 100$
等权重	多因子中最常用 $W_i = \frac{1}{N} \times 100$
等股数	$W_i = \frac{P_i}{\sum_{i=1}^{N} P_i} \times 100$
流通市值平方根加权	$W_i = \frac{\sqrt{P_i \times NS_i}}{\sum_{i=1}^{N} \sqrt{P_i \times NS_i}} \times 100$
总市值平方根加权	$W_i = \frac{\sqrt{P_i \times TS_i}}{\sum_{i=1}^{N} \sqrt{P_i \times TS_i}} \times 100$
自由流通股本加权	根据指数公司指数权重预估
基准权重加权	基准指数成分权重
用户自定义配比	在重写 GetSamples 时传入”比例(%)”列

其中：

N: **每组中**所有证券个数（注意：每组的个数可能不同）

P_i: 证券 i 收盘价

NS_i: 证券 i 流通股本

TS_i： 证券 i 总股本

3.4.3 行业资金分配方法(FIndustryMethod)

3.4.3.1 不做行业中性处理

此种方法，只考虑个股的资金分配，不考虑不同行业在基准中的权重差异。下面范例以个股等权重为例来说明

代码	名称	综合因子	行业名称	行业内排名	分组	权重(%)
SH600010	包钢股份	45.40	申万钢铁	1	1	20.0%
SH600019	宝钢股份	18.10	申万钢铁	2	1	20.0%
SH600585	海螺水泥	81.80	申万建筑建材	1	1	20.0%
SH601318	中国平安	100.00	申万金融服务	1	1	20.0%
SH601601	中国太保	90.90	申万金融服务	2	1	20.0%
						100%
SH600010	包钢股份	27.20	申万钢铁	1	2	20.0%
SH600019	宝钢股份	18.10	申万钢铁	2	2	20.0%
SH600068	葛洲坝	72.70	申万建筑建材	2	2	20.0%
SH601628	中国人寿	88.40	申万金融服务	3	2	20.0%
SH601166	兴业银行	80.50	申万金融服务	4	2	20.0%
						100%
SH600010	包钢股份	27.20	申万钢铁	1	3	16.7%
SH600019	宝钢股份	18.10	申万钢铁	2	3	16.7%
SH601668	中国建筑	9.00	申万建筑建材	3	3	16.7%
SH601669	中国电建	0.00	申万建筑建材	4	3	16.7%
SH600030	中信证券	78.20	申万金融服务	6	3	16.7%
SH600036	招商银行	78.50	申万金融服务	5	3	16.7%
						100%

每个行业权重分配如下：

分组	申万金融服务	申万建筑建材	申万钢铁	小计
组 1	40.0%	20.00%	40.0%	100.0%
组 2	40.0%	20.00%	40.0%	100.0%
组 3	33.3%	33.3%	33.3%	100.0%

不做行业中性配置的处理方法，结果是每个行业同等对待。此种方法较少使用。

3.4.3.2 行业中性

资金分配的时候，如果简单的按上面的方法来分，就无法体现行业比重的差别。所以多因子框架提供了行业中性的处理。

个股行业中性处理后的比例(%)为：

$$W_i' = W_{b,k} \times \frac{W_i}{\sum_{j=1}^{Nk} W_j}$$

其中：

$W_{b,k}$：个股所属行业在基准中的行业权重

W_i：组内每个个股的权重（未做行业中性处理前）

Nk：个股所属行业的组内个股个数

代码	名称	综合因子	行业名称	行业内排名	分组	未做处理前权重(%)	基准行业权重(%)	权重(%)
SH600010	包钢股份	45.40	申万钢铁	1	1	20.00%	2.51%	1.25%
SH600019	宝钢股份	18.10	申万钢铁	2	1	20.00%	2.51%	1.25%
SH600585	海螺水泥	81.80	申万建筑建材	1	1	20.00%	7.81%	7.81%
SH601318	中国平安	100.00	申万金融服务	1	1	20.00%	89.69%	44.84%
SH601601	中国太保	90.90	申万金融服务	2	1	20.00%	89.69%	44.84%
						100.00%		100.00%
SH600010	包钢股份	27.20	申万钢铁	1	2	20.00%	2.51%	1.25%
SH600019	宝钢股份	18.10	申万钢铁	2	2	20.00%	2.51%	1.25%
SH600068	葛洲坝	72.70	申万建筑建材	2	2	20.00%	7.81%	7.81%
SH601628	中国人寿	88.40	申万金融服务	3	2	20.00%	89.69%	44.84%
SH601166	兴业银行	80.50	申万金融服务	4	2	20.00%	89.69%	44.84%
						100.00%		100.00%
SH600010	包钢股份	27.20	申万钢铁	1	3	16.67%	2.51%	1.25%
SH600019	宝钢股份	18.10	申万钢铁	2	3	16.67%	2.51%	1.25%
SH601668	中国建筑	9.00	申万建筑建材	3	3	16.67%	7.81%	3.90%
SH601669	中国电建	0.00	申万建筑建材	4	3	16.67%	7.81%	3.90%
SH600030	中信证券	78.20	申万金融服务	6	3	16.67%	89.69%	44.84%
SH600036	招商银行	78.50	申万金融服务	5	3	16.67%	89.69%	44.84%
						100.00%		100.00%

经过行业中性处理后，**每个行业的配置比例和基准行业配置完全相同**，而行业下的个股配置比例可以采取流通市值加权、等比例配置等方法

分组	申万金融服务	申万建筑建材	申万钢铁	小计
组 1	89.69%	7.81%	2.51%	100.0%
组 2	89.69%	7.81%	2.51%	100.0%
组 3	89.69%	7.81%	2.51%	100.0%

4 因子评价

多因子框架从以下几个方面来评价因子是否有效：

（1） 因子收益率检验：检验每组是否可以取得正的收益率？收益率波动率如何？取得正的收益率概率有多大？

（2） 因子显著性检验：检验每组是否可以取得正的超额收益率？取得正的超额收益率概率多大？

（3） 因子区分度检验：第一组收益率是否显著大于最后一组？因子是否能显著把表现好的股票和表现差的股票分别出来？

（4） 因子延续性检验：由因子得到的分组是否能够在较长的一段时间保持比较好的方向性。即：本期表现的因子，在下期是否也取得好的收益率？

（5） 因子贡献度检验：因子的反映的信息是否有重叠？各个因子的打分对下期收益的贡献到底有多大？

4.1 分组分析

4.1.1 各组的不同收益率类型（GetGroupReturn）

好的因子可以把收益良好和收益差的股票分开，这个评价方法提供因子的收益率/累计收益率情况，用户可以通过观察收益率序列大致看出因子的分组效果如何。

收益率(%)/累计收益率(%)/超额收益率(%)/累计超额收益率(%)返回结果：

字段名称	含义	备注
开始日		
截止日		
交易时点	开始日到截止日实际使用的日收益率个数	
基准	基准的收益率(%)/累计收益率(%)/超额收益率(%)/累计超额收益(%)	
第 1 组 ….. 第 n 组	各组的收益率(%)/累计收益率(%)/超额收益率(%)/累计超额收益(%)	

第一组 vs 最后一组的超额收益/累积超额收益率返回结果：

字段名称	含义	备注
开始日		
截止日		
交易时点	开始日到截止日实际使用的日收益率个数	
第一组 vs 最后一组	第一组收益率(%)最后一组收益率(%)	
第一组 vs 最后一组->累计	第一组累计收益率(%)-最后一组累计收益率(%)	

4.1.2 各组不同的换手率（GetGroupTurnOver）

换手率，评价因子的交易成本暴露。

一般有两种算法：

（1）个数法：T 期各分组中股票数目变化百分比：

（2）权重法：T-1 期到 T 期各组股票权重值变化绝对值加总的一半。注意，由于每日股票的变动，实际调仓前的权重不等于 T-1 期权重。

4.1.3 因子组收益率检验（GetTrailingReturn）

分组之后，各组在各个时间区间的表现如何？组与组之间在同一个时间区间的区分度如何？这些问题就是这个检验方法所探讨的问题。

4.1.3.1 检验目的

各组收益率均值、标准差、收益率大于 0 的概率

4.1.3.2 检验方法

1、 统计了近一年、近 2 年、近 3 年和整体的数据。(可重写函数 GetShowPeriod 实现用户个性化定制时间区间)

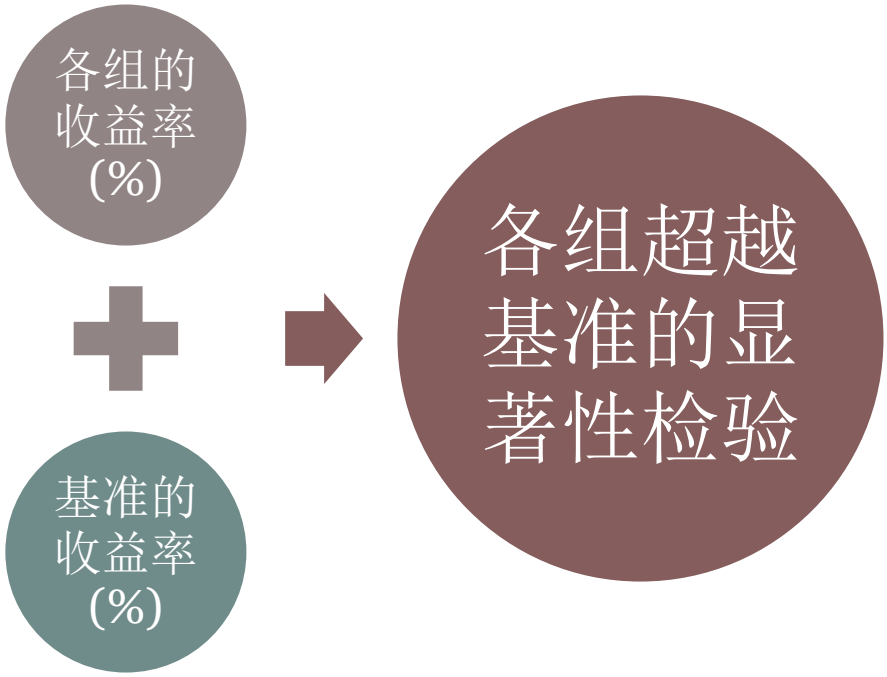
2、 检验对象：各组收益率均值、标准差、胜率

4.1.3.3 评价指标

评价指标	说明	备注
平均收益(%)	各组的平均收益	越大越好，说明第一组收益率显著大于最后一组
标准差(%)	各组的标准差(%)	
夏普比率	各组的夏普比率	
胜率(%)	各组的胜率(%)	各组取得正的收益率概率

4.1.4 因子显著性检验（GetSignificanceStatistics）

分组之后，我们总是希望第一组显著的超越基准（说明分组可以选出好的股票），最后一组被基准超越（说明分组可以剔除差的股票）。



4.1.4.1 检验目的

研究各组超越基准的情况。我们希望第一组明显超越基准，最后一组明显被基准超越。

4.1.4.2 检验方法—t 检验法

统计样本：(各分组的收益率序列-基准的收益率序列)

原理：统计学单个正态总体均值的假设检验

系统标准差是否已知	H_0（原假设）	H_1（备择假设）	检验统计量及其分布	否定域
δ^2 未知	$\mu=\mu_0$	$\mu\neq\mu_0$	$T=\sqrt{n}(E(X)-\mu_0)/S$ $T\mid_{\mu=\mu_0}\sim t_{n-1}$	$\lvert T\rvert>t_{n-1}(\alpha/2)$
	$\mu\leq\mu_0$	$\mu\succ\mu_0$		$T>t_{n-1}(\alpha)$
	$\mu\geq\mu_0$	$\mu\prec\mu_0$		$T<-t_{n-1}(\alpha)$
δ^2 已知	$\mu=\mu_0$	$\mu\neq\mu_0$	$U=\sqrt{n}(E(X)-\mu_0)/\sigma$ $U\mid_{\mu=\mu_0}\sim N(0,1)$	$\lvert U\rvert>u_{\alpha/2}$
	$\mu\leq\mu_0$	$\mu\succ\mu_0$		$U>u_\alpha$
	$\mu\geq\mu_0$	$\mu\prec\mu_0$		$U<-u_\alpha$

其中：

n：样本个数。

$E(X)$：样本均值。

μ_0：估计量。

S：样本标准差。

σ：系统标准差。

该检验对应到的原假设和备择假设如下：

H_0:第i组-基准≥0，H_1:第i组-基准<0。

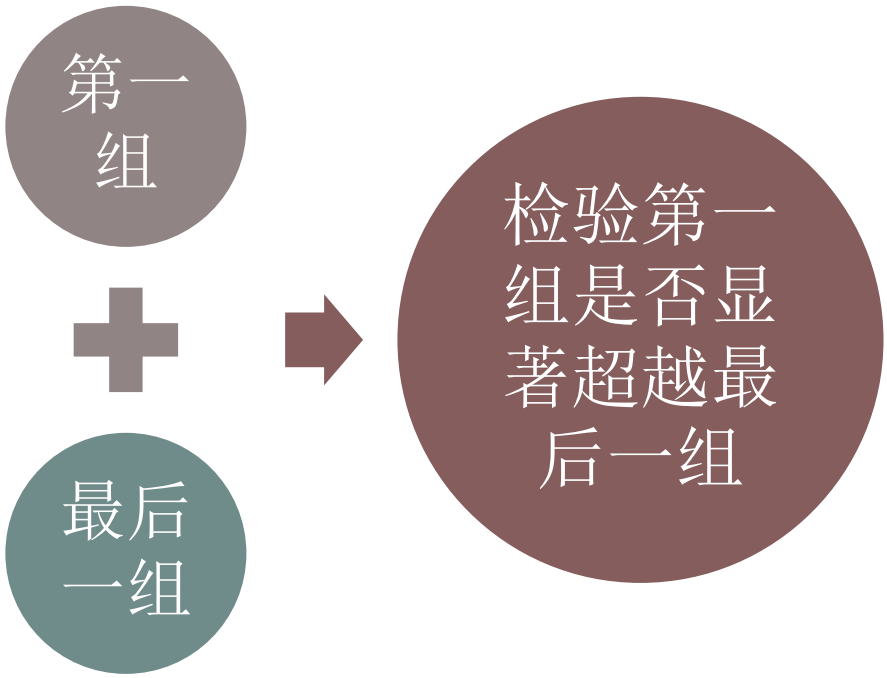
4.1.4.3 评价指标

评价指标	说明	备注
超额均值(%)	各组收益率相对基准的超额收益率(%)的均值	越大说明该组超越基准越明显
跟踪误差(%)	超额收益率的标准差，衡量各组偏离基准的程度。	越小越好
信息比率	风险收益率指标，超额收益率均值/超额收益率标准差	越大说明超越基准越明显
T-Stat	T 统计量	该值越大说明超越基准越多
P-Value	P 值，计算的是原假设 H0：超额>=0 时的左侧 t 检验 显著性概率(%)。	p 值不能太小，越小表示其超额越显著小于 0。
超额概率(%)	超基准收益率(%)中大于 0 的概率(%)	

4.2 多空分析

4.2.1 因子区分度检验（GetLongShortStatistics）

分组之后，总是希望分组能把收益率高的分到前面的组，收益率低的分到后面的组。因子区分度检验评价方法就是为了检验这个问题而设计的。



4.2.1.1 检验目的

检验第一组收益率是否显著超越最后一组收益率

4.2.1.2 检验方法

1、 时间维度：时间维度—近 1 年、近 2 年、近 3 年、整体。(可重写函数 GetShowPeriod 实现用户个性化定制时间区间)

2、 检验对象：第一组相对最后一组的超额收益率(%)。一般用月度收益率（Monthly Extra Return）来度量

4.2.1.3 评价指标

评价指标	说明	备注
平均收益(%)	第一组相对最后一组超额收益率的平均收益(%)	越大越好，说明第一组收益率显著大于最后一组
标准差(%)	第一组相对最后一组超额收益率的标准差。第一组相对最后一组超额收益率的波动	越小越好。
胜率(%)	第一组战胜最后一组的概率(%)	越大越好，表示第一组收益率显著大于最后一组
夏普比率	超额收益的平均值/超额收益的标准差	越大越好
T-Stat	检验对象对应的 T 统计量	越大越好
P-Value	P 值，计算的是原假设 H0：第一组收益-最后一组收益>=0 时的左侧 t 检验 显著性概率(%)。	越大越好 ， p 值越小，其多空越显著小于 0，区分度越不明显

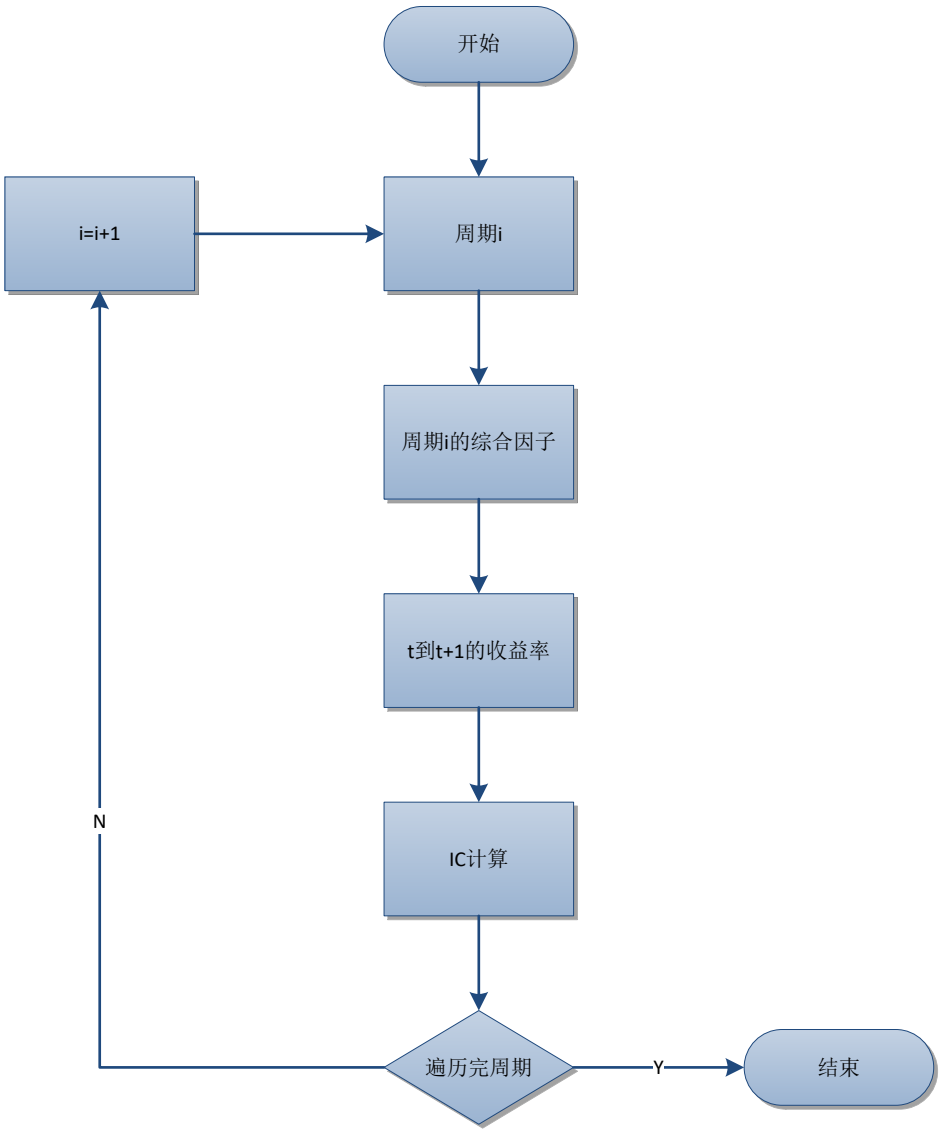
4.3 IC 分析

4.3.1 因子延续性检验 (GetInformationCoefficient)

事实上，很难找到某个因子，一直保持某个的方向，不同时期因子的方向可能会改变。举一个类似例子，‘增发股票’这个事件可能在牛市的时候是一个利好消息，在熊市的时候确是一个利空消息（因为按常理，人的心理变化了，对同一事件的反应不同，最后反应到对股票的购买欲上面）。

那么，既然不能找出方向一直保持不变的因子，我们还是希望能找出延续性很好的因子（即能在比较长时间保持方向不变的因子）。这个评价方法就是来解决这个问题的。

4.3.1.1 IC 计算流程



4.3.1.2 IC 计算方法

4.3.1.2.1 Normal IC

T 期末各股票综合因子打分序列F_t和 T 期到 T+1 期各股票的涨幅(%)序列R_{t+1}间的 Pearson 相关系数。计算公式：

$$normIC = corr(F_t, R_{t+1})$$

4.3.1.2.2 RankIC

T 期末各股票综合因子打分序列F_t和 T 期到 T+1 期各股票的涨幅(%)序列R_{t+1}间的 Spearman 相关系数。计算公式：

$$RankIC = Scorr(F_t, R_{t+1})$$

与 Pearson 相关系数相比，Spearman 秩相关系数可以排除非线性的因素，但也会带来一定的信息丢失。在实践中秩相关系数均值往往更大一些，显著比例也要更高一些。

4.3.1.2.3 Kendall 秩相关

T 期末各股票综合因子打分序列F_t和 T 期到 T+1 期各股票的涨幅(%)序列R_{t+1}间的 kendall 秩相关系数，一般不使用。

4.3.1.2.4 行业调整 IC

对股票收益率序列根据股票所属行业的平均收益率进行调整。

$$r_i^a = r_i - r_{stock\in Indus_j} \quad (j = 1、2 \cdots n)$$

其中，r_i为股票 i 的收益率，$r_{stock\in Indus_j}$表示股票 i 所属行业 j 的平均收益率，在计算行业平均收益率时，可以选择等权重或按照流通市值加权。

记调整后的股票收益率序列为R_t^a，则行业调整 IC 为 T 期末股票综合因子打分序列F_t和 T 期到 T+1 期各股票的调整后收益率序列R_{t+1}^a间的相关系数。

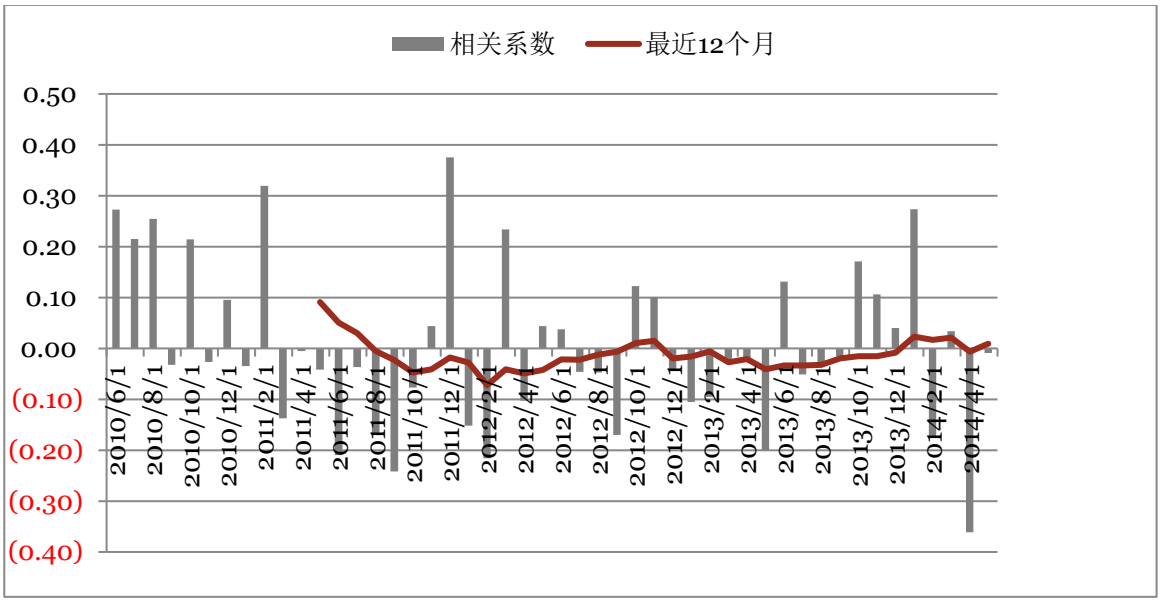
$$RankIC_a = corr(F_t, R_{t+1}^a)$$

4.3.1.2.5 加权 IC

按照股票在基准成分股的流通市值占比作为对应个股权重，计算 T 期末股票综合因子打分序列F_t与 T 期到 T+1 期各股票的涨幅(%)序列R_{t+1}间的加权相关系数，即为基准市值加权 IC。

4.3.1.3 IC（相关系数）的作用

通过各期的 IC 值可以大致看出因子的延续性，在哪一段是正向的，哪一段是反向的；比如：这个图中，柱状的系列 1 代表对应日期的相关系数，红线系列 2 是 1 年的 IC 均线，如果红线始终在 x 轴上方（或者下方），那么说明延续性很好。如果红线总是在 x 轴上下波动，那么说明因子的延续性很差。比如下面的图显示在 2011 年 8 月份开始，到 2012 年 8 月份，所示因子都是表现出比较好的反向延续性。



4.4 回归分析

4.4.1 因子贡献度检验 1 (GetLinearEvaluationResult)

4.4.1.1 检验目的

多因子框架可以对多个因子进行有效性的测试，那么就有一个问题，各个因子所起的作用究竟如何？各个因子分别对下期收益的影响到底怎么样？如何在之后调整各个因子的比例(%)以达到更好的效果？
那么贡献度检验方法就是检验各个因子的得分对下期收益的贡献度，以此作为之后调整因子比例(%)的依据。

4.4.1.2 检验方法

用各个因子标准化后得分和下期收益做最小二乘参数估计（带常数项），即自变量通过某种线性变换去拟合因变量，寻找各个自变量与因变量的关系。

4.4.1.3 两种展现形式

（1） 每期的数据分别去做小二乘参数估计。
（2） 用所有期的数据去做最小二乘（每期变因子不能使用这种方法）。

4.4.1.4 评价指标

得到各个因子对应的回归系数、常数项，以及基于此计算的推荐的配置比例(%)。详细含义解析如下：
（1） 回归系数的正负的含义：如果回归系数是负的，那么说明该系数对应的因子对下期收益的贡献是负的，应该在之后配置因子的时候把该因子的方向设置为跟原来相反。如果是正的说明该系数对应的因子对下期收益的贡献是正的。
（2） 回归系数的值的含义：令某个因子对应的回归系数为α_i，那么$|\alpha_i|$越大说明对应到的因子对股票的区分度越好，下期收益高和下期收益低的股票被区分的很明显，所以对于$|\alpha_i|$高的股票建议配置较高的比例(%)。
（3） 评价结果中推荐比例(%)的计算方法：评价结果中给出了一个推荐比例(%),计算方法如下，

$$\omega_i = \frac{\alpha_i}{\sum_{j=1}^{N}|\alpha_j|} \times 100$$

其中，α_i是第i个因子对应的回归系数，N 是回归系数的总个数，ω_i是多因子框架计算到的推荐比例(%)。当结果ω_i为负数时代表，建议你给因子配$|\omega_i|$这么大的比例(%)，然后因子方向调成跟原来相反的。

4.4.2 因子贡献度检验 2(GetPrincipalComponentResult)

4.4.2.1 检验目的

（1） 在实际处理多因子的问题中常遇到两个问题：1）由于因子的个数太多，可能彼此间存在一定的相关性，因而使得所观测到的数据在一定程度上反映的信息有所重叠。2)人们希望知道没有重叠的这些因子对于下期收益的贡献如何。

（2） 在考虑因变量 Y 与 p 个自变量 $X_1,X_2,\cdots,X_p$ 的回归模型中，如果自变量间有较强的线性相关（多重共线性）时，利用最小二乘参数估计法（即上面的评价方法），一般效果较差。而这里采用的主成分回归法，首先是计算这些因子的前 m 主成分，达到一个降维的效果，这样既简化了回归方程的结构又消除了变量间相关性带来的影响。但是只有主成分分析这步的话，因为主成分是原始变量的线性组合，这就造成了解释上的困难。所以这里使用逆变换将其变成原始变量的回归方程，这就是主成分回归法。

4.4.2.2 检验方法

使用标准化之后的数据做***主成分回归***（主成分回归相关推导及理论知识可参见多元统计分析相关书籍）。

参考链接：http://www.tinysoft.com.cn/tsdn/helpdoc/display.tsl?id=12877

4.4.2.3 评价指标

各因子对应的主成分回归系数、主成分回归常数项，以及基于此计算的推荐的比例(%)配置。

4.4.2.3.1 两种展现方式

（1） 按调仓日每期数据做主成分回归。

（2） 所有期的数据做主成分回归（各期变因子不能使用此种方法）。

4.4.2.3.2 评价指标

参见因子因子贡献度检验 1——评价指标。

4.5 因子值分析

4.5.1 因子稳定性检验 (GetSignalStability)

4.5.1.1 检验目的

检验因子信号的稳定性

4.5.1.2 评价指标

（1）整体因子值稳定性：自相关系数

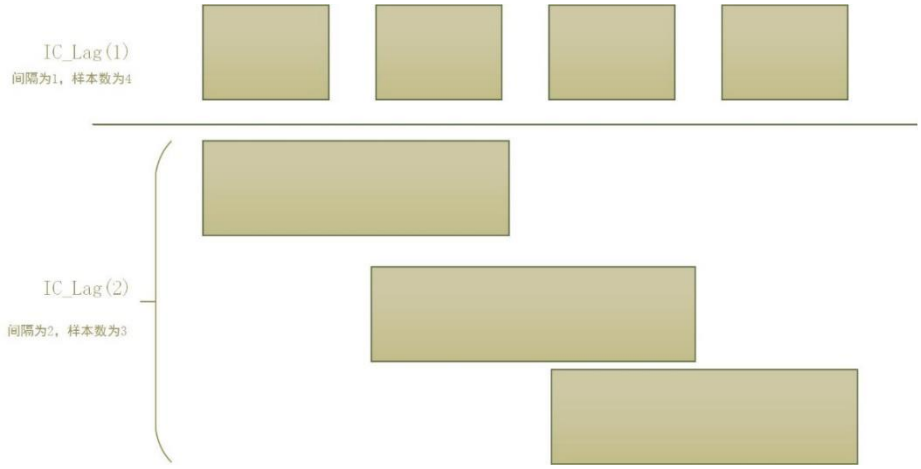
（2）分组稳定性：买入衰减、买入反转、卖出衰减、卖出反转

评价指标	说明	备注
自相关系数	前后期样本的因子暴露相关系数	越大说明信号越稳定
买入衰减(%)	T 期 Top 组的股票成分，在 T+1 期继续在 Top 组中的比例。	该值越大说明买入组成分越稳定
买入反转(%)	T 期 Top 组的股票成分，在 T+1 期转变为在 Bottom 组中的比例。	越小越好

卖出衰减(%)	T 期 Bottom 组的股票成分，在 T+1 期继续在 Bottom 组中的比例	该值越大说明卖出组成分越稳定
卖出反转(%)	T 期 Bottom 组的股票成分，在 T+1 期不在 Bottom 组，而是在买入组，即 Top 组的比例，即完全由卖出信号反转为买入信号。	越小越好

4.6 衰减分析

衰减分析刻画的是因子表现随着调仓时间间隔延长时的表现变化。
以 IC 信号衰减为例，一般提到 IC 间隔 N=1 期，即用 T 期末因子值与个股 T+1 收益率之间的相关性。如果把所有 T 期末因子和 T+1 期间收益率的 IC 值算出来求平均，记为 IC_Lag(1)，遍历 N=1、2、……，这个 IC_lag 随间隔 N 的序列，反映因子有效性随时时间的衰减情况。



4.6.1 延续性衰减(按调仓期) (GetDecay_InformationCoefficient)

4.6.1.1 评价目的

展示随着调仓间隔的增加，因子有效性的变化情况，属于 IC 分析的拓展。

4.6.1.2 评价指标

■ 简单统计：Retype=1

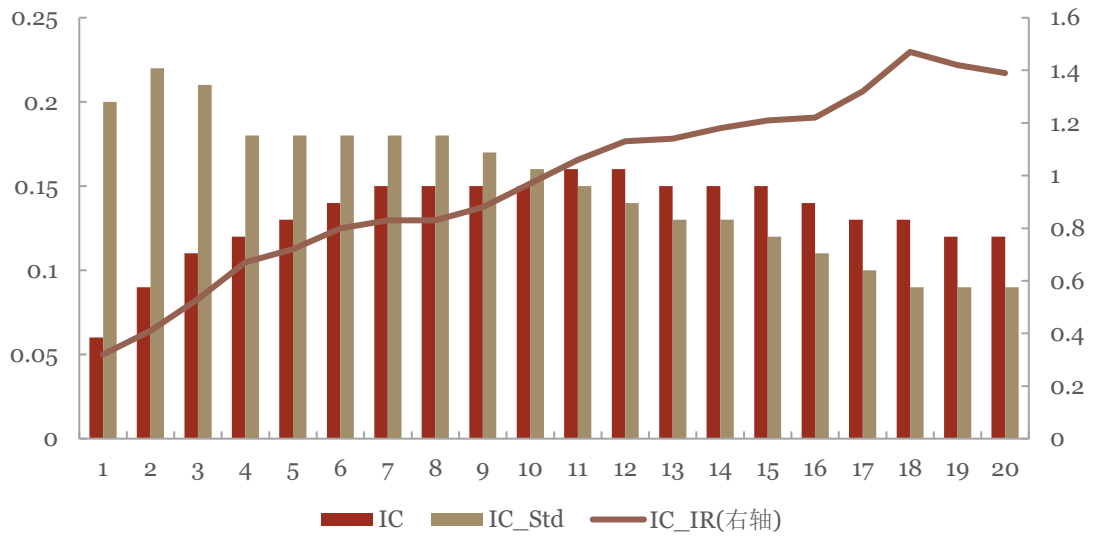
字段名称	计算方法	统计含义	备注
间隔期数			从 1 开始
IC	$IC = \text{mean}(t)$ （其中 t 代表 IC 序列）	ic 均值，核心体现因子对个股收益的预测方向有效性，正负值对应预测方向（正值为正向预测、负值为反向预测）	
IC_Std	$IC_Std = \text{StdDev}(t)$	衡量 IC 序列的波动幅度，数值越小说明因子预测能力的稳定性越强，预测效果越可控。	

IC_IR	$\text{IC_IR}=\frac{\text{mean}(t)}{\text{StdDev}(t)}$ 。	衡量单位波动下的 IC 收益水平，是因子风险调整后预测效率的核心指标，数值越高说明因子性价比越强（收益与稳定性平衡越好）	
显著比率(%)	$\frac{\sum I(\text{P-value}\le 0.05)}{\text{len}}\times 100\%$	IC 显著不为 0 的比例，按每期计算 IC 时的 P 值为显著标准。	
样本数		样本数越多，统计结果的代表性、可靠性越强，避免小样本偏差对结论的影响。	

■ 详情统计：RetType=2

指标分类	字段名称	计算方法	统计含义
基础水平类	均值	$\text{mean}(t)$ （其中 t 为 IC 序列）	因子 IC 的整体平均水平，反映因子预测方向的整体有效性（正负代表方向）
	绝对值均值	$\text{mean}(\text{abs}(t))$	剔除正负方向，衡量因子 IC 的平均绝对强度，体现整体预测力度
	标准差	$\text{std}(t)$	衡量 IC 序列的波动程度，反映因子预测能力的稳定性
	ICIR（IC 信息比率）	$\frac{\text{mean}(t)}{\text{std}(t)}$	单位波动下的 IC 均值，衡量因子风险调整后的预测效率（值越高性价比越强）
显著性检验类	T 值	基于原假设 H_0:$\text{mean}(t)=0$ 的双侧 t 检验，取检验对应的 T 统计量	检验 IC 均值是否显著非 0，T 值越大，因子整体有效性越显著
	P-value	双侧 t 检验对应的概率值，显著性水平设定为 0.05	IC 均值为 0 的概率， 则认为因子整体有效（拒绝原假设）
	胜率(%)	$\frac{\sum I(t>0)}{\text{length}(t)}\times 100\%$	因子预测方向正确的频率，胜率越高，预测方向越精准
	显著比例(%)	$\frac{\sum I(\text{abs}(t)>0.05)}{\text{length}(t)}\times 100\%$	因子 IC 显著有效的时段占比，比例越高，因子有效时段越多
持续性/稳定性类	同向比例(%)	$\frac{\sum I(\text{Sign}(t_{i+1}\times t_i)=1)}{\text{length}(t)-1}\times 100\%$	相邻 IC 符号相同的比例，反映因子预测能力的持续性（比例越高越稳定）
	状态转换比率(%)	$\frac{\sum I(\text{Sign}(t_{i+1}\times t_i)=-1)}{\text{length}(t)-1}\times 100\%$	相邻 IC 符号相反的比例，反映因子预测能力的波动频率（比例越高越不稳定）
分布特征类	偏度	$\text{Skewness}(t)$	衡量 IC 分布的不对称性，偏离 0 越远，IC 分布越偏向正/负方向
	峰度	$\text{Kurtosis}(t)$	衡量 IC 分布的陡峭程度，偏离 0 越远，IC 极端值（强有效/无效）出现概率越高

4.6.1.3 图形展示



4.6.2 延续性衰减（按每日）（GetDecay_IC）

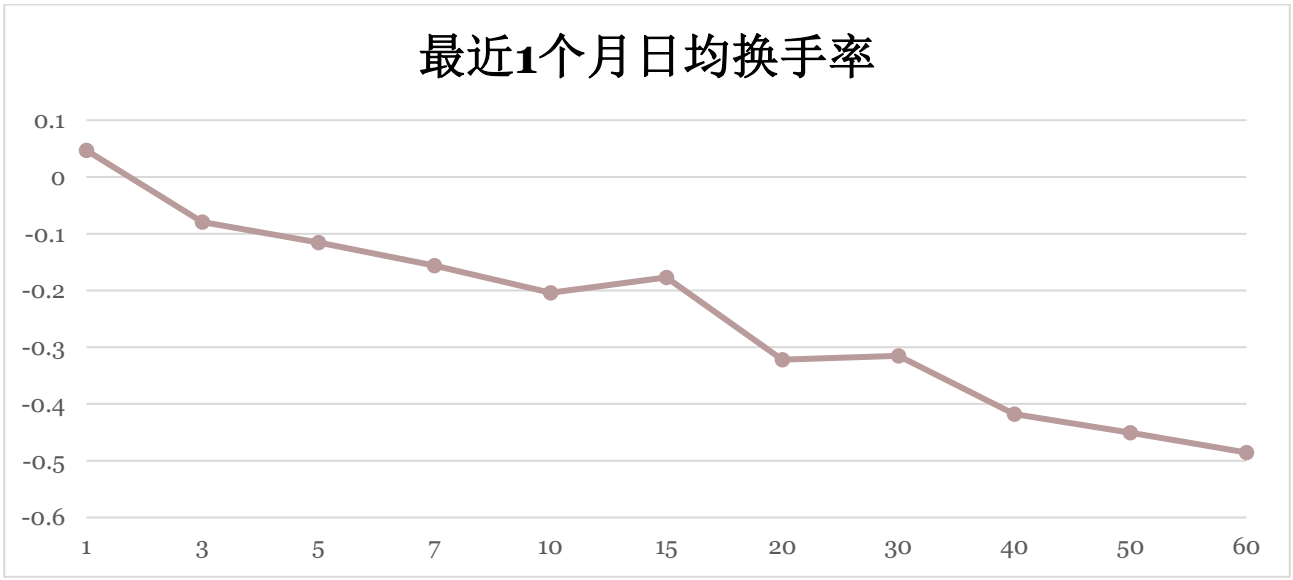
4.6.2.1 评价目的

观察展示随着间隔天数的增加，因子有效性的变化情况，属于 IC 分析的拓展。

4.6.2.2 评价指标

见延续性衰减(按调仓期)（GetDecay_InformationCoefficient）

4.6.2.3 图形展示



4.6.3 稳定性衰减(按调仓期)（GetDecay_SignalStability）

4.6.3.1 评价目的

展示随着调仓间隔的增加，因子信号稳定性的变化情况

4.6.3.2 评价指标

见因子信号稳定性检验

5 TSMultiFactor：天软多因子框架

5.1 天软多因子常见属性

5.1.1 属性汇总

所属	属性	类型	含义	默认值
常用属性	FBegT	DateTime	开始日期	当前系统时间往前推 3 年
	FEndT	DateTime	截止日期	当前时间
	FIndexId	String	基准代码	SH000300
	FCycle	String	调仓周期，字符串	Cy_month()
	FFactorArr	Array	因子库	
	FFillMissType	Integer	缺失值处理方法	0：剔除
	FAbnormalType	Integer	极值处理方法	0：中位数法
	FStandarizeType	Integer	标准化方法	1：Z 值法
	FNeuType	Integer	中性化方法	0：不进行中性化
	FOrthType	Integer	正交化方法	0：不进行正交化
	FConstr	String	剔除条件	空字符串
	FBuyRatio	Numeric	开仓费率(%)	0.1
	FSellRatio	Numeric	平仓费率(%)	0.1
分组配权单元	FGroupType	Integer	分组方式	1：百分位
	FGroups	String		五分位
	FGroupHy	Integer	分组时是否考虑行业因素	-1：分组时不考虑行业，直接按综合得分排名分组。
	FIndustryMethod	Integer	行业资金分配方法	-1：不进行行业中性处理
	FRateType	Integer	个股资金分配方法	4：等权重
行业	FIndustryType	Integer	行业分类方式	1：按申万一级行业来划分行业

网格相关	FGridComputing	Boolean	是否使用天软网格功能。	false(只有开通了天软网格才能使用网格功能)
	FGridNo	Integer	网格数目	5
回测相关	FpriceType	Integer	成交价类别	2：收盘价
	FvolModType	Integer	成交量取整模式	0：不取整
	FdividendType	Integer	是否分红再投资	0：分红不再投资
	FAllotmentType	Integer	是否参与配股	0：不自动参与配股
	FBacktestType	Integer	使用的回测方法	0：表示使用回测框架清算
进阶属性	FBMType	Integer	基准类型	0：单个基准
	FBMDetail	Array	复合基准成分	Array()

5.1.2 FBegT

- 含义：回测开始时间
- 默认：从当前系统时间往前推 3 年

5.1.3 FEndT

- 含义：回测截止时间
- 默认：当前系统时间

5.1.4 FIndexId

- 含义：基准代码
- 默认：SH000300

5.1.5 FCycle

- 含义：设置调仓周期
- 默认：Cy_month(),按月调仓

5.1.6 FFactorArr

- 含义：设置因子库。主要是因子名称、因子公式、因子方向、比例(%)
- 说明：此参数为二维字符串下标数组，具体定义如下：

字段	类型	单位	非空	备注
因子名称	String		是	用于显示的名称，每个因子的因子名称*必须不同* 不能含;, 一般不建议以下划线开头， 避免与 updateStockInfoData 增加的字段冲突
因子公式	String		是	tsl 语言中的指标公式，可以解析的字符串

因子方向	Integer		是	1：正向因子 0：反向因子
因子比例(%)	Numeric	(%)	是	代表该因子在所有因子中的权重，百分数

- 注意事项：

  本参数是多因子分析框架最核心的部分。此参数需要用户根据自己对因子的理解，独立设置。

  特别注意，因子表的附加属性对于数据处理方式的影响方式，详细内容请参看章节因子的附加属性

- 范例：

```
//两个因子：每股收益为正向因子，60%权重；收盘为反向因子，40%权重
FactorArr:=array(
("因子名称":"每股收益","因子公式":"Last12MData(RDate,9900000)","因子方向":1,"因子比例(%)":60),
("因子名称":"收盘","因子公式":"StockClose(EndT)","因子方向":0,"因子比例(%)":40));
obj:=CreateObject('TSMultiFactor');
obj.FBegT:=20131231T;
obj.FEndT:=20140530T;
obj.FIndexId:='SH000016';
obj.FCycle:=cy_month();
   //设置因子库
obj.FFactorArr:=FactorArr;
obj.FAbnormalType:=0;
obj.FStandarizeType:=1;
obj.FGroupType:=1;
obj.FGroups:="0,1/3*100;1/3*100,2/3*100;2/3*100,100";
obj.FConstr:='';
obj.FBuyRatio:=0.1;
obj.FSellRatio:=0.1;
Ret:=obj.BackTest();
return obj.GetGroupReturn(1,cy_day());
```

- 默认：

5.1.7 FFillMissType

- 含义：缺失值处理方法
- 取值：

取值	含义	备注
-1	不处理	
0,nil	剔除	缺省
1	平均值填充	
2	中位数填充	

4	中性值填充	在极值处理和标准化处理中跳过不处理，标准化处理后，补充标准化方法的中性值。例如 zscore 法会对缺失值补 0
13	市值回归填充	
201	按行业平均值填充	
202	按行业中位数填充	
213	按行业市值回归填充	

- 默认：0（剔除）

5.1.8 FAbnormalType

- 含义：极值处理方法
- 取值：

取值	含义	备注
-1	不做极值处理	当无量纲化选择百分比排名打分法时，请务必选择‘不做极值处理’
0,nil	中位数取极值法	缺省
1	3 倍标准差法	
2	四分位处理法	
3	十分位处理法	
4	鲁棒标准差法	*不建议使用*
5	压缩去极值 MAD 法	
200	行业内中位数法	先按行业分组后，在每个行业内按照中位数去极值法进行数据处理
201	行业内 3 倍标准差法	先按行业分组后，在每个行业内按照 3 倍标准差法进行数据处理
202	行业内四分位法	先按行业分组后，在每个行业内按照四分位处理法进行数据处理
203	行业内十分位法	先按行业分组后，在每个行业内按照十分位处理法进行数据处理
204	行业内鲁棒标准差法	先按行业分组后，在每个行业内按照鲁棒标准差法进行数据处理，*不建议使用*
205	行业内压缩去极值法	先按行业分组后，在每个行业内按照压缩去极值 MAD 法进行数据处理

- 默认：0（中位数取极值法）

5.1.9 FStandarizeType

- 含义：数据无量纲化方法
- 取值：

取值	含义	备注
0,nil	不处理	
1	Z 值法	缺省
2	极差标准化	使数据平移伸缩变换到[-1,1]

3	极差正规化	使数据平移伸缩变换到[0,1]
4	百分比排名打分法	根据数据在整体中的百分比去打分
201	行业内 Z 值法	先按行业分组，在每个行业内按照 Z 值法进行数据处理
202	行业内极差标准化	先按行业分组，在每个行业内按照极差标准化法进行数据处理
203	行业内极差正规化	先按行业分组，在每个行业内按照极差正规化法进行数据处理
204	行业内百分比打分法	先按行业分组，在每个行业内按照百分比打分法进行数据处理

- 默认：1（Z 值法）

5.1.10 FNeuType

- 含义：中性化方法
- 取值：

取值	含义	备注
0,nil	不处理	缺省
1	行业中性化	由 updateStockInfoData 增加的行业控制
2	（对数）市值中性化	由 updateStockInfoData 增加的行业、_中性化用市值

- 默认：0（不处理）
- 注意：

（1） ***如果需要进行多个中性化处理，取值给对应方法取值的和即可***。1 表示行业中性化；3=1+2，表示进行一级行业中性化以及（对数）市值中性化。

5.1.11FOrthType

- 含义：正交化方法
- 取值：
- +

取值	含义	备注
0,nil	不处理	缺省
1	对称正交化	
2	规范正交化	
3	施密特正交化	

默认：0（不处理）

5.1.12FGroupType

- 含义：分组方式
- 取值：

实际值	含义	备注

0	按个数分组	比如：选前 10 名为一组
1	按百分位分组	比如：20%为一组
2	按值区间分组	比如：因子值小于 20 的分一组
3	按权重分组	比如：分 4 组，每组累计权重为 25%

- 说明：

  此参数要与 FGroups 一起配合使用

（1）取值为 0： FGroups 的分组代表个数（个数是从 1 开始）。例如此时 FGroups:=”1,9;10,19;20,29;30,39;40,49”;，那么代表第一组是从 1-9，第二组 10-19，第三组是 20-29，第四组 30-39，第五组 40-49。

（2）取值为 1： FGroups 的分组代表百分位。例如此时 FGroups:="0,20;20,40;40,60;60,80;80,100";，那么代表第一组是 0-20%,第二组 20%-40%，第三组 40%-60%，第四组 60%-80%，第五组 80%-100%。

（3）取值为 2：

✧ 含义：Fgroups 的分组代表综合因子所处的区间***(注：关于如何按原值分组见 FAQ)***，

✧ 业务场景：一般在因子值为一些 分类、得分时使用，比如：***基金的五星级评分，只可以为 1 到 5 的整数，或者因子为是否超预期-1、0、1。***

✧ 业务范例：

例如：此时 Fgroups:=”0,20;20,40” ***（需结合标准化方式使用，这里遵从规范第一组综合因子值最大）***，那么**代表第一组的综合因子在区间[20,40)中，第二组的综合因子在区间[0,20)中**。

注：字符串或者数组输入模式下，对于 FGroupType=2，Fgroups 从小到大给，定义第一组取最后定义，与其他分组方式不同

（4）取值为 3： FGroups 的分组代表每组累计权重。例如 FGroups:="0,30;30,60;60,100"，代表第一组累计权重范围为 0-30%，第二组累计权重范围为 30%-60%，第三组累计权重范围为 60%-100%。

（5）关于的开闭区间是如何取的，详见因子分组

- 默认：1（按百分位分组）

5.1.13FGroups

- 含义：分组的百分位|分组序号|分组数|分组值区间
- 取值：支持 数字、字符串、整数及整数字符串

”num1,num2;num2,num3;num3,num4;…”：分组百分位或分组序号

Int，“int”：分组数

array(num1,num2,num3)：分组节点

取值与含义需结合成员变量 FGroupType 配合解释如下：

Fgroups	取值范例	个数分组 FGroupType=0	百分位分组 FGroupType=1	值区间分组 FGroupType=2	权重分组 FGroupType=3
字符串	"1,9;10,19;20,29"	数量分组，比如 1-9 表示前 9 个在第一组	百分位间隔，比如 1-9 表示 1%-9%在第一组	值间隔，遵循大的放在第一组。比如综合	累计权重范围，1-9 表示累计权重范围

				因子位于[1,9)之间表示第三组	在 1%-9%的为第一组
整数	3	×	分组数，个股等分法，比如三分位	分组数，值等宽法，根据最大最小值划分区间为三份(等宽)	分组数，权重等分法，根据分组数确认每组累计权重（33.33%）
字符串型整数	"3"	×	同上	同上	同上
数组（一般不建议使用）	array(0,20,40,60)	×	百分位间隔，比如 0-20 表示前 20%在第一组	值间隔，遵循大的放在第一组。比如综合因子位于[0,20)之间表示第三组	累计权重范围，0-20 表示累计权重小于 20%的为第一组

- 使用范例：

（1） 对于类似 N 分位，可以使用本参数和 FGroupType 一起配合使用。如：

```
//三分位
FGroupType:=1;
FGroups:="0,1/3*100;1/3*100,2/3*100;2/3*100,100";  或  Fgroups:=3;  或  Fgroups:="3";
//五分位
FGroupType:=1;
FGroups:="0,20;20,40;40,60;60,80;80,100";或 Fgroups:=5;或 Fgroups:="5";
```

（2）

值区间	组名	含义
<0	第 3 组	综合因子值<0
[0,1)	第 2 组	>=0 且<1
>=1	第 1 组	>=1 分组

按如上方式进行值区间分组：

```
FgroupType := 2;
Fgroups :=array(0,1);
或 Fgroups :="-inf,0;0,1;1,inf";
```

- 特殊说明：

（1）按个数分组 FGroupType=0 时， -1 表示最后一个，-10 表从后往前数第 10 个， 用于在不知道分组总数时使用。

- 默认： "0,20;20,40;40,60;60,80;80,100"（五分位）

5.1.14FGroupHy

- 含义：分组时是否考虑行业因素
- 取值：

取值	含义	说明	备注
-1	分组时不考虑行业	分组时，直接按照综合因子得分高低去分组，不考虑行业影响。	默认
1	按行业分组	（1）先在各行业分别分组，再把各行业中对应组的股票合并起来即为这组的股票 （2）如果某个行业的股票数少于分组数，依然对其分组	
2	按行业分组，行业样本小于组数时不再分组	（1）先在各行业内分别分组，再把各行业中对应组的股票合并起来即为这组的股票。 （2）如果某个行业的股票数少于分组数，那么每个组中都包含这些股票。	

- 说明：
  - 注意事项：
    （1） ***分组时考虑行业 Fgrouphy=1、2 只适用于比例分组，暂时不适用于数量分组和值区间分组***
    （2） ***分组时考虑行业 Fgrouphy=2，不适用于权重分组***
    参见 FGroupType。
- 默认：-1（分组时不考虑行业）

5.1.15FIndustryType

- 含义：行业分类
- 取值：

取值	含义	备注
1	申万一级行业	框架内部自动按照对应行业计算行业。
2	证监会一级行业	
3	中证一级行业	
4	申万一级行业扩展	将申万一级行业非银金融细再分成券商、保险、多元金融三个行业
-1	用户自定义分类方式	选择这种方式时，需要在重写 IndustrializeData 时，返回结果中添加”行业”这一列。

- 说明：该行业分类作用于分组时考虑行业场景和行业资金分配方法为行业中性场景
- 默认：1

5.1.16FIndustryEval

- 含义：行业表达式
- 说明：该行业表达式只作用于因子值行业中性化
  做行业中性化时，获取个券行业的表达式，系统环境包括当前个券和日期
- 默认：’MF_Industry(6,1)’（2021 版申万一级行业）
- 备注：废弃属性，已失效

5.1.17FConstr

- 含义：样本剔除条件表达式
- 说明：
  比如，你要剔除掉每股收益小于等于 0 的股票，那么只需要采用如下形式修改这个属性：
  对象名.FConstr:="Last12MData(RDate,9900000)<=0";
- 默认：空字符串（不剔除任何样本）

5.1.18FIndustryMethod

- 含义：行业资金分配方法
- 取值：

取值	含义	备注
-1	不做行业中性处理	按属性 FrateType 设置的或者自己传入的比例(%)分配资金。
1	行业中性	

- 说明：

如果选择行业中性处理(FIndustryMethod=1)：

（1） 系统自动从基准代码 FIndexId 中获得指定日基准的权重

（2） 用户需要同时设置行业分类 FIndustryType

- 默认：-1（不做行业中性处理）

5.1.19FRateType

- 含义：多因子中每个分组中的个股资金分配方式
- 取值：

取值	含义	备注
-2	用户自定义自定组资金配比	用户重写当期指定股票池的比例，需重写 AllocateWeightByGroup
-1	用户自定义资金配比	用户重写当期所有组的比例，需重写整个 AllocateWeight
0	总股本加权，即总市值加权	$W_i = \frac{P_i \times TS_i}{\sum_{i=1}^{N} P_i \times TS_i} \times 100$
1	流通股本加权，即流通市值加权	$W_i = \frac{P_i \times NS_i}{\sum_{i=1}^{N} P_i \times NS_i} \times 100$
4	等权重	$W_i = \frac{1}{N} \times 100$ N：组合中所有证券个数
5	等股数	$W_i = \frac{P_i}{\sum_{i=1}^{N} P_i} \times 100$
6	流通市值平方根加权	$W_i = \frac{\sqrt{P_i \times NS_i}}{\sum_{i=1}^{N} \sqrt{P_i \times NS_i}} \times 100$
7	总市值平方根加权	$W_i = \frac{\sqrt{P_i \times TS_i}}{\sum_{i=1}^{N} \sqrt{P_i \times TS_i}} \times 100$

31	自由流通股本加权，即自由流通市值加权	$W_i = \frac{P_i \times FS_i}{\sum_{i=1}^{N} P_i \times FS_i} \times 100$ 根据指数公司指数权重预估
61	基准权重加权	基准指数成分股权重，来自 GetBenchmarkWeight 的个股比例

- 默认：4（等权重）

5.1.20FBuyRatio

- 含义：开仓费率(%)
- 默认：0.1

5.1.21FSellRatio

- 含义：平仓费率(%)
- 默认：0.1

5.1.22FGridComputing

- 含义：是否使用网格
- 说明：只有开通了天软网格功能的用使用网格。如果你想开通网格可以联系你的客服代表。
- 默认：False

5.1.23FGridNo

- 含义：网格数
- 说明：开通的网格数，可参阅相关的网格文档中提到的网格数，测试你的网格数据：
- 默认：5

5.1.24FisTradeDay

- 含义：是否考虑停牌
- 说明：指在计算因子值样本中的停牌证券是否去掉。
- 取值：不对 FisTradeDay 进行设置默认是 1.

取值	含义	备注
1	去掉停牌证券	
0	不去掉停牌证券	

5.1.25FpriceType

实际值	含义	备注
2	收盘价	
3	日均价	

4	开盘价	
5	系统昨收	交易所公布的系统昨收（复权后的）

这个属性与天软回测框架里的 FpriceType 设置是一致的。

5.1.26FVolModType

- 含义：成交量取整模式
- 取值

实际值	含义	备注
0	成交量不取整	
-1	成交量自适应取整	根据证券类型的不同，采取不同的取整模式

5.1.27FDividendType

- 含义：是否分红再投资
- 取值

实际值	含义	备注
0	分红不再投资	股票类策略一般采取
1	分红再投资	基金类策略一般采用

5.1.28FAllotmentType

- 含义：是否参与配股
- 取值

实际值	含义	备注
0	不自动参与配股	
1	自动参与配股	

5.1.29FBacktestType

- 含义：使用的回测方法
- 取值

实际值	含义	备注
0	使用回测框架清算	见回测时所做的假设
1	简单清算	简单清算主要适用非保证金交易，只支持满仓回测和买卖费率，通过属性 FBuyRatio 和 FSellRatio 设置费率。

5.1.30FBMType

- 含义：基准类型
- 取值

实际值	含义	备注
0	市场指数作为基准	缺省
1	复合基准	

- 默认：0
- 说明：

  当用户设置 FBMType:=0;时说明基准类别为单个基准，直接设置基准指数代码 FIndexId；

  当用户设置 FBMType:=1;时说明基准类别为复合基准，复合基准指的是采用多个市场指数按照一定的比例进行加权得到最终的基准，此时需要设置 FBMDetail 属性。

  注意：当前只支持基准的收益率采用复合基准模式，暂不支持基准行业 采用复合模式（默认仍从 FIndexID 取）

5.1.31FBMDetail

- 含义：复合基准成分
- 说明： 当用户设置基准类型为复合基准时需要给出对应的市场指数及其比例(%)
- 取值：二维数组

含义	类型	非空
代码	String	是
比例(%)	Numeric	是

- 默认：array();
- 说明： 当用户选择使用复合基准设置 FBMType:=1;时，用户还需要设置 FBMDetail，给出复合基准的成分。

  例如：采用沪深 300 指数与上证指数，按照沪深 300 比例为 60%，国债指数比例类 40%的方式进行加 权得到最终基准，那么设置

  ```
  FBMDetail:=array(("代码":"SH000012","比例(%)":40), ("代码":"SH000300","比例(%)":60));
  ```

- 注意：复合基准成分的比例(%)之和需为 100；

5.2 天软多因子常见方法

5.2.1 方法汇总

所属分类	接口类型	属性	说明	备注
流程相关	BackTest		多因子框架总流程，包括指标计算、极值处理、数量化回测这些部分	得到基于回测的评价部分都要先调用这个函数

	CalculateFactorValue		多因子框架前部分流程，即回测之前的流程	得到指标相关的量的时候，可以只调用这个。
用户访问	GetTimeSeries	可重写	调仓周期	
	GetSamples	可重写	得到 T 期证券列表	
	GetFactors	可重写	得到指定日的因子配置	
	GetFactorParams	可重写	得到指定日的自定义因子公式参数	
因子查询	GetGroupFactorValue	多态	得到调仓日因子的值&分组	
	factorValue_QueryfactorGroupfullTable			
	factorValue_QueryfactorGroupfullSturct		查询指定日分组详情(按组名聚合)	
	factorValue_QueryfactorfullTable		查询指定期因子暴露矩阵，分组前数据	
检验查询	GetGroupReturn		得到每组的收益&/累计收益率	按日、按月
	GetGroupTurnOver		得到各组在每个调仓时点 T-1 期到 T 期的换手率	
	GetTrailingReturn		因子收益率检验	
	GetSignificanceStatistics		因子显著性检验	
	GetLongShortStatistics		因子区分度检验	
	GetInformationCoefficient		因子延续性检验 IC	
	GetSignalStability		因子信号稳定性检验	
	GetLinearEvaluationResult		使用最小二乘参数估计（带常数项）检验各因子的贡献度	
	GetPrincipalComponentResult		主成分分析法检验各因子的贡献度	
外部数据	updateStockInfoData		接入股票基本信息，含市值和行业等	
	IndustrializeData		分组和行业中性配置时个股自定义行业	只有当自定义行业且分组考虑行业时候，才需要重载此方法
	GetBenchmarkWeight		设置基准行业分类及个股在基准中的权重	只有当自定义行业且行业中性，才需要重载此方法
	GetBenchmarkDailyReturn	可重写	返回基准收益率序列	
数据处理	FillMissData		缺失值处理	

	NormalizeData	可重写，但重写概率低	极值处理	
	StandarizeData		标准化处理	
	NeutralizeData		中性化处理	
	OrthData		正交化处理	
	OrthData1		正交化处理	
	ScoreData		综合因子计算	
	GroupData	同上	分组处理	
	AllocateWeight		个股资金分配处理	
	AllocateWeightByGroup		T 期指定组个券分配处理	注意与 AllocateWeight 的区别
评价方法时间段显示相关	GetShowPeriod	可重写	我们默认在评价方法中显示，近一年、近两年、近三年、整体这四个时间段。如果用户想显示其它的时间段，可重写此函数。	

5.3 流程相关接口函数

5.3.1 BackTest

- 定义：BackTest()
- 参数：无
- 返回：无
- 说明：

（1） 多因子框架总流程。从第一步得到调仓周期一直到量化回测这一步。类似回测框架中的 BackTest()

（2） **调用其他接口前，一般需要先调用 BackTest 进行回测，才能调用其他接口。**

5.3.2 CalculateFactorValue

- 定义： CalculateFactorValue()
- 参数：无
- 返回：无
- 说明：

（1） 多因子框架回测前的流程。当只需要得到因子计算相关的数据时，可以只调用这个流程。当到用户仅仅需要得到因子计算相关的数据时，并不涉及到回测清算过程，为了加快速度，就可以调用这个流程函数，而不是 BackTest()。

（2） 一般而言，在做多因子的时候一定要做回测。因此，***本模型较少直接被调用***

- 范例：

```
FactorArr:=array(
("因子名称":"每股收益","因子公式":"Last12MData(RDate,9900000)","因子方向":1,"因子比例(%)":60),
("因子名称":"收盘","因子公式":"StockClose(EndT)","因子方向":0,"因子比例(%)":40));
obj:=CreateObject('TSMultiFactor');
obj.FBegT:=20131231T;
obj.FEndT:=20140530T;
obj.FFactorArr:=FactorArr;
//注：此处仅仅是得到因子指标的计算值，故不要掉总过程 BackTest();
obj.CalculateFactorValue();
//得到因子值和分组数据
return obj.GetGroupFactorValue();
```

5.4 用户访问接口函数

主要提供默认的因子计算单元设置，支持用户重载

5.4.1 GetTimeSeries

- 含义：得到因子调仓日期
- 定义：GetTimeSeries():Array
- 参数：无
- 返回：一维日期数组
- 备注：

（1） 系统默认按照调仓周期 FCycle 获得开始日 FBegT 和截止日 FEndT 之间的调仓周期。常见的调仓周期如：每月末调整一次

（2） **此方法可重写**。若 TSMultiFactor 类中的缺省调仓日期方法无法满足需求，可在子类重写该方法

- 调用基类范例：

```
obj:=CreateObject('TSMultiFactor');
obj.FBegT:=20131231T;
obj.FEndT:=20140530T;
obj.FCycle:=cy_month();
//得到多因子调仓日
return obj.GetTimeSeries();
```

- 重载范例：

```
obj:=CreateObject('MutiFactorclass');
obj.FBegT:=20131231T;
obj.FEndT:=20140530T;
return obj.GetTimeSeries();
```

```
Type MutiFactorclass=class(TSMultiFactor)
   Function GetTimeSeries();override;
   Begin
      ov:=BackupSystemParameters2();
      SetSysParam(pn_stock(),"SH000001");
      SetSysParam(pn_cycle(),cy_day());
      r:=array();
      n:=0;
      For nI:=FBegT to FEndT do
      Begin
              //如果是周一并且是交易日那么这天调仓
         if DayOfTheWeek(nI)=1 and istradeday(nI) then
         Begin
            r[n]:=nI;
            n++;
         End
         else continue;
      End
      return r;
   End;
end;
```

5.4.2 GetSamples

- 含义：得到调仓日 T 的初始样本
- 定义：GetSamples(vEndT:DateTime):Array
- 参数：vEndT：调仓日。
- 返回：调仓日证券列表，一维股票代码数组
- 备注：用户可重写 GetSamples(vEndT)得到每期的样本
- 调用基类范例：

```
obj:=CreateObject('TSMultiFactor');
obj.FBegT:=20131231T;
obj.FEndT:=20140530T;
DayArr:=obj.GetTimeSeries();
   //得到调仓日初始样本
return obj.GetSamples(DayArr[0]);
```

- 重写范例：

```
obj:=CreateObject('MutiFactorclass');
obj.FBegT:=20131231T;
obj.FEndT:=20140530T;
DayArr:=obj.GetTimeSeries();
return obj.GetSamples(DayArr[0]);
```

```
Type MutiFactorclass=class(TSMultiFactor)
   Function GetSamples(vEndT);override;
   Begin
      r:=GetBKByDate('SH000016',vEndT);
      return r;
   End
End
```

5.4.3 GetFactors

- 含义：得到调仓日 T 的因子库
- 定义：GetFactors(EndT:DateTime):Array();
- 参数：EndT：调仓日 T。
- 返回：二维数组，得到调仓日 T 的因子库。具体返回值和 FFactorArr 定义一致
- 说明：

（1） 如果在不同期，使用同一个因子，只需要在 FFactorArr 设置即可。在验证因子是否有效的回测中，一般使用固定因子

（2） 如果在不同期，要使用不同的因子，需要重载本方法。在实际根据因子进行投资和动态跟踪中，可能需要根据某些因子是否有效/失效，因子是正向贡献/反向贡献，需要动态的调整因子，此时一般使用动态因子

- 调用基类范例：

```
obj:=CreateObject('TSMultiFactor');
obj.FBegT:=20131231T;
obj.FEndT:=20140530T;
obj.FFactorArr:=array(
      ("因子名称":"my_pe","因子公式":"1/stockpe(EndT)","因子方向":1,"因子比例(%)":50),
      ("因子名称":"近一月涨幅(%)","因子公式":"stockzf(incmonth(endt,-1)+1,endt)","因子方向":1,"因子比例(%)":50));
DayArr:=obj.GetTimeSeries();
return obj.GetFactors(DayArr[0]);
```

- 重写范例：

```
obj:=CreateObject('MutiFactorclass');
obj.FBegT:=20131231T;
obj.FEndT:=20140530T;
DayArr:=obj.GetTimeSeries();
return obj.GetFactors(DayArr[0]);

Type MutiFactorclass=class(TSMultiFactor)
   Function GetFactors(EndT);override;
```

```
    Begin
        //假设我们偶数月使用估值因子，奇数月使用动量因子（仅为演示如何重载 GetFactors，无实际意义）
        A:=MonthOf(EndT) div 2;
        If A<>0 then
            r:=  array((" 因 子 名 称 ":" 近 一 月 涨 幅 (%)"," 因 子 公 式 ":"stockzf(incmonth(endt,-1)+1,endt)","因子方向":0,"因子比例(%)":100.00));
        Else
            r:= array(("因子名称":"my_pe","因子公式":"1/stockpe(Endt)","因子方向":1,"因子比例(%)":100.00));
        Return r;
    End
end
```

5.4.4 GetFactorParams

- 含义：得到调仓日 T 自定义因子公式参数
- 定义：GetFactorParams(EndT: DateTime,FactorParam:Array) :Array();
- 参数：
  - EndT：调仓日 T。
  - FactorParam：通过 config_Set('因子公式参数',value)配置的静态自定义因子公式参数。参看 静态参数
- 返回：数组，自定义因子公式参数
- 说明：该函数一般在实现动态因子公式参数时需要重写，默认是直接返回设定的静态参数。用户在实现动态参数时，需要根据实际进行当期因子公式参数处理，返回合法的自定义参数数组。其中行索引必须是字符串索引，因为代表着因子公式参数名称。

5.5 因子查询相关接口

5.5.1 GetGroupFactorValue

- 含义：查询在不同调仓日每个因子的数值、极值处理后值、标准化后值、综合因子得分、组名等

5.5.1.1 所有调仓日因子值

- 含义：得到因子指标计算值。因子各个调仓日期因子计算值、极值处理后值、标准化后值、综合因子得分、组名、资金分配比例(%)和下期收益
- 定义：GetGroupFactorValue():VarArray
- 参数：无
- 返回：二维数组，返回字段如下：

字段	类型	单位	备注
代码	String		证券代码
名称	String		证券名称
截止日	DateTime		调仓日
报告期	Numeric		调仓日前最新的报告期
因子名称	Numeric		如果因子名称是 PE,那么这里的名称就是 PE
因子名称_奇异处理后	Numeric		PE_奇异处理后
因子名称_标准化后	Numeric		PE_标准化后
综合因子	Numeric		得到综合因子得分，是分组的依据
组名	String		第 1 组…
数值	Numeric		临时值，回测框架内部使用
比例(%)	Numric	%	证券在调仓日目标配置权重

单因子和多因子的综合因子的差异，请参看章节单因子与多因子的处理差异

5.5.1.2 指定调仓日因子值

- 含义：指定调仓日因子值
- 定义： GetGroupFactorValue(vEndT) :VarArray
- 参数：vEndT：指定调仓日
- 返回：同#所有调仓日因子值

5.5.1.3 指定组 ID 因子值

- 含义：指定组因子值
- 定义： GetGroupFactorValue(GroupID) :VarArray
- 参数：GroupID：组号，1 代表第 1 组，2 代表第 2 组，依次类推；GroupID：组号，-1 代表倒数第 1 组，-2 代表倒数第 2 组，依次类推
- 返回：同#所有调仓日因子值

5.5.1.4 指定组名因子值

- 含义：指定组因子值
- 定义： GetGroupFactorValue(GroupName) :VarArray
- 参数：GroupName：组名，”第 1 组”代表第 1 组，”第 2 组”代表第 2 组，依次类推
- 返回：同#所有调仓日因子值

5.5.1.5 指定调仓日指定分组的因子值

- 含义：指定调仓日指定分组的因子值

- 定义： GetGroupFactorValue(vEndT,GroupID) :VarArray
- 参数：

  （1）vEndT:日期，当前时点。

  （2）GroupID：组号，1 代表第 1 组，2 代表第 2 组，依次类推；-1 代表倒数第 1 组，-2 代表倒数第 2 组，依次类推
- 返回：同#所有调仓日因子值

5.5.1.6 GetGroupFactorValue 常见应用范例

```
FactorArr:=array(
("因子名称":"每股收益","因子公式":"Last12MData(RDate,9900000)","因子方向":1,"因子比例(%)":60),
("因子名称":"收盘","因子公式":"StockClose(EndT)","因子方向":0,"因子比例(%)":40));
obj:=CreateObject('TSMultiFactor');
obj.FBegT:=20131231T;
obj.FEndT:=20140530T;
obj.FFactorArr:=FactorArr;
obj.BackTest();
DayArr:=obj.GetTimeSeries();
//所有调仓日,所有组的因子值
t1:=obj.GetGroupFactorValue();
//指定调仓日,所有组的因子值
t2:=obj.GetGroupFactorValue(DayArr[0]);
//所有调仓日,第一组的因子值
t3:=obj.GetGroupFactorValue(1);
//所有调仓日,第一组的因子值
t4:=obj.GetGroupFactorValue("第 1 组");
//所有调仓日,倒数第一组的因子值
t5:=obj.GetGroupFactorValue(-1);
//指定调仓日,第一组的因子值
t6:=obj.GetGroupFactorValue(DayArr[0],1);
return array(t1,t2,t3,t4,t5,t6);
```

5.5.2 factorValue_QueryfactorGroupfullTable

- 含义：查询指定日分组详情，分组后数据
- 定义：factorValue_QueryfactorGroupfullTable(groupid:Int Or String,endt:Date, Querytyp:Int):VarArray
- 参数：

  Groupid:组名，或者组号，为 nil，表示取所有组

  组号，1 代表第 1 组，2 代表第 2 组，依次类推；GroupID：组号，-1 代表倒数第 1 组，-2 代表倒数第 2 组，依次类推

组名，”第 1 组”代表第 1 组，”第 2 组”代表第 2 组，依次类推

Endt：调仓日，因子计算日，为 nil，表示不对日期做限制

Querytyp：查询类型，默认 nil 等价 Querytyp =3

选项	功能	说明
1	分组详情	字段：代码、截止日、组名、比例(%)
2	分组详情 + 描述因子+综合因子	
3	分组详情 + 数据处理过程 + 综合因子	历史分组后数据，查询结果同 GetGroupFactorValue

- 范例：

// QueryTyp=1:



数组数据

	代码	截止日	组名	比例(%)
0	SZ300502	45911	第1组	1.69
1	SH605117	45911	第1组	1.69
2	SH605499	45911	第1组	1.69
3	SH600809	45911	第1组	1.69
4	SH600519	45911	第1组	1.69
5	SZ300394	45911	第1组	1.69
6	SZ002027	45911	第1组	1.69
7	SZ300308	45911	第1组	1.69
8	SZ300628	45911	第1组	1.69
9	SZ300033	45911	第1组	1.69

//QueryTyp=2：（净资产收益率(%)ROE 是完成数据处理之后的值）



数组数据

	代码	名称	截止日	报告期	净资产收益率(%)ROE	总资产报酬率(%)ROA	行业	行业名称	_市值	_标准化用市值	_中性化用市值	_回归加权用市值	综合因子	下期收益	组名	比例(%)
0	SZ300502	新易盛	45911	20250630	2.48	2.27	SWHY730000	申万通信	33,626,907.00	0.01	1.39	0.01	2.35	0.00	第1组	1.69
1	SH605117	德业股份	45911	20250630	2.48	2.27	SWHY630000	申万电力设备	6,737,223.50	0.00	-0.48	0.00	2.35	0.00	第1组	1.69
2	SH605499	东鹏饮料	45911	20250630	2.48	2.27	SWHY340000	申万食品饮料	15,588,949.71	0.00	0.50	0.00	2.35	0.00	第1组	1.69
3	SH600809	山西汾酒	45911	20250630	2.48	2.27	SWHY340000	申万食品饮料	25,363,056.18	0.01	1.07	0.00	2.35	0.00	第1组	1.69
4	SH600519	贵州茅台	45911	20250630	2.48	2.27	SWHY340000	申万食品饮料	190,783,367.26	0.04	3.04	0.01	2.35	0.00	第1组	1.69
5	SZ300394	天孚通信	45911	20250630	2.46	2.27	SWHY730000	申万通信	15,408,428.39	0.00	0.48	0.00	2.35	0.00	第1组	1.69
6	SZ002027	分众传媒	45911	20250630	2.33	2.27	SWHY720000	申万传媒	11,958,141.37	0.00	0.19	0.00	2.29	0.00	第1组	1.69
7	SZ300308	中际旭创	45911	20250630	2.04	2.27	SWHY730000	申万通信	48,638,704.71	0.01	1.83	0.01	2.18	0.00	第1组	1.69
8	SZ300628	亿联网络	45911	20250630	1.99	2.27	SWHY730000	申万通信	2,805,426.35	0.00	-1.50	0.00	2.16	0.00	第1组	1.69
9	SZ300033	同花顺	45911	20250630	1.91	2.27	SWHY710000	申万计算机	10,122,350.55	0.00	-0.01	0.00	2.13	0.00	第1组	1.69
10	SZ000568	泸州老窖	45911	20250630	1.86	2.27	SWHY340000	申万食品饮料	20,635,889.95	0.00	0.82	0.00	2.11	0.00	第1组	1.69

//QueryTyp=3：（净资产收益率(%)ROE 是因子原值，详见 GetGroupFactorValue）



数组数据

	代码	名称	截止日	报告期	净资产收益率(%)ROE	总资产报酬率(%)ROA	行业	行业名称	_市值	_标准化用市值	_中性化用市值	_回归加权用市值	净资产收益率(%)ROE_缺失值处理后	总资产报酬率(%
0	SZ300502	新易盛	45911	20250630	48.91	38.99	SWHY730000	申万通信	33,626,907.00	0.01	1.39	0.01	48.91	
1	SH605117	德业股份	45911	20250630	34.61	19.28	SWHY630000	申万电力设备	6,737,223.50	0.00	-0.48	0.00	34.61	
2	SH605499	东鹏饮料	45911	20250630	45.46	17.42	SWHY340000	申万食品饮料	15,588,949.71	0.00	0.50	0.00	45.46	
3	SH600809	山西汾酒	45911	20250630	34.40	22.41	SWHY340000	申万食品饮料	25,363,056.18	0.01	1.07	0.00	34.40	
4	SH600519	贵州茅台	45911	20250630	37.69	30.42	SWHY340000	申万食品饮料	190,783,367.26	0.04	3.04	0.01	37.69	
5	SZ300394	天孚通信	45911	20250630	33.89	30.58	SWHY730000	申万通信	15,408,428.39	0.00	0.48	0.00	33.89	
6	SZ002027	分众传媒	45911	20250630	32.58	21.98	SWHY720000	申万传媒	11,958,141.37	0.00	0.19	0.00	32.58	
7	SZ300308	中际旭创	45911	20250630	29.84	21.39	SWHY730000	申万通信	48,638,704.71	0.01	1.83	0.01	29.84	
8	SZ300628	亿联网络	45911	20250630	29.39	26.29	SWHY730000	申万通信	2,805,426.35	0.00	-1.50	0.00	29.39	

5.5.3 factorValue_QueryfactorGroupfullSturct

- 含义：查询指定日分组详情(按组名聚合)，分组后数据

- 定 义 ： factorValue_QueryfactorGroupfullSturct(groupid:Int Or String,endt:Date, Querytyp:Int):VarArray
- 参数：同 factorValue_QueryfactorGroupfullTable
- 返回：factorValue_QueryfactorGroupfullTable 结果按 组名聚合后的结果
- 范例：



5.5.4 factorValue_QueryfactorfullTable

- 含义：查询指定期因子暴露矩阵，分组前数据
- 定义：factorValue_QueryfactorfullTable(Endt:Date, QueryTyp:Int):VarArray
- 参数：

  Endt：

  指定期，因子计算日，为 nil，表示不对日期做限制

  Querytyp：查询类型，默认 nil 等价 Querytyp =3

选型	功能	说明
1	查询因子计算过程	全连接，如果是缺失剔除的，存在为 nil 的。
2	查询描述因子	返回数据处理后的因子值，也有综合因子/行业/下期收益 字段
3	查询因子计算过程 + 综合因子	兼容历史分组前数据，查询结果同 GetGroupFactorValue（不含分组、比例(%)），不展示缺失值填充过程，也有综合因子 行业字段。
4	查询综合因子	有 行业/下期收益 字段

- 范例：

```
// Querytyp=1（净资产收益率(%)ROE 指输入因子原值）
```


数组数据

	代码	名称	净资产收益率(%)ROE_正交化后	总资产报酬率(%)ROA_正交化后	净资产收益率(%)ROE	总资产报酬率(%)ROA	行业	行业名称	_市值	_标准化用市值	_中性化用市值	_回归加权用市值
0	SZ000001	平安银行				0.75	SWHY480000	申万银行	22,995,636.77	0.00	0.95	0.00
1	SH688506	百利天恒	-2.63	-2.77	-74.20	-29.00	SWHY370000	申万医药生物	3,580,610.98	0.00	-1.22	0.00
2	SZ002129	TCL中环	-2.63	-2.43	-41.21	-8.79	SWHY630000	申万电力设备	3,438,073.86	0.00	-1.27	0.00
3	SH688599	天合光能	-2.63	-1.84	-29.50	-5.52	SWHY630000	申万电力设备	3,787,737.09	0.00	-1.15	0.00
4	SH600588	用友网络	-2.63	-2.58	-29.30	-9.60	SWHY710000	申万计算机	5,251,925.02	0.00	-0.77	0.00
5	SZ000002	万 科A	-2.63	-1.60	-26.94	-4.16	SWHY430000	申万房地产	6,636,300.95	0.00	-0.50	0.00

```
// Querytyp=2（净资产收益率(%)ROE 是完成数据处理之后的值）
```

数组数据

	代码	名称	截止日	报告期	净资产收益率(%)ROE	总资产报酬率(%)ROA	行业	行业名称	_市值	_标准化用市值	_中性化用市值	_回归加权用市值	综合因子	下期收益
0	SZ000002	万　科A	45911	20250630	-2.63	-1.60	SWHY430000	申万房地产	6,636,300.95	0.00	-0.50	0.00	-2.01	0.00
1	SH600900	长江电力	45911	20250630	0.65	0.23	SWHY410000	申万公用事业	67,220,318.21	0.01	2.20	0.01	0.39	0.00
2	SZ000063	中兴通讯	45911	20250630	0.00	-0.20	SWHY730000	申万通信	17,615,672.98	0.00	0.64	0.00	-0.12	0.00
3	SZ000157	中联重科	45911	20250630	-0.35	-0.29	SWHY640000	申万机械设备	5,196,756.76	0.00	-0.79	0.00	-0.31	0.00
4	SZ000425	徐工机械	45911	20250630	0.06	-0.15	SWHY640000	申万机械设备	9,159,100.92	0.00	-0.12	0.00	-0.07	0.00
5	SH600009	上海机场	45911	20250630	-0.55	-0.30	SWHY420000	申万交通运输	6,185,875.74	0.00	-0.58	0.00	-0.40	0.00
6	SH600010	包钢股份	45911	20250630	-1.02	-0.82	SWHY230000	申万钢铁	7,782,437.98	0.00	-0.31	0.00	-0.90	0.00
7	SZ000568	泸州老窖	45911	20250630	1.86	2.27	SWHY340000	申万食品饮料	20,635,889.95	0.00	0.82	0.00	2.11	0.00
8	SH600795	国电电力	45911	20250630	0.14	-0.61	SWHY410000	申万公用事业	9,256,686.30	0.00	-0.11	0.00	-0.31	0.00
9	SH600015	华夏银行	45911	20250630	-0.22	-0.75	SWHY480000	申万银行	11,248,060.73	0.00	0.12	0.00	-0.58	0.00

```
// Querytyp=4
```

数组数据

	截止日	代码	名称	综合因子	下期收益
0	45911	SZ000002	万　科A	-2.01	0.00
1	45911	SH600900	长江电力	0.39	0.00
2	45911	SZ000063	中兴通讯	-0.12	0.00
3	45911	SZ000157	中联重科	-0.31	0.00
4	45911	SZ000425	徐工机械	-0.07	0.00
5	45911	SH600009	上海机场	-0.40	0.00
6	45911	SH600010	包钢股份	-0.90	0.00
7	45911	SZ000568	泸州老窖	2.11	0.00

5.6 检验查询相关接口

5.6.1 GetGroupReturn

- 含义：得到各组的某个周期的收益情况
- 定义：GetGroupReturn(pType,cy);
- 参数：

（1）pType：数值类型，返回的收益类型

取值	含义	备注
0	收益率(%)	返回的表格格式相同
1	累计收益率(%)	
2	超额收益率(%)	
3	超额累计收益率(%)	
4	第一组 vs 最后一组超额收益率(%),第一组 vs 最后一组累计超额收益(%)	返回的表格格式有别于前四种
5	第一组 vs 最后一组超额收益率(%),第一组 vs 最后一组累计超额收益(%)以及相对净值	相对于 4，主要是增加了”相对净值”字段

（2）Cy：数据显示周期

取值	含义	备注
-1	得到按调仓周期分的阶段收益	
Tsl 中的周期字符串	得到按对应周期分割的收益	eg：cy_month()

- 返回：

（1） 当 pType=0 或 1 或 2 或 3 时，返回如下形式数组：

字段	类型	备注
开始日	DateTime	
截止日	DateTime	
交易点数	Integer	开始日到截止日之间的市场交易天数。
基准	Numeric	当 pType=0 或 2 时，代表基准的在开始日到截止日的收益率(%)； 当 pType=1 或 3 时，代表基准在截止日的累计收益率(%)；
第 1 组	Numeric	第 1 组对应的收益率
……	Numeric	……
第 n 组	Numeric	第 n 组对应的收益率

（2）当 pType=4 时，返如下形式数组：

字段	类型	备注
开始日	DateTime	
截止日	DateTime	
交易点数	Integer	开始日到截止日之间的市场交易天数。
第一组 vs 最后一组	Numeric	第一组减去最后一组的收益率(%)
第一组 vs 最后一组->累计	Numeric	第一组减去最后一组的累计收益率(%)

- 范例：

```
FactorArr:=array(
("因子名称":"每股收益","因子公式":"Last12MData(RDate,9900000)","因子方向":1,"因子比例(%)":60),
("因子名称":"收盘","因子公式":"StockClose(EndT)","因子方向":0,"因子比例(%)":40));
obj:=CreateObject('TSMultiFactor');
obj.FBegT:=20131231T;
obj.FEndT:=20140530T;
obj.FIndexId:='SH000016';
obj.FCycle:=cy_month();
obj.FFactorArr:=FactorArr;
obj.FGroupType:=1;
obj.FGroups:="0,1/3*100;1/3*100,2/3*100;2/3*100,100";
Ret:=obj.BackTest();
return array(
```

```
'日累计收益率(%)':obj.GetGroupReturn(1,cy_day()),
'日累计超额收益率(%)':obj.GetGroupReturn(3,cy_day()),
'月度收益率(%)':obj.GetGroupReturn(0,cy_month()),
'月度超额收益率(%)':obj.GetGroupReturn(2,cy_month()),
'多空月度收益率':obj.GetGroupReturn(4,cy_month()),
'按调仓日分割的超额收益率(%)':obj.GetGroupReturn(2,-1),
);
```

5.6.2 GetGroupTurnOver

- 含义：得到各组在每个调仓时点 T-1 期到 T 期的换手率
- 定义：GetGroupTurnOver(weight_type:Int);array
- 参数：

  weight_type: 换手率计算方法

取值	含义	备注
1	个数法换手率	默认
2	权重法换手率	

- 返回：

字段名称	含义	备注
截止日	当前调仓日	
上期截止日	T-1 个调仓日	
基准	基准当期换手率	基准指数成分算
第 1 组	各组当期换手率	
……		
第 n 组		

5.6.3 GetTrailingReturn

- 含义：因子收益率检验：各组收益率检验
- 定义：GetTrailingReturn(cy:String,if_benchmark):Array;
- 参数：

  （1） Cy：考察对象的收益率序列的周期。

取值	含义	备注
-1	得到按调仓周期分的阶段收益	
Tsl 中的周期字符串	得到按对应周期分割的收益	如：cy_month()

  （2） if_benchmark：是否返回基准组，默认 0

- 返回：

字段	类型	备注

组名	String	格式形如:第 n 组，如果 if_benchmark 为真，包含”基准”组
平均收益(%)	Numeric	得到各段时期的这些指标
标准差(%)	Numeric	
夏普比率	Numeric	
胜率(%)	Numeric	

■ 说明：

（1） 得到各个组各段时期的平均收益、标准差、夏普比率、胜率(%)，是***组与组之间的比较***。

（2） 一般而言，我们期望组与组之间收益率有显著性差异。

注：此处夏普比率不考虑无风险收益率(%)。

5.6.4 GetSignificanceStatistics

■ 含义：因子显著性检验：各组相对于基准显著性检验

■ 定义：GetSignificanceStatistics(cy:String):Array;

■ 参数：Cy：考察对象的收益率序列的周期

■ 返回：各组与基准的收益率差的显著性检验的结果。

注：

1） 下面 X_i 代表第 i 组的收益率(%)序列，Y 代表基准收益率(%)序列。

2） T-Stat、P-Value 详见显著性检验相关知识。

字段	类型	说明	备注
组名	String		格式形如:第 n 组
超额均值(%)	Numeric		得到各段时期的这些指标
跟踪误差(%)	Numeric		
信息比率	Numeric		
T-Stat	Numeric	$X=X_i-Y$ T-Stat 是指序列 X 的 T 统计量。	
P-Value	Numeric	T-Stat 分位数对应的累计函数的 y 值。越大超越改组基准越多。这个值代表改组超越基准的显著性概率为这么多。	
超额概率(%)	Numeric		

■ 说明：

■ 范例：参见 TSFL_TSMutiFactor_01。

■

5.6.5 GetLongShortStatistics

■ 含义：因子区分度检验：多空组合收益率差

- 定义：GetLongShortStatistics(cy:String):Array;
- 参数：Cy：考察对象的收益率序列的周期
- 返回：

字段	说明	备注
项目	下面是固定的字符串”第一组 vs 最后一组”。	
平均收益(%)		考察对象为第一组收益率(%)-最后一组收益率(%)，各段时间的这些量
标准差(%)		
胜率(%)		
夏普比率		
T-Stat		
P-Value		

- 说明：是第一组与最后一组的比较
- 范例：参见 TSFL_TSMutiFactor_01。

5.6.6 GetInformationCoefficient

- 含义：因子延续性检验：得到因子延续性评价指标 IC
- 定义：GetInformationCoefficient(retType:Int):Array;
- 参数：

  retType：控制返回格式

取值	说明	备注
0	返回详情【默认】	
1	返回简单统计	
2	返回详情统计	
3	返回 retType=0 和 2 的结果	

- 说明：

（1）下期收益对于开始日期和截止日期的定义是：开始日期是当期调仓日+1，截止日期是下期调仓日。下期收益就是指开始日期到截止日期的涨幅(%)。

（2）在执行本接口之前，通过如下方式选择具体的 ic 类型，

```
//可选 ic 类型，0、norm ic，默认；1、Rankic ，2、kendall 秩相关
//3、行业调整 IC（等权）；4、行业调整 IC（流通市值加权）；5、加权 IC（基准权重加权）
obj.config_Set('IC 类型', 1);
```

- 返回：本期综合因子得分与下期收益的的相关系数的数组。

  retType=0 时，返回 ic 统计所使用的原始详情数据

字段	类型	备注
因子截止日	DateTime	计算因子及分组日

检验开始日	DateTime	下期开始日
检验截止日	DateTime	下期截止日
IC	NumBeric	IC
P-value	NumBeric	原假设 IC=0 时的 P 值

retType=1 时，返回简单统计

字段名称	类型	备注
开始日	DateTime	
截止日	DateTime	
IC	NumBeric	
IC_Std	NumBeric	
IC_IR	NumBeric	
显著比率(%)	NumBeric	
样本数	Int	
备注	String	

retType=2 时，返回详情统计

字段名称	类型	备注
开始日	DateTime	
截止日	DateTime	
均值	NumBeric	
绝对值均值	NumBeric	
标准差	NumBeric	
ICIR	NumBeric	
T 值	NumBeric	
P-value	NumBeric	
胜率(%)	NumBeric	
同向比例(%)	NumBeric	
状态转换比率(%)	NumBeric	
显著比例(%)	NumBeric	
偏度	NumBeric	
峰度	NumBeric	
备注	String	

retType=3 时，返回详情(retType=0)和详情统计(retType=2)

索引名称	类型	备注
详情	Array	retType=0 的返回
统计	Array	retType=2 的返回

5.6.7 GetInformationCoefficientbyICType

- 含义：因子延续性检验：根据指定的 IC 类型计算因子延续性评价指标 IC，同 GetInformationCoefficient，增加 ICtype 参数
- 定义：GetInformationCoefficientbyICType(ICType):Array;
- 参数：

（1）ICType：整数类型，IC 类型

取值	含义	备注
0	NormIC	默认
1	RankIC	
200	行业调整 IC（等权）	
210	行业调整 IC（流通市值加权）	
20	加权 IC（基准权重加权）	

- 返回：本期综合因子得分与下期收益的的相关系数的数组。
- 说明：

（1）下期收益对于开始日期和截止日期的定义是：开始日期是当期调仓日+1，截止日期是下期调仓日。下期收益就是指开始日期到截止日期的涨幅(%)。

- 返回：

字段	类型	备注
因子截止日	DateTime	计算因子及分组日
检验开始日	DateTime	下期开始日
检验截止日	DateTime	下期截止日
IC	Numberic	IC
P-value	Numberic	原假设 IC=0 时的 P 值

5.6.8 GetSignalStability

- 含义：因子信号稳定性检验，得到因子信号稳定性评价的指标
- 定义：GetSignalStability():Array
- 参数：无
- 说明：描述 T-1 期到 T 期的信号稳定性
- 返回：

字段	类型	备注

截止日	DateTime	T 期截止日
上期截止日	DateTime	T-1 期截止日
间隔期数	Int	
自相关系数	Numberic	
买入衰减(%)	Numberic	
买入反转(%)	Numberic	
卖出衰减(%)	Numberic	
卖出反转(%)	Numberic	

5.6.9 GetLinearEvaluationResult

- 含义：得到各因子标准化后得分和下期收益的最小二乘参数估计(带常数项)结果。
- 定义：GetLinearEvaluationResult (ShowType:Int):Array;
- 参数：ShowType:返回类型。

取值	含义	备注
0 或者空	拿*每期*的数据做最小二乘参数估计(带常数项)	
1	拿*所有期*的数据一起做最小二乘参数估计（带常数项）	每期变因子法不能使用该种方法。

- 返回：

当参数 showType 取 0 或空时，

字段	含义	备注
截止日	每期的调仓日	
回归系数@因子名称	各期对应因子的回归系数	如：回归系数@总市值
回归系数@常数项	回归系数中的常数项	
推荐因子配置比例(%)@因子名称	由回归结果得到的推荐因子配置比例(%)	如：推荐因子配置比例(%)@近 1 月涨幅(%)

当参数 showType 取 1 时，

字段	含义	备注
项目	这时候的最小二乘参数估计是用所有期数据一起做的。	字符串“整体“。
回归系数@因子名称	对应因子的回归系数	如：回归系数@总市值
回归系数@常数项	回归系数中的常数项	
推荐因子配置比例(%)@因子名称	由回归结果得到的推荐因子配置比例(%)	如：推荐因子配置比例(%)@近 1 月涨幅(%)

- 说明：详见因子贡献度检验 1。

- 范例：参见 TSFL_TSMutiFactor_01。

5.6.10GetPrincipalComponentResult

- 含义：得到主成分分析结果。
- 定义：GetPrincipalComponentResult (Fep:Float,ShowType:Int):Array;
- 参数：

Fep：累计贡献率阀值。当 Fep 取到整数时，代表取的主成分的个数。

取值	范例	说明
小数	0.85	代表累计贡献率的阀值（阀值的时候 100%用 1.000 表示，一定要用小数）。
整数	2	代表取的主成分的个数。

ShowType:返回类型。

取值	说明	备注
0 或者空	分别用每期的数据做主成分回归	
1	用所有期数据一起做主成分回归	

- 说明：因变量是下期收益。
- 返回：

当参数 ShowType 取 0 或者空时，

字段	说明	备注
截止日	每期的调仓日	
主成分回归系数@因子名称	对应因子的主成分回归系数	如：主成分回归系数@总市值
主成分回归系数@常数项	主成分回归测常数项	
推荐因子配置比例(%)@因子名称	由主成分回归结果得到的推荐因子配置比例(%)	如：推荐因子配置比例(%)@近 1 月涨幅(%)

当参数 ShowType 取 1 时，

字段	说明	备注
项目	这时候的主成分回归是用所有期数据一起做的。	字符串“整体“
主成分回归系数@因子名称	对应因子的主成分回归系数。	如：主成分回归系数@总市值
主成分回归系数@常数项		
推荐因子配置比例(%)@因子名称	由主成分回归结果得到的推荐因子配置比例(%)	如：推荐因子配置比例(%)@近 1 月涨幅(%)

- 说明：详见因子贡献度检验 2。

5.6.11GetDecay_InformationCoefficient

- 含义：得到因子延续性衰减分析结果

- 定义：GetDecay_InformationCoefficient(begt:Date,endt:Date,retType:Int,LagN:Int):Array
- 参数：

  Begt：调仓日筛选开始日

  Endt：调仓日筛选截止日

  retType：0、返回详情 1、返回简单统计【默认】 2、返回详情统计 3、0+2

  LagN：最大间隔期数，默认为 5，支持数组给定，例如 array(1,2,3,4,5)与给定 5 的含义一样
- 说明：参考延续性检验的说明，来选择 ic 类型
- 返回：

  retType=0 时，返回 ic 衰减统计所使用的原始详情数据

字段	类型	备注
因子截止日	DateTime	计算因子及分组日
检验开始日	DateTime	下期开始日
检验截止日	DateTime	下期截止日
间隔期数	Int	
IC	NumBeric	IC
P-value	NumBeric	原假设 IC=0 时的 P 值

retType=1 时，返回 ic 衰减统计

字段名称	类型	备注
间隔期数	Int	
IC	NumBeric	
IC_Std	NumBeric	
IC_IR	NumBeric	
显著比率(%)	NumBeric	
样本数	Int	

retType=2 时，返回详情统计

字段名称	类型	备注
间隔期数	Int	
均值	NumBeric	
绝对值均值	NumBeric	
标准差	NumBeric	
ICIR	NumBeric	
T 值	NumBeric	
P-value	NumBeric	
胜率(%)	NumBeric	

同向比例(%)	NumBeric	
状态转换比率(%)	NumBeric	
显著比例(%)	NumBeric	
偏度	NumBeric	
峰度	NumBeric	

Type=3 时，返回详情(Type=0)和详情统计(Type=2)

索引名称	类型	备注
详情	Array	retType=0 的返回
统计	Array	retType=2 的返回

5.6.12GetDecay_InformationCoefficientByICType

- 含义：得到因子延续性衰减分析结果，在 GetDecay_InformationCoefficient 的基础上增加 ICType 参数。
- 定义：
  GetDecay_InformationCoefficientByICType(begt:Date,endt:Date,retType:Int,LagN:Int,ICType:Int):Array
- 参数：其他参数参考 GetDecay_InformationCoefficient 接口
  ICType：参考 GetInformationCoefficientbyICType 接口的 ICType 参数
- 备注：其他内容参考 GetDecay_InformationCoefficient 接口

5.6.13GetDecay_SignalStability

- 含义：得到因子信号稳定性衰减分析结果
- 定义：GetDecay_SignalStability(begt:Date,endt:Date, retType:Int,LagN:Int):Array
- 参数：
  Begt：调仓日筛选开始日
  Endt：调仓日筛选截止日
  retType：0、返回详情 1、返回统计【默认】
  LagN：最大间隔期数，默认为 5
- 返回：
  retType=1，因子信号稳定性衰减分析

字段名称	类型	备注
间隔期数	Int	
自相关系数	NumBeric	
买入衰减(%)	NumBeric	
买入反转(%)	NumBeric	
卖出衰减(%)	NumBeric	

卖出反转(%)	NumBeric	
样本数	Int	

retType=0，返回衰减分析所使用的详情数据

字段名称	类型	备注
截止日	DateTime	
上期截止日	DateTime	
间隔期数	Int	
自相关系数	NumBeric	
买入衰减(%)	NumBeric	
买入反转(%)	NumBeric	
卖出衰减(%)	NumBeric	
卖出反转(%)	NumBeric	

5.6.14GetDecay_IC

- 含义：得到因子延续性衰减分析结果
- 定义：GetDecay_IC(LagN:Array,cycle:String,retType:Int):Array
- 参数：

  LagN：一维数组，间隔期数配置，计算 IC 时相隔的周期（cycle）数 ，array(1,5,10,15,20)，同样支持给定整数，例如给定 3，与 array(1,2,3)等效

  cycle： 周期，字符串，例如”日线”或者 cy_day()

  retType：0、返回详情【默认】 1、返回简单统计 2、返回详情统计 3、0+2
- 说明：参考延续性检验的说明，来选择 ic 类型
- 返回：

  retType=0 时，返回 ic 统计所使用的原始详情数据

字段	类型	备注
截止日	DateTime	计算因子及分组日
间隔 1 周期	NumBeric	LagN 配置的第一个间隔期数的 IC
…	…	...
间隔 n 周期	NumBeric	LagN 配置的第 n 个间隔期数的 IC

示例：

	截止日	1日	5日	10日	15日	20日
0	2012-01-31	-0.0200	-0.0158	0.0580	0.1646	0.1395
1	2012-02-29	0.0448	0.0376	0.0491	0.1002	0.0180
2	2012-03-30	0.1686	0.0444	0.0775	0.0049	0.0617
3	2012-04-27	0.1265	0.0401	0.1157	0.0573	-0.0055
4	2012-05-31	0.0697	-0.0019	0.0794	0.0158	-0.0315
5	2012-06-29	0.1258	0.0436	0.0036	0.0070	-0.0183
6	2012-07-31	0.1392	0.0877	-0.0700	-0.0488	-0.1222
7	2012-08-31	0.1225	0.1823	0.1340	0.0786	0.1071
8	2012-09-28	-0.0310	-0.0578	-0.0667	-0.1022	-0.1167
9	2012-10-31	0.0224	0.0424	0.0009	-0.0314	-0.1042
10	2012-11-30	-0.2066	-0.0700	-0.0996	-0.0406	-0.0657
11	2012-12-31	-0.1065	0.1329	0.1818	-0.0140	-0.0051
12	2013-01-31	0.0052	-0.0480	0.0305	0.0216	0.0696
13	2013-02-28	0.1008	0.0843	0.0241	-0.0112	0.0273
14	2013-03-29	0.0534	0.0643	0.0191	0.0455	-0.0176
15	2013-04-26	-0.1282	-0.0486	-0.0255	0.0502	0.0629
16	2013-05-31	-0.1299	-0.1923	-0.0876	-0.0593	-0.0778
17	2013-06-28	-0.1001	0.0193	0.0592	0.0252	0.0963
18	2013-07-31	0.0712	0.1071	0.0792	0.0219	-0.0058
19	2013-08-30	0.0641	0.0584	-0.0146	0.0210	0.0113
20	2013-09-30	0.0584	0.0492	0.0315	-0.0451	-0.0494
21	2013-10-31	-0.0203	0.0426	0.0202	0.0075	0.0266
22	2013-11-29	-0.0712	0.0692	0.0777	0.0481	0.0401
23	2013-12-31	-0.0099	-0.0644	-0.0771	-0.0476	-0.1033
24	2014-01-30	0.0759	0.0369	0.0391	-0.0002	0.0092
25	2014-02-28	0.0094	-0.0267	-0.0852	-0.0676	-0.1127
26	2014-03-31	0.0900	0.1256	0.0685	0.0500	-0.0453
27	2014-04-30	0.0882	0.0062	-0.0211	-0.0430	-0.0408
28	2014-05-30	-0.0937	-0.0664	-0.0438	-0.0337	-0.0267
29	2014-06-30	-0.0381	-0.0413	0.0449	-0.0274	-0.0315

retType=1 时，返回简单统计

字段名称	类型	备注
统计类别	String	根据配置个间隔期数分组
IC	NumBeric	
IC_Std	NumBeric	
IC_IR	NumBeric	
显著比率(%)	NumBeric	
样本数	Int	

retType=2 时，返回详情统计

字段名称	类型	备注
统计类别	String	根据配置个间隔期数分组
均值	NumBeric	
绝对值均值	NumBeric	
标准差	NumBeric	
ICIR	NumBeric	
T 值	NumBeric	
P-value	NumBeric	
胜率(%)	NumBeric	
同向比例(%)	NumBeric	
状态转换比率(%)	NumBeric	
显著比例(%)	NumBeric	
偏度	NumBeric	
峰度	NumBeric	

retType=3 时，返回详情(retType=0)和详情统计(retType=2)

索引名称	类型	备注
详情	Array	retType=0 的返回
统计	Array	retType=2 的返回

5.6.15GetDecay_ICByICType

- 含义：得到因子延续性衰减分析结果，在 GetDecay_IC 的基础上增加 ICType 参数。
-
- 定义：GetDecay_ICByICType(LagN:Array,cycle:String,retType:Int,ICType:Int):Array
- 参数：其他参数参考 GetDecay_IC 接口
  ICType：参考 GetInformationCoefficientbyICType 接口的 ICType 参数
- 备注：其他内容参考 GetDecay_IC 接口

5.7 外部数据获取接口

对公开资讯数据（包含市值/行业）的访问接口，支持用户重载

5.7.1 updateStockInfoData

- 含义：设置股票基本信息
- 定义：updateStockInfoData(factorRawTable, vEndt);Array;
- 参数：
  (1) factorRawTable：因子原值，主要包含代码、截止日、因子原值

(2) vEndt：调仓日

- 返回：对 factorRawTable 添加以下字段，并保持其他字段返回

字段	含义	算法	单位
行业	行业代码	由 IndustrializeData 添加	
行业名称	行业名称	同上	
_市值		流通市值	万元
_标准化用市值	标准化用市值权重，市场组合中性	市值加权	和为 1
_中性化用市值	用于行业市值回归中性化	对数市值，去极值、标准化后	正态，均值为 0
_回归加权用市值	回归加权用市值，异方差	市值平方根加权	和为 1

5.7.2 IndustrializeData

- 含义：分组和行业中性配置时个股自定义行业
- 定义：IndustrializeData(factortable:Array,vEndT:TDateTime):Array;
- 参数：

（1） factortable ：完成数据处理之后的数组，主要包含代码、截止日、因子原值、奇异值处理后的值、无量纲化后的值、综合因子得分这些。

（2） vEndT: 调仓日。

- 返回： 后续分组后资金分配需要考虑行业时，在原来 Data 的基础上加上一列”行业”的矩阵。
- 说明：**如果用户需要使用自定义行业且分组考虑行业时候，才需要重载此方法** 。即当以下情形才会重载：

（1）因子分组时，考虑行业因素（FGroupHy=1 或 FGroupHy=2）且所用的行业分类不是申万一级行业、中证一级行业、证监会一级行业（FIndustryType=-1）

（2）行业资金分配方法为行业中性（FIndustryMethod=1）且所用的行业分类不是申万一级行业、中证一级行业、证监会一级行业（FIndustryType=-1）

否则只需要设置分组方式 FGroupHy、行业分类 FIndustryType、行业资金分配方法 FIndustryMethod 接口。

5.7.3 GetBenchmarkWeight

- 含义：设置基准行业分类及个股在基准中的权重
- 定义：GetBenchmarkWeight(vEndT:TDateTime):Array;

（1）vEndT: 调仓日

- 说明：

只有当行业资金分配方法为行业中性（FIndustryMethod=1）且所用的行业分类不是申万一级行业、中证一级行业、证监会一级行业（FIndustryType=-1）才需要重载此方法

- 返回：

返回对应调仓日用户自定义基准的个股权重(%)和行业表格。必须要的三列是：“代码”,”比例(%)”,”行业”，列的顺序无关。

5.7.4 GetBenchmarkDailyReturn

- 含义：获取基准日收益率序列，只在 FBMType=1 时使用
- 定义：GetBenchmarkDailyReturn(vBegT:date,vEndT:date):Array
- 参数：

  vBegt：开始日

  vEndt：截止日
- 返回：二维数组

字段名称	类型	备注
截止日	DateTime	
基准收益率(%)	Numeric	

- 备注：**此方法可重写**。若用户想根据自己的想法来计算基准收益率序列，可在子类重写该方法

5.8 数据处理相关接口

数据相关的这几个函数在一般情况下，***只需要了解即可***。只有当你想使用你自己的方法处理数据时才需要重写这几个函数。重写的时候把握入口是什么，出口是什么。

5.8.1 因子处理单元

5.8.1.1 FillMissData

- 含义：缺失值处理
- 定义：FillMissData(factorMatrix:Array,Factors:Array,index:Array)：factorMatrix(Array);
- 参数：
  - factorMatrix: 矩阵，***未进行缺失值处理之前的数据***。每一列代表一个因子的取值，跟用户所给因子表的因子顺序相同
  - Factors:二维数组，因子表的关键信息，包含“因子名称”、“因子公式”、“因子方向”、“因子比例(%)”字段
  - Index:一维数组，数据缺失值处理后的索引。因为缺失值处理可能会删除无效数据，需要返回索引
- 返回：标准的矩阵(字段是从 0 开始的递增的数字的二维数组)，***缺失值处理之后的数据***。
- 说明：

目前，我们的多因子框架提供了三种处理方法：0、剔除，1、平均值填充，2、中位数填充。默认是剔除。通过设置 FFillMissType 来选择哪种方法。

当你想使用其他缺失值处理方法的时候，可以重写 FillMissData 函数，否则不需要对这个函数进行任何处理。参数 Data 每列是各只股票对应的因子值，重写的时候也分别对每列进行缺失值处理。

- 范例：

```
obj := CreateObject('TMyMultiFactor');

    //回测开始日&截止日
```

```
    obj.FBegT := 20210401t;
    obj.FEndT := 20210520t;

        //*****最关键的因子库：用户自设*****//
    obj.FFactorArr := array(
    ("因子名称":"净资产收益率(%)ROE","因子公式":"Last12MData(RDate,9900100)","因子方向":1.0,"因子比例(%)":40.0),
    ("因子名称":"总资产报酬率(%)ROA","因子公式":"Last12MData(RDate,9900105)","因子方向":1.0,"因子比例(%)":20.0),
    ("因子名称":"销售毛利率(%)","因子公式":"Last12MData(RDate,9900103)","因子方向":1.0,"因子比例(%)":40.0),
    );

        //开始回测
    obj.CalculateFactorValue();

    ret := obj.GetGroupFactorValue();

    return ret;

Type TMyMultiFactor=class(TSMultiFactor)

    function FillMissData(factorMatrix,Factors,index);override;
    begin
        index := array();
        //剔除第一个因子，即 "净资产收益率(%)ROE" 小于 10 的个券
        ret := select *,(index union= array(thisrowindex)) as nil from factorMatrix where [0]>=10 end;
        return ret;
    end
end
```

5.8.1.2 NormalizeData

- 含义：极值处理
- 定义：NormalizeData(factorMatrix:Array, Factors:Array)：factorMatrix(Array);
- 参数：
  - factorMatrix: 矩阵，***未进行极值处理之前的数据***。每一列代表一个因子的取值，跟用户所给因子表的因子顺序相同
  - Factors:二维数组，当期因子配置
- 返回：标准的矩阵(字段是从 0 开始的递增的数字的二维数组)，***极值处理之后的数据***。
- 说明：

目前，我们的多因子框架提供了两种极值处理方法：1、中位数法，2、三倍标准差法。默认提供的是中位数法。通过设置 FAbnormalType 来选择哪种方法。

当你想使用其他极值处理方法的时候，可以重写 NormalizeData 函数，否则不需要对这个函数进行任何处理。参数 Data 每列是各只股票对应的因子值，字段名就是因子配置那里的因子名称，你重写的时候也分别对每列进行极值处理。

我们推荐用户使用我们提供的两种极值处理方法：中位数法和三倍标准差法，因为用户随便想的极值处理方法未必有着两种方法好。

为什么要进行极值处理，为什么使用中位数法、三倍标准差法。详情参见：极值处理方法。

■ 范例：

```
obj:=CreateObject('MutiFactorclass');
   obj.FBegT:=20131231T;
   obj.FEndT:=20140530T;
   obj.FFactorArr:=
array(
("因子名称":"收盘价","因子公式":"close()","因子比例(%)":100,"是否缺失值处理
":1.0,"缺失值处理方法":1.0,"是否去奇异值":1.0,"极值处理方法":0.0,"是否标准化
":1.0,"标准化处理方法":1.0,"是否中性化处理":1.0,"中性化处理方法":1.0),
 );
   obj.CalculateFactorValue();
return obj.GetGroupFactorValue();

Type MutiFactorclass=class(TSMultiFactor)
//重载极值处理方法
//极值处理方法改为：指标值大于 3 那么将这个值置为 3，如果指标值小于-3 置为-3
   Function NormalizeData(factorMatrix,Factors):factorMatrix;override;
   Begin
      cols:=mcols(factorMatrix,1);
      ret:=array();
      for i:=0 to length(cols)-1 do
      Begin
         tempArr:= factorMatrix[:,cols[i]];
         for j:=0 to length(tempArr)-1 do
         Begin
            if tempArr[j]>3 then tempArr[j]:=3;
            else if tempArr[j]<-3 then tempArr[j]:=-3;
            else continue;
         End
         ret |=`tempArr;
      End
      return ret;
   End
END
```

5.8.1.3 StandarizeData

- 含义：标准化处理
- 定义：StandarizeData(factorMatrix:Array, Factors:Array): factorMatrix(Array);
- 参数：

（1） factorMatrix:矩阵，极值处理之后的矩阵。接下来是对这个矩阵进行标准化处理。

（2） Factors：当期因子配置。

- 返回：标准的矩阵(字段是从 0 开始的递增的数字的二维数组)，标准化之后的矩阵。
- 说明：

1）该过程可重写

2）重写的时候不要改变参数的值。

3）对于***反向因子***的标准化处理过程，需要做一个绕均值的反转变换(2*E(X)-X)。比如，对数组进行极差正规化（将数据平移伸缩映射到[0,1]），如果是反向因子就还要做如下变换 2*0.5-Data[:,nI];

5.8.1.4 NeutralizeData

- 含义：中性化处理
- 定义：NeutralizeData (factorTable:Array, Factors:Array)：factorTable (Array);
- 参数：
  - factorTable: 二维数组，***未进行中性化处理之前的数据***。字段名包含“代码”、“名称”、“截止日”、“报告期”和因子名称。
  - Factors:二维数组，当期因子配置
- 返回：***中性化处理之后的数据***。字段名包含“代码”、“名称”、“截止日”、“报告期”和因子名称
- 说明：

目前，我们的多因子框架提供了三种可选的中性化方法：1、行业中性化，2、对数市值中性化。默认是 0，不进行中性化。通过设置 FNeuType 来选择哪种方法。具体设置方法参考 FNeuType 的说明。其中选择的中性化方法包含 1 时，可以通过设置 FindustryEval 来修改行业获取表达式，具体参考 FindustryEval。

当你想使用其他中性化处理方法的时候，可以重写 NeutralizeData 函数，否则不需要对这个函数进行任何处理。

- 范例：

```
obj := CreateObject('TMyMultiFactor');

        //回测开始日&截止日
obj.FBegT := 20210401t;
obj.FEndT := 20210520t;

        //*****最关键的因子库：用户自设*****//
obj.FFactorArr := array(
```

```
   ("因子名称":"净资产收益率(%)ROE","因子公式":"Last12MData(RDate,9900100)","因子方向":1.0,"因子比例(%)":40.0),
   ("因子名称":"总资产报酬率(%)ROA","因子公式":"Last12MData(RDate,9900105)","因子方向":1.0,"因子比例(%)":20.0),
   ("因子名称":"销售毛利率(%)","因子公式":"Last12MData(RDate,9900103)","因子方向":1.0,"因子比例(%)":40.0),
   );

      //开始回测
obj.CalculateFactorValue();

ret := obj.GetGroupFactorValue();

return ret;

Type TMyMultiFactor=class(TSMultiFactor)

   function NeutralizeData(factorTable,Factors);override;
   begin
      // 获取股票代码
      stockarr := factorTable[:,'代码'];

      // 中性化的数据
      neu_factorTable := stockarr;
      endt := factorTable[0,'截止日'];
      with *,array(PN_Date():endt) do
         neu_factorTable ::= spec(MF_Industry(5,1),mcell);
      neu_factorTable := get_dummies2(neu_factorTable);
      neu_factorTable := reindex2(neu_factorTable);

      // 中性化
      factor_field := mCols(factorTable,1)[4:];  //因子值字段
      factor_factorTable := factorTable[:,factor_field];  //因子值数据
      resid := array();
      for i,v in `factor_factorTable do
      begin
         Regress_CMLS(v,neu_factorTable,u);
         resid |= u;
      end

      ret := factorTable;
      ret[:,factor_field] := resid;
      return ret;
```

```
    end
end
```

5.8.1.5 OrthData

- 含义：正交化处理
- 定义：OrthData (factorTable:Array)：factorTable (Array);
- 参数：
  - factorTable: 二维数组，***未进行正交化处理之前的数据***。字段名包含“代码”、“名称”、“截止日”、“报告期”和因子名称。
- 返回：***正交化处理之后的数据***。字段名包含“代码”、“名称”、“截止日”、“报告期”和因子名称
- 说明：

目前，我们的多因子框架提供了三种正交化处理方法：1、对称正交化，2、规范正交化，3、施密特正交化。默认是 0，不进行正交化。通过设置 FOrthType 来选择哪种方法。

当你想使用其他正交化处理方法的时候，可以重写 OrthData 函数，否则不需要对这个函数进行任何处理。。

- 范例：

```
obj := CreateObject('TMyMultiFactor');

      //回测开始日&截止日
obj.FBegT := 20210401t;
obj.FEndT := 20210520t;

      //*****最关键的因子库：用户自设*****//
obj.FFactorArr := array(
   ("因子名称":"净资产收益率(%)ROE","因子公式":"Last12MData(RDate,9900100)","因子方向":1.0,"因子比例(%)":40.0),
   ("因子名称":"总资产报酬率(%)ROA","因子公式":"Last12MData(RDate,9900105)","因子方向":1.0,"因子比例(%)":20.0),
   ("因子名称":"销售毛利率(%)","因子公式":"Last12MData(RDate,9900103)","因子方向":1.0,"因子比例(%)":40.0),
   );

      //开始回测
obj.CalculateFactorValue();

ret := obj.GetGroupFactorValue();

return ret;

Type TMyMultiFactor=class(TSMultiFactor)
```

```
    function OrthData(factorTable);override;
    begin
        factor_field := mCols(factorTable,1)[4:];  //因子值字段
        factor_data := factorTable[:,factor_field];  //因子值数据
        factor_data := reindex2(factor_data);
        factor_data := SymOrthogonal(factor_data,0);    //对称正交

        ret := factorTable;
        ret[:,factor_field] := factor_data;
        return ret;
    end
end
```

5.8.1.6 OrthDataCore

- 含义：正交化处理
- 定义：OrthDataCore (factorTable:Array,Factors:Array)：factorTable (Array);
- 参数：
  - factorTable: 二维数组，***未进行正交化处理之前的数据***。字段名包含“代码”、“名称”、“截止日”、“报告期”和因子名称。
  - Factors：二维数组，因子表
- 返回：***正交化处理之后的数据***。字段名包含“代码”、“名称”、“截止日”、“报告期”和因子名称
- 说明：

目前，我们的多因子框架提供了三种正交化处理方法：1、对称正交化，2、规范正交化，3、施密特正交化。默认是 0，不进行正交化。通过设置 FOrthType 来选择哪种方法。

当你想使用其他正交化处理方法的时候，可以重写 OrthData1 函数，否则不需要对这个函数进行任何处理。

OrthData ***是历史的接口，如果用户历史代码已经重写该函数，会导致无法使用分组正交功能，需要重新重写新的正交化处理函数 OrthDataCore***。

- 范例：

```
obj := CreateObject('TMyMultiFactor');

    //回测开始日&截止日
obj.FBegT := 20210401t;
obj.FEndT := 20210520t;

    //*****最关键的因子库：用户自设*****//
obj.FFactorArr := array(
    ("因子名称":"净资产收益率(%)ROE","因子公式":"Last12MData(RDate,9900100)","
因子方向":1.0,"因子比例(%)":40.0),
```

```
    ("因子名称":"总资产报酬率(%)ROA","因子公式":"Last12MData(RDate,9900105)","因子方向":1.0,"因子比例(%)":20.0),
    ("因子名称":"销售毛利率(%)","因子公式":"Last12MData(RDate,9900103)","因子方向":1.0,"因子比例(%)":40.0),
    );

        //开始回测
    obj.CalculateFactorValue();

    ret := obj.GetGroupFactorValue();

    return ret;

    Type TMyMultiFactor=class(TSMultiFactor)

        function OrthDataCore(factorTable,Factors);override;
        begin
            factor_field := mCols(factorTable,1)[4:];  //因子值字段
            factor_data := factorTable[:,factor_field];  //因子值数据
            factor_data := reindex2(factor_data);
            factor_data := SymOrthogonal(factor_data,0);    //对称正交

            ret := factorTable;
            ret[:,factor_field] := factor_data;
            return ret;
        end
end
```

5.8.1.7 ScoreData

- 含义：多因子打分处理,即得到综合因子打分。
- 定义：ScoreData(factorMatrix:Array,MyFactorArr:Array): OnefactorMatrix (Array);
- 参数：

（1） factorMatrix：标准化处理之后的矩阵。

（2） MyFactorArr：当期因子配置。

- 返回：一列的矩阵，当期综合因子得分。
- 说明：可重写，但一般不需要重写。

5.8.2 分组配权单元

5.8.2.1 GroupData

- 含义：因子分组处理
- 定义：GroupData(Data:Array,vEndT:TDateTime):Array;

- 参数：

（1） Data：多因子打分处理之后的矩阵，主要用到”代码”、”截止日”、”综合因子”等字段。

（2） VEndT：

- 返回：矩阵，当期分组后的结果。在 data 的基础上增加”组名”字段。
- 说明：可重写，但**一般不需要重写**

5.8.2.2 AllocateWeight

- 含义：个股资金分配处理
- 定义：AllocateWeight(Data:Array,vEndT:DateTime):Array;
- 参数：

（1） Data：因子分组之后的所有组。

（2） VEndT ：调仓日。

- 返回：矩阵，当期资金分配后的结果，即在数组 Data 的基础上新加一列”比例(%)”,对应到某期某个股票的资金分配比例(%)。
- 说明：可重写，但**一般不需要重写。**

5.8.2.3 AllocateWeightByGroup

- 含义：个股资金分配处理(针对某组的重写)
- 定义：AllocateWeightByGroup(Data:Array,vEndT:DateTime):Array;
- 参数：

（1） Data：因子分组之后的某组成分，为一维数组。

（2） VEndT ：调仓日。

- 返回：T 期指定组的资金配比 ，必须含”代码”和”比例”字段。
- 注意：重写该方法时，FRateType 应设置为-2
- 说明：注意此方法与 AllocateWeight 的区别，AllocateWeight 是要对所有组数挨个个券比例分配，过程如果需要行业中性还需要自己写行业中性的算法，过程繁杂。而 AllocateWeightByGroup 针对的是每组的重写，参数 Data 是每组的数据，因此用户只关注每组的个股资金分配即可，也不必考虑行业中性，可以说 AllocateWeightByGroup 是 AllocateWeight 一个子集，重写了 AllocateWeightByGroup 就不必重写 AllocateWeight，反之亦然。

5.9 评价方法时间段显示相关接口

5.9.1 GetShowPeriod

- 含义：因子评价时，评价日期分段序列
- 定义：GetShowPeriod():Array;
- 参数：无
- 返回：返回一个表结构，具体如下。

字段名	类型	含义	备注
名称	String	名称	eg:近一年

开始日	DateTime	开始时间	比如近一年的开始时间是 incyear(FEndT,-1)+1。
截止日	DateTime	截止时间	比如近一年的截止时间是 FendT。

- 说明：

（1） 用于控制 GetTrailingReturn、GetSignificanceStatistics 和 GetLongShortStatistics 几个评价方法中的时间显示。

（2） 对于几种评价方法，我们默认是显示近 1 年、近 2 年、近 3 年以及整体的一个时间划分的结果。如果用户需要订制个性化的时间显示区间。可以通过重写 PeriodDivid 来实现

- 范例：

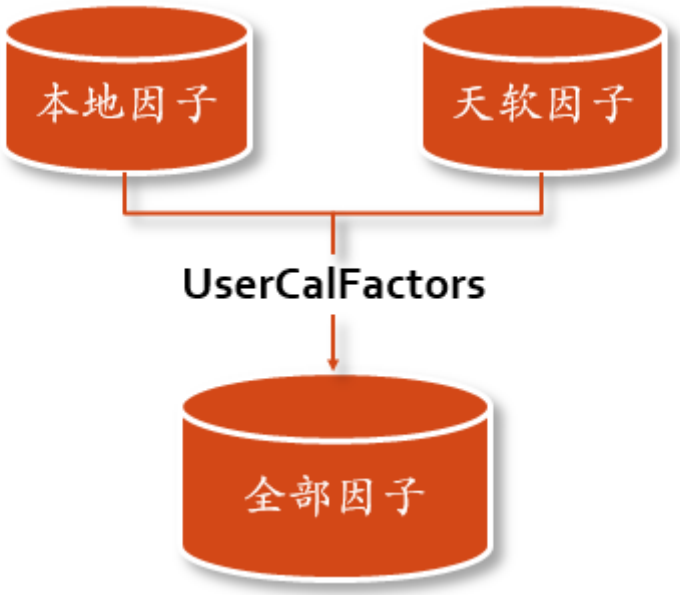
```
FactorArr:=array(
("因子名称":"每股收益","因子公式":"Last12MData(RDate,9900000)","因子方向":1,"因子比例(%)":60),
("因子名称":"收盘","因子公式":"StockClose(EndT)","因子方向":0,"因子比例(%)":40));
obj:=CreateObject('MutiFactorclass');
obj.FBegT:=20101231T;
obj.FEndT:=20140530T;
obj.FFactorArr:=FactorArr;
obj.BackTest();
return array(
'因子收益率':obj.GetTrailingReturn(cy_month()),
'因子显著性检验':obj.GetSignificanceStatistics(cy_month()),
'因子区分度检验':obj.GetLongShortStatistics(cy_month()),
);

Type MutiFactorclass=class(TSMultiFactor)
   Function GetShowPeriod();override;
   begin
      t1:=incyear(FEndT,-1)+1;
      t2:=incyear(FEndT,-2)+1;
      t3:=incyear(FEndT,-3)+1;
      r:=array(
               ("名称":"一年前","开始日":t1,"截止日":FEndT),
               ("名称":"两年前","开始日":t2,"截止日":t1-1),
               ("名称":"三年前","开始日":t3,"截止日":t2-1),
               );
      return r;
   end
end
```

5.10如何在因子库中添加自己已经计算好的因子(本地因子)

研究多因子时，有些因子值并非要在执行的时候才计算获取，因为如果有的因子值计算量大时间长，为了避免每次运行时都要花大量时间计算，可以先一次性把该因子的全市场值全部取下来存在本地上，执行时可以直接与本地交互获取该因子值，避免了每次运行都要话费大量时间运算。

除了我们预先从 TS 预先计算好的因子值外，有些因子还可能通过第三方数据源获取，例如本地数据库、或者通过与 matlab、Python 与第三方应用交互而得的因子值，我们统称为本地因子。本地因子值是存放在本地的，多因子框架使用了 ***UserCalFactors*** 方法获取本地因子值，取到本地因子再与天软因子两表做一个 JION 得到全部因子，由于系统无法得知用户的本地因子来源，因此本步骤需要用户去重写 UserCallFactor 方法完成。



5.10.1UserCalFactors

- 含义：用户重写这个函数，把每期的自己计算因子连接到参数 Data 上。
- 定义：UserCalFactors(Data,EndT):Array;
- 参数：

（1） Data：二维数组，天软因子值表。此参数传进来作用有两点：1、为取本地因子得到样本；2、与本地因子值合并并返回。

（2） EndT：日期。

- 返回：返回一个字符串作为自定义函数的标识，在解析公式的时候直接跳过。
- 说明：

5.10.2UserCalFactors 范例:

```
Function  UserCalFactors_Example();
  obj:=CreateObject('TMyMultiFactor');
  obj. FFactorArr :=
  array(
```

```
    ("因子名称":"净资产收益率(%)","因子公式":"Last12MData(RDate,9900100)","因子方向":1.00,"因子比例(%)":30.00),
    ("因子名称":"本地因子 1","因子公式":"TSMF_UserDefinedValue()","因子方向":1.00,"因子比例(%)":30.00),
    ("因子名称":"本地因子 2","因子公式":"TSMF_UserDefinedValue()","因子方向":1.00,"因子比例(%)":20.00));
          //设置用户自定义因子值
     rdo2 ImportFile(ftxls(),'',dir,r);
    obj.userFactors := r;
     obj.BackTest();
          //本范例重点在如何取本地因子值，因此其他属性不多设置，按系统默认。
End

Type TMyMultiFactor=class(TSMultiFactor)
    userFactors;
    Function UserCalFactors(Data,EndT);override;//
    begin
          r:=select ["代码"],["本地因子 1"],["本地因子 2"] from userFactors where ['截止日']=EndT end;
          if istable(Data) then
          r:= select [1].*,[2].* from Data join r with ([1].['代码'] on [2].['代码']) end;
          return r;
     end
end
```

以上范例并非可以返回结果，只是提供范例模板，需要注意的几点：

- 本地因子的因子公式一律命名为：TSMF_UserDefinedValue()，方便计算天软因子时识别。并且一定写在因子列表的最后面。
- 系统先计算完天软因子，在取本地因子，因此取本地因子之前，样本已经固定了，取本地因子时可从 Date 得到 T 期样本。
- 效率方面：如果本地因子值库是很大的数据，而且并非所有的因子值都被利用了，可以考虑使用 SQL 方式获取每 T 期与本地交互获取因子值。如果数据量比较小，可以考虑一次性导入本地因子值。

6 进阶用法

6.1 因子计算单元

6.1.1 因子的附加属性

除了因子的基本属性以外，因子还可有相应的附加属性：是否缺失值处理、缺失值处理方法、是否去奇异值、极值处理方法、是否标准化、标准化处理方法、是否中性化、中性化处理方法，通过对以上附加属性进行设置，可指定因子按照不同的数据处理方法进行相关数据处理，对应字段说明如下：

属性	类型	说明	备注
是否缺失值处理	Int	1 表示做缺失值处理 0 表示不做缺失值处理	缺失值处理方法启用开关，0 表示不启用，即不做缺失值处理
缺失值处理方法	String/Integer	-1/"off"：不处理 default：根据属性参数 FFillMissType 对应值处理 0：剔除 1：平均值填充 2：中位数填充	方法选项参数小于-1 当前均认为是不处理，[-100,-1)为天软预留编码段，如果用户有自己的缺失值处理方法，建议方法选项参数编码小于-100
是否去奇异值	Int	1 表示去奇异值 0 表示不去奇异值	极值处理方法启用开关，0 表示不启用，即不做极值处理
极值处理方法	String/Integer	-1/"off"：不处理 default：根据属性参数 FAbnormalType 对应值处理 0：中位数法 1：3 倍标准差法 2：四分位法 3：十分位法 4：鲁棒标准差法 200：行业内中位数法 201：行业内 3 倍标准差法 202：行业内四分位法 203：行业内十分位法 204：行业内鲁棒标准差法	方法选项参数小于-1 当前均认为是不处理，[-100,-1)为天软预留编码段，如果用户有自己的极值处理方法，建议方法选项参数编码小于-100
是否标准化	Int	1 表示标准化 0 表示不进行标准化	标准化处理方法启用开关，0 表示不启用，即不做标准化处理
标准化处理方法	String/Integer	-1/0/"off"：不处理 default：根据属性参数 FStandarizeType 对应值处理 1：zscore 标准化 2：缩放数据到[-1,1] 3：缩放数据到[0,1]	方法选项参数小于-1 当前均认为是不处理，[-100,-1)为天软预留编码段，如果用户有自己的标准化处理方法，建议方法选项参数编码小于-100

		4：百分比打分法 201：行业内 zscore 标准化 202：行业内缩放数据到[-1,1] 203：行业内缩放数据到[0,1] 204：行业内百分比打分法	
是否中性化	Int	1 表示中性化 0 表示不进行中性化	中性化处理方法启用开关，0 表示不启用，即不做中性化处理
中性化处理方法	String/Integer	-1/0/"off"：不处理 default：根据属性参数 FNeuType 对应值处理 1：申万一级行业中性化 2：（对数）市值中性化 "##行业"：行业中性化 "##市值"：市值中性化 因子名称：可指定使用某个表内的因子进行中心化 以上字符串类输入中，##行业、##市值、因子名称可自由组合，中间以;隔开即可，例如：“##行业;##市值”	方法选项参数小于-1 当前均认为是不处理，[-100,-1)为天软预留编码段，如果用户有自己的中性化处理方法，建议方法选项参数编码小于-100
正交化组别	Int	用于分组，相同的值表示在同一组	
正交化处理方法	String/Integer	-1/0/"off"：不处理 default：根据属性参数 FOrthType 对应值处理 其他取值参考 FOrthType	方法选项参数小于-1 当前均认为是不处理，[-100,-1)为天软预留编码段，如果用户有自己的中性化处理方法，建议方法选项参数编码小于-100

6.1.1.1 场景一：一套处理方法，指定某个因子是否处理

- 设置说明：因子属性表中只有【是否 XXX】类方法启用开关字段
- 处理说明：表示对应的数据处理方法是否启用，若为 0 表示不启用该方法，则对应的属性参数设置失效；若为 1 表示启用该方法，则按照对应的属性参数设置选择对应的数据处理方法。

6.1.1.2 场景二：对不同的因子设置不同的数据处理方法

- 设置说明：因子属性表中只有【XXX 处理方法】类方法选择字段
- 处理说明：此时默认【是否 XXX】字段均为 1，然后按照【XXX 处理方法】字段对应的方法选项进行数据处理。需要注意的是，若【XXX 处理方法】输入的是“default”字符串，则对应的方法选项以各属性参数设置为准。

6.1.1.3 场景三：混用模式

- 设置说明：因子属性表中同时有【是否 XXX】和【XXX 处理方法】字段
- 处理说明：先根据方法启用开关【是否 XXX】判断对应的数据处理方法是否启用，若选择 0（不启用），则表示不进行相应的数据处理；若选择 1（启用），则再根据相应的【XXX 处理方法】中选择的方法选项进行数据处理

因子列表范例如下：

因子名称	因子公式	因子方向	因子比例(%)	是否缺失值处理	缺失值处理方法	是否去奇异值	极值处理方法	是否标准化	标准化处理方法	是否中性化	中性化处理方法
E/P	1/StockPE_VI(EndT)	1.0	25.0	1.0	0.0	1.0	1.0	1.0	1.0	0.0	1.0
B/P	1/StockPNA_VI(EndT)	1.0	25.0	1.0	0.0	1.0	1.0	1.0	1.0	0.0	1.0
SR/P	1/StockPMI_VI(EndT)	1.0	25.0	1.0	0.0	1.0	1.0	1.0	1.0	0.0	1.0
CF/P	1/StockPCF_VI(EndT)	1.0	25.0	1.0	0.0	1.0	1.0	1.0	1.0	0.0	1.0

在实际应用中，如果没有用到以上附加属性，可以忽略之，因子列表只需要基本属性即可。

6.1.2 因子公式自定义参数

6.1.2.1 自定义因子公式参数

有一些因子公式包含了其他的参数，可以通过类方法 config_Set 来设置自定义的因子公式的静态参数以及成员函数 GetFactorParams 来实现跟调仓时间相关的动态参数，如果说因子公式参数还跟当前股票相关等复杂需求，会介绍一些技巧来实现。

6.1.2.1.1 静态参数

如果只是一个静态的因子公式参数，直接通过 config_Set 来设置即可。例如，config_Set('因子公式参数',array('def_param1':100,'def_param2':200,'def_param3':300))，设置之后，就可以直接在因子公式中像使用日期和报告期内置参数一样使用该参数。例如因子公式为"my_factor(endt,def_param1)"，在计算时会直接使用到'def_param1'这个自定义因子公式参数。

6.1.2.1.2 动态参数

如果自定义的因子公式参数是根据调仓日期变化的，这个时候就需要用户重写 GetFactorParams 函数，来根据用户实际需要返回每一期的自定义因子公式参数值。详情可参考 GetFactorParams。

6.1.2.1.3 其他需求

如果用户自定义的因子公式参数不仅仅跟调仓日期相关，还跟证券和报告期等其他复杂的因素相关。建议可以将相关因素对因子公式参数的影响封装一个单独的处理模型，然后，在因子公式中使用表达式来表示该动态参数。

例如："my_factor(def_param1)"中，def_param1 参数跟 endt，rdate 以及 stockid 都相关，这时候其中一种方法就是建议另外定义一个用户函数，专门处理这种关系，对于多个地方用到的可以方便引用：

1. 定义了一个用户函数为"my_factor_param1(ednt,rdate,stockid)"，返回参数值
2. 然后把因子公式改为" my_factor(my_factor_param1(ednt,rdate,stockid))"

另外一种方法当然就是直接在原本的因子公式函数 my_factor 里面处理，得到对应的 def_param1 参数，这种方法适用于少量因子公式用到这种处理的时候，如果很多地方都用到同一种参数跟特定参数的函数关系，建议使用第一种方法实现。

6.1.2.2 使用网格计算时系统参数的传递

6.1.2.2.1 产生原因

为什么需要用户显式地指定需要传递的系统参数，而不是把当前所有的系统参数传递到因子公式计算环境？这样做主要是为了加快效率。

6.1.2.2.2 默认参数

使用网格时，为了加快效率，默认是不带全部系统参数的，只传递 pn_stockarr()，也就是需要计算因子值的样本池。

6.1.2.2.3 使用场景

有一些因子公式在计算时可能与当前系统参数相关，在使用网格时，也需要把这些相关的系统参数传递到网格中。例如在做因子公式参数优化时，不仅可以通过添加自定义因子公式参数的方式，还可以通过传递系统参数的方式。

假设因子公式为”my_factor()”，在写 my_factor 函数计算因子值时，使用了名字为”sys1”的系统参数，不使用网格情况下，只需要在当前环境设置系统参数”sys1”的值就可以，但是如果使用了网格，默认情况下是不传递这个系统参数的，需要用户手动配置，才可以将这个系统参数传递到网格中，使用方法见下文。

6.1.2.2.4 使用方法

通过调用类方法 config_Set('系统参数',value)来设置要传递到网格中的系统参数。其中'系统参数'是配置项名称，表示配置系统参数，value 则是配置项的值。value 取值如下：

- 整数 0，表示除了 pn_stockarr()，其他不传递
- 整数 1，表示传递全部当前系统参数
- 一维数组，***每一个元素代表当前系统中的系统参数的名字，如果系统参数中不存在对应名字的系统参数就会报错***，例如 config_Set('系统参数',array('sys1','sys2','sys3'))，表示将当前系统中的名字为'sys1'、'sys2'、'sys3'的系统参数传递到网格计算的环境中，如果'sys1'、'sys2'、'sys3'任何一个不在当前系统参数中，就会报错。

6.1.2.2.5 进阶功能

假设用户想要在进行网格计算时特殊处理一些问题，可以通过进阶功能，也就是配置额外系统参数来完成。通过调用类方法 config_Set('额外系统参数',value)来设置要额外传递到网格中的系统参数。其中'额外系统参数'是配置项名称，表示配置额外的系统参数，value 则是配置项的值。value 是一个数组。

例如 config_Set('额外系统参数',array('add_sys1':100,'add_sys2':200,'add_sys3':300))，设置了'add_sys1'，'add_sys2'，'add_sys3'三个额外的系统参数，取值分别是 100，200，300。这三个系统参数只会在网格中存在，在当前系统环境中不存在。可用于使用网格时的一些特殊控制，用户自己根据实际情况使用，一般情况下，用不到这个功能。

6.2 数据处理单元

6.2.1 在数据处理单元获取当前股票代码、名称、股票基本信息

在 FillMissData 、NormalizeData、StandarizeData、NeutralizeData、OrthData 、ScoreData 中，可以通过以下查询函数，获取当前因子数据表的更多上下文信息，方便重写。

查询函数	功能	格式说明
fvtmpdata_querystocks	获取当前股票池	一维数组，数字下标与 factorMatrix 一一对应
fvtmpdata_queryfactortable	获取当前因子数据表	字段名包含“代码”、“名称”、“截止日”、“报告期”和因子名称
fvtmpdata_queryfactors	获取当期因子配置表	
fvtmpdata_querystockinfodata	获取当前股票的基本信息	

查询范例：

```
//标准化处理
Function StandarizeData(factorMatrix,factorArr):factorMatrix;override;
begin
   debugreturn
         array("当前数据表":fvtmpdata_queryfactors(),
               "当前股票池":fvtmpdata_querystocks(),
               "当前载荷矩阵":factorMatrix,
               "当前股票基本信息":fvtmpdata_querystockinfodata(),
               "当前因子数据表":fvtmpdata_queryfactortable(),
               );

end;
```

返回：

- 当前载荷矩阵 factorMatrix：

factorMatrix 为数据下标，行标可以和 fvtmpdata_querystocks 、列标可以和 fvtmpdata_queryfactors 对应

数组数据

	0	1
0	-14.78	-4.16
1	16.55	6.03
2	10.36	3.66
3	6.99	3.16
4	10.92	3.91
5	5.15	3.09

- 当前因子数据表 factortable：

数组数据

	代码	名称	截止日	报告期	净资产收益率(%)ROE	总资产报酬率(%)ROA
0	SZ000002	万　科A	45911	20250630	-14.78	-4.16
1	SH600900	长江电力	45911	20250630	16.55	6.03
2	SZ000063	中兴通讯	45911	20250630	10.36	3.66
3	SZ000157	中联重科	45911	20250630	6.99	3.16
4	SZ000425	徐工机械	45911	20250630	10.92	3.91
5	SH600009	上海机场	45911	20250630	5.15	3.09

- 当前股票基本信息 stockinfodata：

行标可以和 factorMatrix、fvtmpdata_querystocks、factortable 对应

数组数据

	代码	行业	行业名称	_市值	_标准化用市值	_中性化用市值	_回归加权用市值
0	SZ000002	SWHY430000	申万房地产	6,636,300.947	0.001	-0.500	0.002
1	SH600900	SWHY410000	申万公用事业	67,220,318.208	0.013	2.203	0.008
2	SZ000063	SWHY730000	申万通信	17,615,672.977	0.004	0.640	0.004
3	SZ000157	SWHY640000	申万机械设备	5,196,756.762	0.001	-0.785	0.002
4	SZ000425	SWHY640000	申万机械设备	9,159,100.918	0.002	-0.124	0.003
5	SH600009	SWHY420000	申万交通运输	6,185,875.736	0.001	-0.582	0.002
6	SH600010	SWHY230000	申万钢铁	7,782,437.975	0.002	-0.314	0.003
7	SZ000568	SWHY340000	申万食品饮料	20,635,889.947	0.004	0.825	0.004
8	SH600795	SWHY410000	申万公用事业	9,256,686.304	0.002	-0.111	0.003
9	SH600015	SWHY480000	申万银行	11,248,060.732	0.002	0.116	0.003
10	SH600016	SWHY480000	申万银行	15,816,106.953	0.003	0.514	0.004

6.2.2 数据处理相关配置

通过 config_Set(配置项名称,指定配置值)设置

- 外部数据级别
  - 默认为 0，对外部数据（因子数据）进行数据处理
  - 设置为 1，则不会对外部数据进行处理。但是如果设置了正交化处理，则外部数据会参与正交化处理
- 是否存储因子处理过程
  - 默认为 1，会存储因子数据处理过程
  - 设置为 0，不存储因子数据处理过程

6.3 其他说明

6.3.1 业务类型函数

为了方便输入方法选型，汇总类型函数如下：

分类	类型函数	说明	取值
因子方向	mfe_Direction_Reverse()	因子方向-反向(越小越好)	0
	mfe_Direction_Positive()	因子方向-正向(越大越好)	1
极值处理方法	mfe_AbnormalType_Median()	极值处理方法-中位数法/MAD 法	0
	mfe_AbnormalType_ZValue()	极值处理方法-3 倍标准差法	1
	mfe_AbnormalType_Quartile()	极值处理方法-四分位处理法/箱形图法	2
	mfe_AbnormalType_Decile()	极值处理方法-十分位处理法	3
	mfe_AbnormalType_RobustStd()	极值处理方法-鲁棒标准差法	4
	mfe_AbnormalType_RMAD()	值处理方法-压缩去极值 MAD 法	5
数据无量纲化方法	mfe_STDZType_ZScore()	数据无量纲化方法-Z 值法	1
	mfe_STDZType_RangeSTDZ()	数据无量纲化方法-极差标准化->[-1,1]	2
	mfe_STDZType_RangeNormal()	数据无量纲化方法-极差正规化->[0,1]	3
	mfe_STDZType_PCTRank()	数据无量纲化方法-百分比排名打分法	4
分组方式	mfe_GroupType_Count()	分组方式-个数分组	0
	mfe_GroupType_Percentile()	分组方式-(个股数)百分位分组	1
	mfe_GroupType_Value()	分组方式-值区间分组	2
	mfe_GroupType_Weight()	分组方式-权重(占比)分组	3
行业分组法	mfe_GroupHyType_off()	行业分组法-不考虑行业因素	-1
	mfe_GroupHyType_ConsHY()	行业分组法-1-考虑行业分组	1
	mfe_GroupHyType_ConsHYES()	行业分组法-2-按个股数分组(个股数少于分组数全复制)	2

分组清算类型	mfe_BackTestType_ComplexLiquid()	分组清算类型-复杂回测	0
	mfe_BackTestType_SimpleLiquid()	分组清算类型-简单回测(比例类+矩阵式)	1
IC 类型	mfe_ICType_NormIC()	IC 类型-NormIC	0
	mfe_ICType_RankIC()	IC 类型-RankIC	1
	mfe_ICType_IndusWeightIC()	IC 类型-行业调整 IC（等权）	200
	mfe_ICType_IndusMVWeightIC()	IC 类型-行业调整 IC（流通市值加权）	210
	mfe_ICType_IndWeightIC()	IC 类型-加权 IC（基准权重加权）	20
数据处理设置	mf_const_ops_asabove()	同上	
	mf_const_ops_default()	跟随默认设置处理	
	mf_const_ops_disable()	默认-禁用	
	mf_const_ops_off()	默认-不处理	
	mf_const_ops_userdef()	用户自定义	

6.3.2 多因子回测时所做的假设

1) 以收盘价成交
2) 成交量不取整
3) 分红不再投资
4) 不自动参与配股
5) 资金分配方式，默认等权重，相关成员变量 FRateType。
6) 开仓费率(%)，默认 0.1，相关成员变量 FBuyRatio。
7) 平仓费率(%)，默认 0.1，相关成员变量 FSellRatio。

7 范例

7.1 范例一：每期固定的因子库

```
Function TSFL_TSMutiFactor_01();
Begin
   obj:=CreateObject('MutiFactorclass');
   obj.FBegT:=20131231T;
   obj.FEndT:=20140530T;
   obj.FIndexId:='SH000016';
   obj.FCycle:=cy_month();
   obj.FFactorArr:=array(
      ("因子名称":"my_pe","因子公式":"1/stockpe(EndT)","因子方向":1,"因子比例(%)":50),
```

```
        ("因子名称":"近一月涨幅(%)","因子公式":"stockzf(incmonth(endt,-1)+1,endt)","因子方向":1,"因子比例(%)":50));
    obj.BackTest();
    return array(
                '调仓日':obj.GetTimeSeries(),
                '所有调仓日因子值及分组':obj.GetGroupFactorValue(),

                '-----收益率-----':'-----------',
                '日累计收益率(%)':obj.GetGroupReturn(1,cy_day()),
                '日累计超额收益率(%)':obj.GetGroupReturn(3,cy_day()),
                '月度收益率(%)':obj.GetGroupReturn(0,cy_month()),
                '月度超额收益率(%)':obj.GetGroupReturn(2,cy_month()),
                '多空月度收益率(%)':obj.GetGroupReturn(4,cy_month()),

                '-----因子检验-----':'-----------',
                '因子收益率检验':obj.GetTrailingReturn(cy_month()),
                '因子显著性检验':obj.GetSignificanceStatistics(cy_month()),
                '因子区分度检验':obj.GetLongShortStatistics(cy_month()),
                '因子延续性检验':obj.GetInformationCoefficient(),
                '因子贡献度检验1':obj.GetLinearEvaluationResult(0),//obj.GetLinearEvaluationResult(1)
                '因子贡献度检验2':obj.GetPrincipalComponentResult(0.85,0)//obj.GetPrincipalComponentResult(0.85,0)
                );
End

Type MutiFactorclass=class(TSMultiFactor)
    Function GetSamples(EndT);override;
    begin
        r:= GetBKByDate('SH000016',EndT);
        return r;
    end;
end
```

7.2 范例二：导入外部计算好的因子

参见平台中范例 TSFL_TSMutiFactor_02 以及章节 5.72

7.3 范例三：在不同调仓日配置不同的因子或因子比例

```
Function TSFL_TSMutiFactor_03();
```

```
Begin
   obj:=CreateObject('MutiFactorclass');
   obj.FBegT:= 20121212T;
   obj.FEndT:= Today()-1;
   obj.FIndexId:='SH000016';
   obj.FCycle:=cy_month();
   obj.FFactorArr:=array(
      ("因子名称":"my_pe","因子公式":"1/stockpe(EndT)","因子方向":1,"因子比例(%)":50),
      ("因子名称":"近一月涨幅(%)","因子公式":"stockzf(incmonth(endt,-1)+1,endt)","因子方向":1,"比例(%)":50));
   obj.BackTest();
   return array(
               '调仓日':obj.GetTimeSeries(),
               '所有调仓日因子值及分组':obj.GetGroupFactorValue(),

               '-----收益率-----':'-----------',
               '日累计收益率(%)':obj.GetGroupReturn(1,cy_day()),
               '日累计超额收益率(%)':obj.GetGroupReturn(3,cy_day()),
               '月度收益率(%)':obj.GetGroupReturn(0,cy_month()),
               '月度超额收益率(%)':obj.GetGroupReturn(2,cy_month()),
               '多空月度收益率(%)':obj.GetGroupReturn(4,cy_month()),

               '-----因子检验-----':'-----------',
               '因子收益率检验':obj.GetTrailingReturn(cy_month()),
               '因子显著性检验':obj.GetSignificanceStatistics(cy_month()),
               '因子区分度检验':obj.GetLongShortStatistics(cy_month()),
               '因子延续性检验':obj.GetInformationCoefficient(),
               );
End
Type MutiFactorclass=class(TSMultiFactor)
   Function GetSamples(EndT);override;
   begin
      r:= GetBKByDate('SH000016',EndT);
      return r;
   end;
  Function GetFactors(EndT);override;
  Begin
       MidT:=(FBegT+FEndT)/2;
      //调仓日是前半部分就用 pe 因子，如果调仓日期是后半部分就用近一月涨幅因子
      //在实际使用多因子的过程中，绝对不可能这样处理
```

```
            //用户可能根据因子的评价（收益率检验、显著性检验、区分度检验、延续性检验）进行取舍因子和配置各个因子的比例
            If EndT>MidT then
            Begin
                  R:= FFactorArr[0:0];
                  R[0]['因子比例(%)']:=100;
            End
            Else Begin
                  R:= FFactorArr[1:1];
                  R[0]['因子比例(%)']:=100;
            End
            Return r;
      End;
   End
```

7.4 范例四：因子衰减分析、计算 rank ic 计算类型

参见平台中范例 TSFL_TSMutiFactor_04

7.5 范例五：可转债因子的使用范例

参见平台中范例 TSFL_TSMutiFactor_Conbond_01

7.6 范例六：缺失值处理、中性化处理、正交化处理使用范例

```
obj:=CreateObject('MutiFactorclass');
obj.FBegT:=20131231T;
obj.FEndT:=20140530T;
obj.FIndexId:='SH000016';
obj.FCycle:=cy_month();
obj.FFactorArr:=array(
   ("因子名称":"my_pe","因子公式":"1/stockpe(EndT)","因子方向":1,"因子比例(%)":50),
   ("因子名称":"近一月涨幅(%)","因子公式":"stockzf(incmonth(endt,-1)+1,endt)","因子方向":1,"因子比例(%)":50));

obj.FFillMissType := 0;    //缺失值处理方法，-1:不处理,0:剔除,1:平均值填充,2:中位数填充
obj.FNeuType := 3;          //中性化处理方法，0：不处理，1：申万一级行业中性化，2：（对数）市值中性化
                                                                  //具体使用可以叠加，例如，要进行行业中性化和市值中性化，则取 1+2=3
```

```
obj.FOrthType := 1;          //正交化处理方法，0：不处理，1：对称正交化，2：规范正交化，3：施密特正交化


obj.BackTest();
return array(
            '调仓日':obj.GetTimeSeries(),
            '所有调仓日因子值及分组':obj.GetGroupFactorValue(),

            '-----收益率-----':'-----------',
            '日累计收益率(%)':obj.GetGroupReturn(1,cy_day()),
            '日累计超额收益率(%)':obj.GetGroupReturn(3,cy_day()),
            '月度收益率(%)':obj.GetGroupReturn(0,cy_month()),
            '月度超额收益率(%)':obj.GetGroupReturn(2,cy_month()),
            '多空月度收益率(%)':obj.GetGroupReturn(4,cy_month()),

            '-----因子检验-----':'-----------',
            '因子收益率检验':obj.GetTrailingReturn(cy_month()),
            '因子显著性检验':obj.GetSignificanceStatistics(cy_month()),
            '因子区分度检验':obj.GetLongShortStatistics(cy_month()),
            '因子延续性检验':obj.GetInformationCoefficient(),
               '因子贡献度检验
1':obj.GetLinearEvaluationResult(0),//obj.GetLinearEvaluationResult(1)
            '因子贡献度检验
2':obj.GetPrincipalComponentResult(0.85,0)//obj.GetPrincipalComponentResu
lt(0.85,0)
            );

Type MutiFactorclass=class(TSMultiFactor)
   Function GetSamples(EndT);override;
   begin
      r:= GetBKByDate('SH000016',EndT);
      return r;
   end;
end
```

8 FAQ

见《应用专题-多因子框架 02：天软多因子框架 FAQ》

9 联系方式

深圳市天软科技开发有限公司(www.tinysoft.com.cn)　邮编：518034　电话：（86）755-83937039
深圳市福田区莲花街道梅岭社区新闻路 59 号深茂商业中心 19F

Email: TSMultiFactorTeam@tinysoft.com.cn